



miscdg

TRABAJO FIN DE MÁSTER

Análisis y Diseño de Sistema de Recomendación Musical Basada en Características de Audio

Realizado por
Daniel Arriaza Arriaza

Para la obtención del título de
**Máster en Ingeniería del Software: Cloud, Datos y Gestión
TI**

Dirigido por
José María Luna Romera

Convocatoria de Junio, curso 2024/25

Agradecimientos

Me gustaría agradecer a todas las personas que han sido un apoyo sustancial en el desarrollo de este trabajo.

A mi tutor, José María, por aliviar mis inquietudes respecto al mundo de la investigación y la docencia. Sus consejos y su cercanía me han ayudado muchísimo.

A mi familia, por escuchar todas mis quejas y problemas a lo largo del proyecto y del máster en general.

A mis amigos: Rafa, María y Aitor. A Rafa, por acompañarme casi desde el comienzo de la carrera hasta hoy. A María y Aitor, por convertirlos en un motivo más para ir con ilusión a clase cada día.

A vosotros, gracias por todo.

Resumen

Este trabajo se centra en investigar los sistemas de recomendación musical basados en contenido y los enfoques y técnicas actuales para representar y comparar fragmentos musicales a partir de sus características. Para ello, se ha utilizado el dataset MagnaTagATune, que contiene fragmentos de canciones anotados con etiquetas musicales, características de audio y metadatos.

El estado del arte revisado ha confirmado que, si bien los métodos clásicos de filtrado colaborativo y basado en contenido siguen siendo relevantes, las tendencias actuales incluyen el uso de embeddings generados con deep learning, sistemas híbridos, técnicas basadas en señales musicales y estrategias centradas en las emociones del usuario. Estos enfoques permiten una mayor personalización y abordan desafíos como la diversidad, la evolución de las preferencias y la escasez de datos iniciales.

A lo largo del estudio se ha analizado cómo atributos como el tempo, los MFCC, los vectores chroma, y la STFT pueden utilizarse para generar recomendaciones precisas, especialmente en escenarios donde los sistemas colaborativos tienen problemas como el “cold start”. Se han codificado vectores de alta dimensionalidad utilizando estas características y se han evaluado diversas técnicas de reducción de dimensionalidad; concretamente PCA, Autoencoders y Variational Autoencoders, para mejorar la eficiencia del cálculo de similitud entre fragmentos.

Las recomendaciones se han generado utilizando k-Nearest Neighbors sobre el espacio latente. Los resultados muestran que el uso combinado de características de audio y metadatos de los géneros y etiquetas de las canciones mejora notablemente la calidad de las recomendaciones, sobre todo cuando se utiliza un Autoencoder. Por el contrario, los Variational Autoencoders no han conseguido un rendimiento suficiente y podría deberse a la complejidad del modelo o a una configuración subóptima de los hiperparámetros.

En conclusión, el uso de Autoencoders frente a Variational Autoencoders y PCA ha resultado altamente satisfactorio para la reducción de dimensionalidad y la mejora de la calidad de las recomendaciones musicales, especialmente cuando se integran múltiples fuentes de información.

Palabras clave: Sistemas de recomendación musical, Recomendación basada en contenido, MFCC, tempo, chroma, características espectrales, Autoencoder, PCA, Variational Autoencoder

Abstract

This work focuses on the study of content-based music recommendation systems and the current approaches and techniques for representing and comparing music fragments based on their features. For this purpose, the MagnaTagATune dataset has been used, which contains annotated song fragments with musical tags, audio features, and metadata.

The reviewed state of the art confirms that, although classical content-based and collaborative filtering methods remain relevant, recent trends include the use of deep learning-generated embeddings, hybrid systems, signal-based techniques, and emotion-driven strategies. These approaches enable greater personalization and address challenges such as diversity, evolving user preferences, and the cold-start problem.

Throughout this study, we have analyzed how attributes such as tempo, MFCC, chroma vectors, and STFT can be leveraged to generate accurate recommendations, particularly in scenarios where collaborative filtering systems struggle with limited data. High-dimensional feature vectors have been encoded using these attributes, and several dimensionality reduction techniques have been evaluated; namely PCA, Autoencoders, and Variational Autoencoders, to enhance the efficiency of similarity computation between fragments.

Recommendations have been generated using the k-Nearest Neighbors algorithm in the latent space. The results show that combining audio features with metadata such as genres and tags significantly improves the quality of the recommendations, especially when using Autoencoders. In contrast, Variational Autoencoders did not achieve satisfactory performance, which may be due to model complexity or suboptimal hyperparameter configurations.

In conclusion, the use of Autoencoders, compared to Variational Autoencoders and PCA, has proven highly effective for dimensionality reduction and enhancing the quality of music recommendations, particularly when multiple sources of information are integrated.

Keywords: Music recommendation systems, Content-based recommendation, MFCC, tempo, chroma, spectral features, Autoencoder, PCA, Variational Autoencoder

Índice general

1. Introducción	1
1.1. Introducción	1
1.2. Estructura de este documento	2
2. Fundamentos teóricos	3
2.1. Introducción	3
2.2. Sistemas de recomendación	3
2.3. Características de audio	4
2.3.1. Tempo	4
2.3.2. STFT	5
2.3.3. MFCC	6
2.3.4. Chroma Coefficients	7
2.4. Inteligencia Artificial	8
2.4.1. Embedding	8
2.4.2. Machine Learning	9
2.4.3. Deep Learning	11
3. Estado del arte	15
3.1. Introducción	15
3.2. Características de audio	15
3.3. Sistemas de recomendación	16
3.3.1. Algoritmos utilizados en la literatura	17
3.3.2. Nuevos enfoques en los sistemas de recomendación	18
3.4. Conclusiones	20
4. Estudio previo	22
4.1. Introducción	22
4.2. Objetivos	22
4.3. Metodología	22
4.3.1. Documentación	23
4.3.2. Elección del dataset	23
4.3.3. Implementación	24
4.3.4. Elección de algoritmos	25
4.3.5. Procesamiento de datos	25
4.3.6. Entrenamiento y evaluación	26
4.4. Planificación	27
4.4.1. Cronograma inicial	27
4.4.2. Cronograma real	29
4.5. Presupuesto	31
4.5.1. Recursos Intelectuales	31
4.5.2. Recursos materiales	31
5. Descripción de la propuesta	32

5.1.	Introducción	32
5.2.	Propuesta	32
5.3.	Estructura del sistema de recomendación	32
5.3.1.	Arquitectura final	33
5.4.	Embeddings	35
6.	Implementación	37
6.1.	Introducción	37
6.2.	Herramientas	37
6.2.1.	Entorno de ejecución	37
6.2.2.	Preparación del entorno de desarrollo	37
6.2.3.	Instalación de dependencias	37
6.2.4.	Librerías utilizadas	38
6.3.	Preprocesamiento	38
6.4.	Visualización	40
6.5.	Modelos	48
7.	Pruebas	53
7.1.	Introducción	53
7.2.	Modelos	53
7.3.	Recomendaciones	58
7.3.1.	Distancia euclídea	60
7.3.2.	Distancia Manhattan	63
8.	Conclusiones	68
A.	Bibliografía	69

Índice de figuras

2.1. Detección de onsets. Imagen obtenida de [36].	5
2.2. Ejemplo de melodía y armonía. Imagen obtenida de [3].	7
2.3. Áreas de la Inteligencia Artificial. Imagen obtenida de [38].	8
2.4. Autoencoder. Imagen obtenida de [45].	11
2.5. Espacio latente de Autoencoder. Imagen obtenida de [45].	12
2.6. Variational Autoencoder. Imagen obtenida de [45].	13
2.7. Espacio latente del Variational Autoencoder. Imagen obtenida de [45]. .	14
3.1. Ilustración de sistemas basados en filtrado colaborativo	16
3.2. Ilustración de sistemas basados en filtrado basado en contenido	17
4.1. Metodología del proyecto	23
5.1. Arquitectura del sistema	34
6.1. Densidad de canciones por género	42
6.2. Densidad de canciones por etiqueta	43
6.3. Densidad de fragmentos por género	44
6.4. Comparación de etiquetas más y menos frecuentes	45
6.5. Coocurrencia de géneros	46
6.6. Coocurrencia de etiquetas	47
6.7. Correlación entre géneros y etiquetas	48
7.1. Varianza acumulada por número de componentes	55
7.2. Comparación de resultados: (arriba) Autoencoder, (abajo) Autoencoder full	56
7.3. Comparación de resultados: (arriba) VAE, (abajo) VAE full	56
7.4. Comparación de errores por modelo y métrica	57
7.5. Espacio latente: (izquierda) Autoencoder, (derecha) VAE	58
7.6. Comparación de recomendaciones con distancia euclídea: (arriba) PCA, (abajo) PCA full	61
7.7. Comparación de recomendaciones con distancia euclídea: (arriba) Autoencoder, (abajo) Autoencoder full	62
7.8. Comparación de recomendaciones con distancia euclídea: (arriba) VAE, (abajo) VAE full	63
7.9. Comparación de recomendaciones con distancia Manhattan: (arriba) PCA, (abajo) PCA full	65
7.10. Comparación de recomendaciones con distancia Manhattan: (arriba) Autoencoder, (abajo) Autoencoder full	66
7.11. Comparación de recomendaciones con distancia Manhattan: (arriba) VAE, (abajo) VAE full	67

Índice de extractos de código

6.1. Construcción del autoencoder	49
6.2. Construcción del Variational Autoencoder	50
6.3. Recomendación con k-NN	52

1. Introducción

1.1. Introducción

En la actualidad, el volumen de información y contenido disponible en Internet ha crecido de forma exponencial. Esta abundancia de opciones ha generado la necesidad de herramientas que ayuden a los usuarios a encontrar contenido relevante, personalizado y de interés entre la gran inmensa cantidad de datos. En este contexto, los sistemas de recomendación se han convertido en una pieza clave para mejorar la experiencia del usuario en múltiples plataformas y sectores.

Plataformas de entretenimiento como Netflix, YouTube, Amazon o Spotify utilizan sistemas de recomendación avanzados para ofrecer a sus usuarios productos, vídeos, películas o canciones adaptadas a sus gustos personales. Estos sistemas son responsables de buena parte del contenido que el usuario consume, y su eficacia tiene un impacto directo tanto en la satisfacción del usuario como en los beneficios económicos de las plataformas. De hecho, se estima que más del 80 % de las visualizaciones en Netflix provienen de su sistema de recomendación [16], y un porcentaje similar en plataformas musicales como Spotify [34].

En el ámbito musical, la recomendación personalizada es especialmente importante debido a la gran variedad de géneros, estilos, artistas y nuevas canciones que se publican constantemente. Los sistemas de recomendación no solo facilitan el descubrimiento de nueva música, sino que también contribuyen a fidelizar a los usuarios, aumentar su tiempo de permanencia en la plataforma y fomentar un mayor consumo de contenido.

Generalmente, estos sistemas utilizan enfoques de filtrado colaborativo, que se basan en las interacciones de los usuarios (como reproducciones, valoraciones o playlists compartidas), o filtrado basado en contenido, que se apoya en las características intrínsecas de los ítems recomendados (por ejemplo, el género de una canción, el tempo o sus atributos espectrales). Normalmente, se combinan ambos enfoques formando sistemas híbridos para mejorar la precisión y superar limitaciones como el problema del “cold start”, que es un problema que sucede cuando no hay suficientes datos previos de un usuario o un ítem.

Las técnicas de machine learning y deep learning han impulsado grandes avances en este campo, permitiendo desarrollar modelos más complejos y precisos que son capaces de detectar nuevos patrones y generar recomendaciones más ajustadas a los gustos y necesidades de cada usuario.

El presente trabajo se propone realizar un estudio de la situación actual de los sistemas de recomendación y las técnicas utilizadas, enfocándose en la recomendación musical y en los sistemas basados en contenido. Se revisará el estado de la literatura en este campo, y se propondrá la implementación de un sistema de recomendación que utilice características de audio para representar las canciones. Este sistema permitirá comparar diferentes estrategias de reducción de dimensionalidad, como PCA,

Autoencoders y Variational Autoencoders, y evaluará su rendimiento en un entorno experimental.

1.2. Estructura de este documento

La estructura del documento es la siguiente:

- **Introducción:** Esta sección establece el contexto del trabajo y la estructura del documento.
- **Fundamentos teóricos:** Incluye todos los fundamentos sobre sistemas de recomendación, audio e inteligencia artificial que serán necesarios saber para poder entender las secciones siguientes.
- **Estado del Arte:** Se dedica un análisis de las técnicas utilizadas en la literatura. Se detallan algunos enfoques actuales y los problemas y limitaciones identificados para implementarlos y evaluarlos.
- **Estudio previo:** Constituye la base del proyecto, donde se define la metodología usada, la planificación para todo el estudio y el presupuesto.
- **Descripción de la propuesta:** Esta sección incluye todo el diseño de la arquitectura y de las características de audio utilizadas en la literatura.
- **Implementación:** Se incluyen todas las decisiones tomadas para el preprocesamiento de los datos y la construcción de los modelos, siguiendo la arquitectura definida en la sección anterior.
- **Pruebas:** Se presenta la evaluación de los diferentes modelos y de la calidad de las recomendaciones, comparando qué técnicas dan mejores resultados.
- **Conclusiones:** La sección final sintetiza los puntos clave del proyecto, ofreciendo unas reflexiones y puntos de mejora.

2. Fundamentos teóricos

2.1. Introducción

En esta sección explicaremos los principales conceptos teóricos utilizados a lo largo de todo el desarrollo, tanto teoría sobre sistemas de recomendación, como características de audio y técnicas de machine learning y deep learning.

2.2. Sistemas de recomendación

Los sistemas de recomendación han sido un área que ha ido evolucionando desde el primer paper sobre filtrado colaborativo publicado en la década de 1990 [39]. Son muy utilizados en diversas áreas, aunque nos vamos a centrar en el ámbito musical.

La búsqueda de música ha sido muy beneficiada por la expansión de Internet, pues ha permitido que gigantescas cantidades de canciones estén disponibles para la gente. Sumando el hecho de que los usuarios buscan contenidos según sus gustos personales, los sistemas de recomendación se vuelven más necesarios [4].

Como su nombre indica, un sistema de recomendación de música (Music Recommendation System o MRS en inglés) es un sistema capaz de proveer recomendaciones musicales teniendo en cuenta las preferencias del usuario, de un grupo de usuarios, etc. Los principales enfoques utilizados son tres [4]:

- **Sistemas Basados en Contenido:** Son aquellos que, en base a las características de las canciones y al historial del usuario, intenta predecir qué quiere escuchar y qué canciones similares mostrar.
- **Filtrado Colaborativo:** Permite que las interacciones de los usuarios y los datos generados por estos ayuden a incrementar la calidad de las recomendaciones [14]. Básicamente recopila preferencias o gustos del usuario y los compara con los datos de personas con patrones similares para recomendar aquellas canciones que le gusten a esos usuarios.
- **Filtrado Híbrido:** Este combina los dos enfoques anteriores, teniendo en cuenta tanto la similitud entre canciones como la similitud entre usuarios. La ventaja de este sistema es que elimina el punto más negativo del filtrado colaborativo: que se discriminan aquellas canciones que no tengan interacciones (cold start). Es decir, al utilizar ambos sistemas se puede evitar que solo se recomienden canciones populares.

Un ejemplo de sistema híbrido sería Spotify, que utiliza el historial de los usuarios (filtrado colaborativo), como sus artistas favoritos, canciones más escuchadas, e incluso el momento del día en que escucha ciertos géneros para recomendar canciones en secciones como el “Descubrimiento Semanal” o el “Radar de Novedades” [30]. Además,

analiza características de las canciones como la letra o características extraídas del audio (basado en contenido) para poder hacer recomendaciones cuando hay poca información del usuario (cold start) o para generar playlists temáticas (por género, mood, etc.) [41].

Una vez entendidos los principales enfoques, vamos a centrarnos en los sistemas basados en contenido, por lo que tenemos que hablar de las características de audio.

2.3. Características de audio

Dado que las señales de audio son complejas, se emplean diversas técnicas de extracción de características que permiten describir propiedades como el ritmo, la armonía o la energía.

Por una parte, tendríamos características que miden el ritmo, como el tempo. Además, algunas miden la frecuencia, como la STFT, o el timbre, como los MFCC. El timbre nos permite distinguir dos sonidos de igual tono (frecuencia) e igual intensidad (presión) cuando son emitidos por dos orígenes diferentes. Por ejemplo, dos voces cantando la misma nota [54].

Finalmente, hay otras que miden el pitch, como los vectores chroma. El pitch es la percepción subjetiva de la frecuencia de un sonido, es decir, lo agudo o grave que percibimos el sonido. Es un término complejo de entender debido a que dos notas en dos instrumentos diferentes pueden tener la misma frecuencia fundamental, por ejemplo, un *La* a 440 Hz, pero percibirse con diferente timbre [58].

A continuación, pasaremos a ver un poco más en profundidad las diferentes características nombradas en esta sección.

2.3.1. Tempo

El tempo indica la velocidad de la canción, es decir, cuántos pulsos suceden por minuto (BPM). Se suele representar mediante una figura determinada (como una negra o una corchea) y el número de veces que esa figura cabe en un minuto. Por ejemplo, $\text{♩} = 60$ indica que en un minuto caben 60 negras, por lo que cada negra ocupa 1 segundo.

Uno de los enfoques para beat tracking es la programación dinámica. El objetivo es generar una secuencia de beats que corresponda con los onsets percibidos en la señal, y que estos formen un patrón regular [5].

Un onset es el momento exacto en el que comienza un nuevo sonido (transient). Como se puede observar en la figura 2.1, al tocar un sonido ocurre el attack, que es la fase donde aumenta la energía del sonido. El término transient se refiere al pico corto y de alta energía al comienzo del sonido, por lo que también se puede definir el onset como el instante que marca el inicio del transient, o el punto en el que un transient puede ser detectado [36].

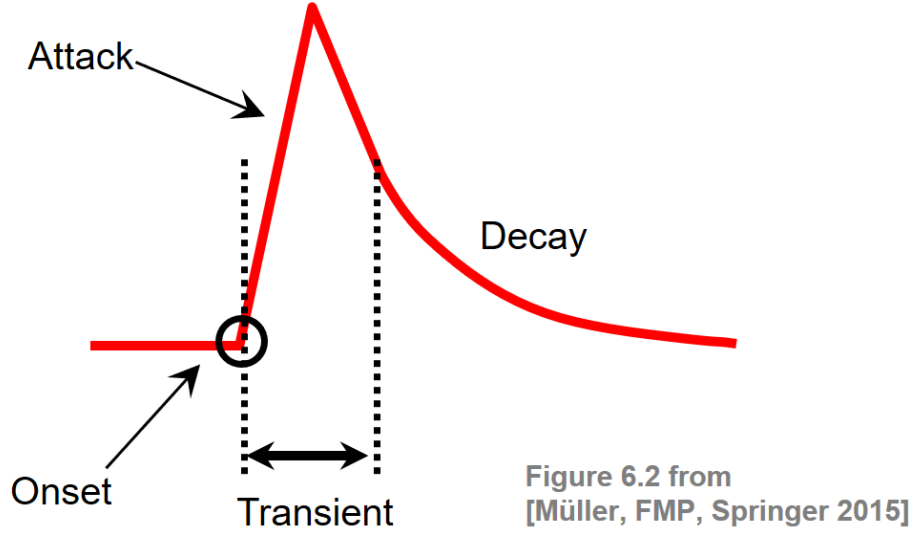


Figura 2.1: Detección de onsets. Imagen obtenida de [36].

Para ello, se utiliza una función objetivo que localiza los beats y otra que penaliza las desviaciones:

$$\Xi(t_i) = \sum_{i=1}^N O(t_i) + \alpha \sum_{i=2}^N F(t_i - t_{i-1}, \tau_p)$$

Donde t_i es la secuencia de N beats, $O(t)$ es la fuerza del onset en el instante t , α es un parámetro para balancear la penalización de F , y τ_p es un tempo ideal.

Para construir la mejor secuencia de beats posibles, se puede tomar la fórmula recursiva:

$$\Xi^*(t) = O(t) + \max_{\tau=0, \dots, t} \{ \alpha F(t - \tau, \tau_p) + \Xi^*(\tau) \}$$

Así, para cada valor de t se decide si debería considerarse como beat teniendo en cuenta si hay una alta activación de onsets $O(t)$ y si el tiempo desde el beat anterior τ es coherente con el tempo esperado τ_p .

2.3.2. STFT

La Short-Time Fourier Transform (STFT) es una Transformada de Fourier que determina cómo varía la frecuencia sinusoidal de la señal a lo largo del tiempo. A diferencia de la Transformada de Fourier (FT), que muestra el contenido espectral global de la señal, la STFT divide la señal en pequeñas ventanas de tiempo y aplica la transformada de Fourier en cada una [35].

Así, la STFT es capaz de convertir la señal de audio $x[n]$ del dominio tiempo al dominio tiempo-frecuencia. Esta se calcula como:

$$\xi(m, k) = \sum_{n=0}^{N-1} x[n + mH] \cdot w[n] \cdot e^{-2j\pi kn/N}$$

Donde $x[n]$ es la señal discreta de audio, $w[n]$ es una ventana discreta de longitud $N \in \mathbb{N}$, N es el tamaño de la ventana, $H \in \mathbb{N}$ es el tamaño de salto y $k \in [0 : K]$. El

número complejo $\xi(m, k)$ denota el k -ésimo coeficiente de Fourier para el m -ésimo marco temporal, donde $K = N/2$ es el índice de frecuencia correspondiente a la frecuencia de Nyquist.

La frecuencia de Nyquist surge del Teorema de muestreo. Este establece que una señal cuya tasa de muestreo es F se puede reconstruir si contiene solamente componentes de frecuencia por debajo de la mitad: $F/2$. Esta frecuencia se conoce como frecuencia de Nyquist [26]. Por ejemplo, los CD usan una tasa de muestreo de 44.100 Hz, por lo que la frecuencia de Nyquist en los CD es 22.050 Hz.

La STFT se suele visualizar a través del espectrograma, que es una representación bidimensional de la magnitud al cuadrado:

$$\theta(m, k) = |\xi(m, k)|^2$$

Donde el eje horizontal representa el tiempo, el eje vertical representa la frecuencia y la intensidad de color en la imagen representa el valor concreto de la frecuencia en un punto.

2.3.3. MFCC

Los Mel Frequency Cepstral Coefficients (MFCC) permiten medir la forma del espectro de frecuencias en la escala de Mel [52]. La escala de Mel es una escala que intenta imitar la percepción del oído humano y divide las frecuencias en intervalos equidistantes. La siguiente función transforma la frecuencia a la escala de Mel:

$$F_{Mel}(x) = 1127 \ln(1 + \frac{x}{700})$$

Para calcular los coeficientes, se calcula la STFT y se hace un mapeo entre la STFT y la escala de Mel. Se utilizan una serie de filtros triangulares que son los que hacen esta transformación:

$$S(n) = \sum_{k=0}^{N-1} |\xi(k)|^2 \cdot J_m(k)$$

Donde $0 \leq m \leq M - 1$, m es el número de filtros triangulares y $J_m(k)$ es el peso de contribución del bin k al filtro triangular m . Es decir, cuánto aporta el bin k al filtro m . Los bins son intervalos entre los samples en el dominio de frecuencias.

Una vez obtenida la energía espectral en la frecuencia de Mel, se puede calcular la Discrete Cosine Transform (DCT) para obtener un determinado número de coeficientes:

$$\Lambda_m(n) = \sum_{m=0}^{M-1} S(n) \cdot \cos(\frac{\pi n(m - 0,5)}{M})$$

Donde $n \in \mathbb{N}$ es el número de MFCC que se quieran. En concreto, en este proyecto trabajaremos con 13 coeficientes de acuerdo con Ulloa [52].

2.3.4. Chroma Coefficients

Esta característica considera propiedades de la armonía y la melodía de la señal. Mientras que la melodía es la secuencia de notas tocadas siguiendo un ritmo y en diferentes intervalos, la armonía resulta de la combinación de la melodía y los acordes (conjunto de tres o más notas diferentes tocadas a la vez) [10].

En la figura 2.2 podemos observar un ejemplo simple para ver la diferencia entre melodía (sería lo que recordamos al tararear una canción) y la armonía (es el resultado sonoro de la interacción entre las notas musicales) [10].



Figura 2.2: Ejemplo de melodía y armonía. Imagen obtenida de [3].

En la música occidental, la música se divide en 12 tonos o pitches diferentes

$$\{C, C\#, D, D\#, E, F, F\#, G, G\#, A, A\#, B\},$$

que si traducimos del sistema de notación anglosajón al latino [20] sería

$$\{Do, Do\#, Re, Re\#, Mi, Fa, Fa\#, Sol, Sol\#, La, La\#, Si\}$$

Esto se conoce como escala cromática [49], teniendo siete notas naturales y cinco alteraciones, $\sharp$ y $\flat$, que elevan la nota un semitono o la disminuyen, respectivamente. En total, en una octava hay 12 notas posibles, ya que notas como $Do\sharp$ y $Re\flat$ son equivalentes enarmónicamente. Para calcular los coeficientes, se calcula la STFT y se obtiene el espectrograma:

$$Y(n, p) = \sum_{k \in P(p)} |\xi(n, k)|^2$$

Donde $p \in [0 : 127]$ es el número de la nota MIDI y representa el tono, y $P(p)$ es una función que mapea k a las notas MIDI. Las notas MIDI son una forma de dar una descripción de la altura de una nota a un instrumento musical electrónico, codificando cada nota como un número entre 0 y 127. Esto se puede codificar en 7 bits, y se podrían cubrir 11 octavas [42].

Una vez obtenido el espectrograma se pueden calcular los coeficientes sumando todos los coeficientes que pertenecen al mismo chroma:

$$\Psi(n, c) = \sum_{p \in [0:127]: p \bmod 12 \equiv c} Y(n, p)$$

Siendo $c \in [0 : 11]$.

En nuestro sistema de recomendación utilizaremos las cuatro características de audio descritas, por lo que ahora veremos qué modelos compondrán dicho sistema.

2.4. Inteligencia Artificial

La Inteligencia Artificial es una disciplina dentro de las ciencias de la computación que consiste en desarrollar un tipo de algoritmos que imiten la inteligencia humana para realizar tareas [23].

Estos algoritmos son capaces de extraer conclusiones a partir de unos datos de entrada. El algoritmo o modelo aprende de esos datos de forma que, al introducir nuevos datos, sea capaz de trabajar correctamente con ese nuevo dato.

Dentro de la inteligencia artificial tenemos el machine learning (ML) y el deep learning (DL), como podemos observar en la figura 2.3. El machine learning se basa en algoritmos que aprenden patrones a partir de datos etiquetados o no etiquetados para tomar decisiones o hacer predicciones. Por su parte, el deep learning es una subárea del ML que utiliza redes neuronales para abordar problemas complejos, como el reconocimiento de voz, imagen o texto.

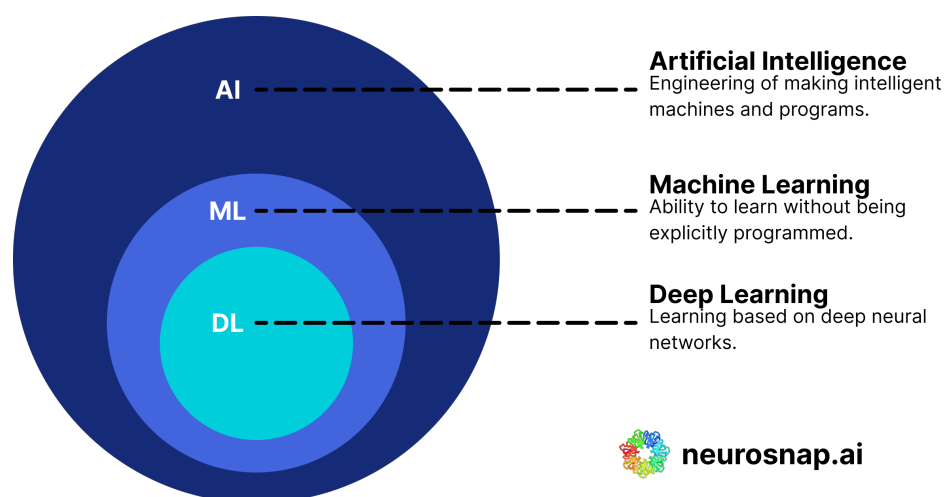


Figura 2.3: Áreas de la Inteligencia Artificial. Imagen obtenida de [38].

Además, es importante representar de forma numérica conceptos complejos como palabras, imágenes o audio para que la IA pueda llegar a entenderlos. Aquí es donde entran en juego los embeddings, que permiten transformar estos datos en vectores de características que pueden ser procesados por modelos de ML y DL. Comprender estos tres elementos —embeddings, machine learning y deep learning— será necesario para profundizar en el funcionamiento del sistema que se desarrolle.

2.4.1. Embedding

Un embedding es una representación numérica de los datos, ya sean imágenes, texto o audio, en un espacio multidimensional. Estos permiten que las máquinas puedan procesar de forma más eficiente los datos para realizar tareas [18].

Estos embeddings se construyen de forma que los algoritmos de machine learning o deep learning sean capaces de entender las relaciones y patrones de los datos originales.

2.4.2. Machine Learning

El machine learning (ML) es un subconjunto de la IA del tipo “memoria limitada”. Es decir, puede almacenar datos de entrada y los datos de todas las decisiones del sistema y luego analizarlos para mejorar con el tiempo [23].

Hay tres tipos principales de algoritmos dentro del ML [23]:

- **Aprendizaje supervisado.** En el aprendizaje supervisado una IA supervisa activamente todo el proceso de aprendizaje. Se le proporcionará a la IA tanto los datos que tendrá que aprender, como los resultados que esos datos deberían producir. El objetivo del modelo es aprender una función que relacione entradas con salidas de forma que pueda generalizar correctamente a nuevos datos no vistos. Se puede utilizar aprendizaje supervisado para resolver los siguientes tipos de problemas:
 1. Clasificación. En los problemas de clasificación la clase objetivo que se quiere predecir es discreta (como “soleado”, “lluvioso” y “nublado” al intentar predecir el tiempo).
 2. Regresión. La clase objetivo a predecir es continua (como el precio de la vivienda).
 3. Series temporales. En las series temporales se utilizan datos observados en intervalos regulares de tiempo (días, semanas, meses...) y sirven para hacer predicciones en base a la tendencia observada en los datos (como intentar predecir las temperaturas medias de los meses del siguiente año a partir de los 10 últimos años).

Un ejemplo de algoritmo supervisado sería k-NN (será explicado en 2.4.2). Este algoritmo puede ser usado tanto en problemas de clasificación como de regresión.

- **Aprendizaje no supervisado:** Implica que no hay intervención humana durante el aprendizaje. El modelo recibe unos datos de entrada e identifica patrones en esos datos de forma independiente. Es especialmente útil cuando no se dispone de salidas esperadas o cuando los datos son demasiado costosos de etiquetar manualmente. Se puede utilizar aprendizaje no supervisado para resolver las siguientes tareas:
 1. Asociación. En los problemas de asociación queremos encontrar relaciones frecuentes entre elementos en conjuntos de datos (como saber qué otros productos se suelen comprar al comprar leche en el supermercado).
 2. Clustering. Consiste en agrupar datos en grupos similares sin etiquetas previas (un ejemplo sería agrupar estudiantes cuyas condiciones de vida y sus notas sean similares).
 3. Resumen. Consiste en reducir o condensar datos manteniendo la mayor cantidad de información posible. Un ejemplo de algoritmo para esta tarea sería PCA.
- **Aprendizaje por refuerzo:** Es el más complejo, ya que no se proporciona ningún conjunto de datos para entrenar la IA, sino que el modelo debe aprender

interactuando con el entorno donde se encuentra. Se utilizan recompensas para que vaya mejorando con el tiempo y maximice la cantidad de recompensas que puede conseguir.

PCA

Principal Component Analysis (PCA) es una técnica de extracción de características donde se combinan las entradas de una forma específica y se eliminan algunas de las variables menos importantes, manteniendo la parte más importante de todas las variables. Asimismo, al utilizar PCA se consigue que todas las nuevas variables sean independientes entre sí [6].

El algoritmo funciona de la siguiente forma:

- Se estandarizan los datos de entrada restando la media de cada variable.
- Se obtiene la matriz de covarianza. Esta matriz contiene estimados de cómo cada dimensión de los datos se relaciona con el resto.
- Se obtienen los autovectores y autovalores de la matriz de covarianza. Cada autovector indica una dirección del espacio (componente principal), y su autovalor asociado representa cuánta varianza de los datos es explicada en esa dirección. Asimismo, a mayor variabilidad en una dirección en particular mayor información, mientras que poca variabilidad indica ruido.
- Se ordenan los autovalores de mayor a menor y se eligen los k autovectores que se correspondan con los mayores autovalores.
- Se construye la matriz de proyección W con los k autovectores seleccionados.
- Se transforma el dataset original x usando la fórmula $y = W' \cdot x$, donde W' es la transpuesta de W [46].

k-NN

k-Nearest Neighbors, o k-NN, es un algoritmo de machine learning que usa la comparación de la proximidad de un punto con el resto de un conjunto de datos. Es un algoritmo supervisado en el que k representa la cantidad de vecinos más cercanos que se tienen en cuenta [13]. Para calcular esa proximidad se suelen utilizar estas funciones de distancia:

- **La distancia euclidiana** mide una recta entre el punto de referencia y el punto a comparar.
- **La distancia de Manhattan** mide el valor absoluto entre dos puntos. Se representa en una cuadrícula, y su nombre viene del diseño de las calles de Manhattan, que también forman una cuadrícula.
- **La distancia de Minkowski** es una generalización de la distancia euclidiana y de Manhattan. Se define un parámetro p que define el tipo de distancia. Por

ejemplo, si $p = 1$, se usa la distancia de Manhattan y si $p = 2$, la distancia euclidiana.

2.4.3. Deep Learning

El deep learning (DL) es un subconjunto del ML que intenta emular las redes neuronales humanas. Pueden recibir, procesar y analizar grandes cantidades de información no estructurada y utilizarlas para aprender sin intervención humana.

Dentro de la gran variedad de arquitecturas que ofrece el DL, existen algunas diseñadas específicamente para extraer representaciones comprimidas y significativas de los datos. Una de las más conocidas y utilizadas con este propósito es el autoencoder.

Autoencoder

Es una red neuronal que se entrena para comprimir y luego reconstruir datos de entrada, aprendiendo así una representación más eficiente en un espacio más reducido. Consiste en dos partes: un encoder que reduce la dimensionalidad de los datos, formando el conocido como espacio latente, y un decoder que reconstruye la entrada a partir de los vectores latentes. En la figura 2.4 se puede ver la estructura de un Autoencoder.

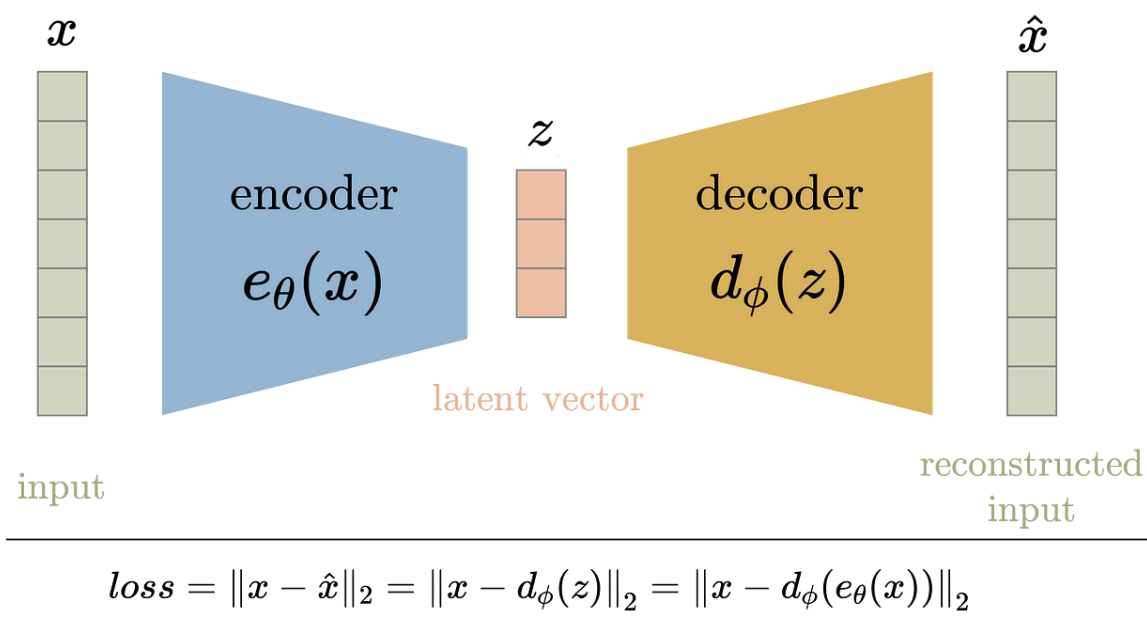


Figura 2.4: Autoencoder. Imagen obtenida de [45].

Como se puede observar también en la figura anterior, la fórmula de pérdida que se suele utilizar es el Error Cuadrático Medio (MSE):

$$loss = \|x - \hat{x}\|^2 = \|x - d_{\phi}(z)\|^2 = \|x - d_{\phi}(e_{\theta}(x))\|^2$$

donde x es la entrada, z es el vector latente, $\hat{x}$ es la entrada reconstruida, e_θ es el resultado de aplicar el encoder a la entrada y d_ϕ es el resultado de aplicar el decoder al vector latente.

La idea es que la red neuronal aprenda a reducir los datos minimizando la diferencia entre la entrada y la entrada reconstruida. Al minimizar estos errores, la red aprenderá una representación latente compacta, es decir, una representación que mantenga la mayor información posible [17].

En la siguiente figura 2.5 se puede observar una representación del espacio latente para el dataset MNIST, que es un dataset que contiene imágenes con dígitos del 0 al 9 [53]. Se puede observar cómo los puntos que corresponden a un mismo dígito tienden a ir en la misma dirección, perteneciendo al mismo clúster. Esto se observa muy bien en casos como el número 0 o el 1, mientras que hay números que se superponen a otros clústeres, como el 8 y el 9.

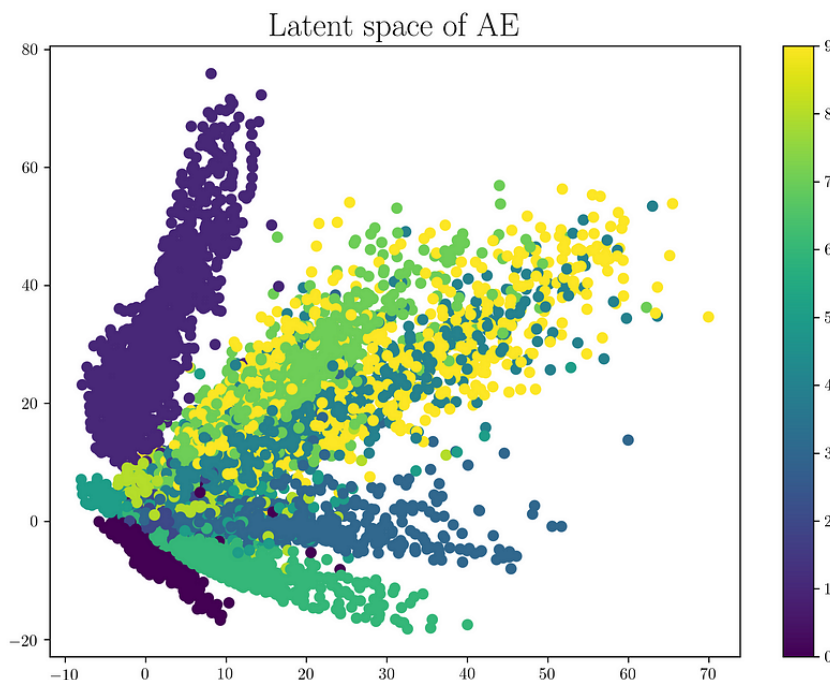


Figura 2.5: Espacio latente de Autoencoder. Imagen obtenida de [45].

Existen variantes de esta arquitectura, como los Variational Autoencoders (VAE), que introducen conceptos probabilísticos en el espacio latente, permitiendo aplicaciones más avanzadas como la generación de nuevos datos.

Variational Autoencoder

El Variational Autoencoder (VAE) extiende las capacidades del Autoencoder y soluciona el hecho de que el espacio latente del Autoencoder no esté regularizado [45].

Mientras que el autoencoder es un modelo *determinista*, pues codifica un único vector latente, el VAE es un modelo *probabilístico*. Los VAE codifican las variables

latentes de los datos de entrenamiento no como un valor discreto fijo z , sino como un rango continuo de posibilidades expresado como una distribución de probabilidad $p(z)$ [25].

En la figura 2.6 se observa la estructura del VAE. Para cada variable, los VAE calculan dos vectores latentes diferentes: un vector de medias μ y un vector de varianzas σ . El VAE tiene las siguientes partes:

- **Encoder:** El encoder calcula la probabilidad posterior $p(z|x)$. Las salidas serán un conjunto de medias y varianzas de logaritmos. Para estabilidad numérica se utiliza la varianza de logaritmos en lugar de la varianza directamente [21].
- **Decoder:** El decoder calcula la probabilidad posterior $p(x|z)$, tomando un vector latente como entrada y generando los parámetros para una distribución condicional de la observación.
- **Truco de la reparametrización:** El truco de la reparametrización introduce un nuevo parámetro $\epsilon \sim \mathcal{N}(0, 1)$. Este parámetro puede verse como un ruido aleatorio para mantener la estoicidad de z [21]. Posteriormente, se reparametriza la variable z como $z = \mu_x + \sigma_x \epsilon$. Esta variable z será la que permita se pueda optimizar mediante retropropagación usando descenso del gradiente.

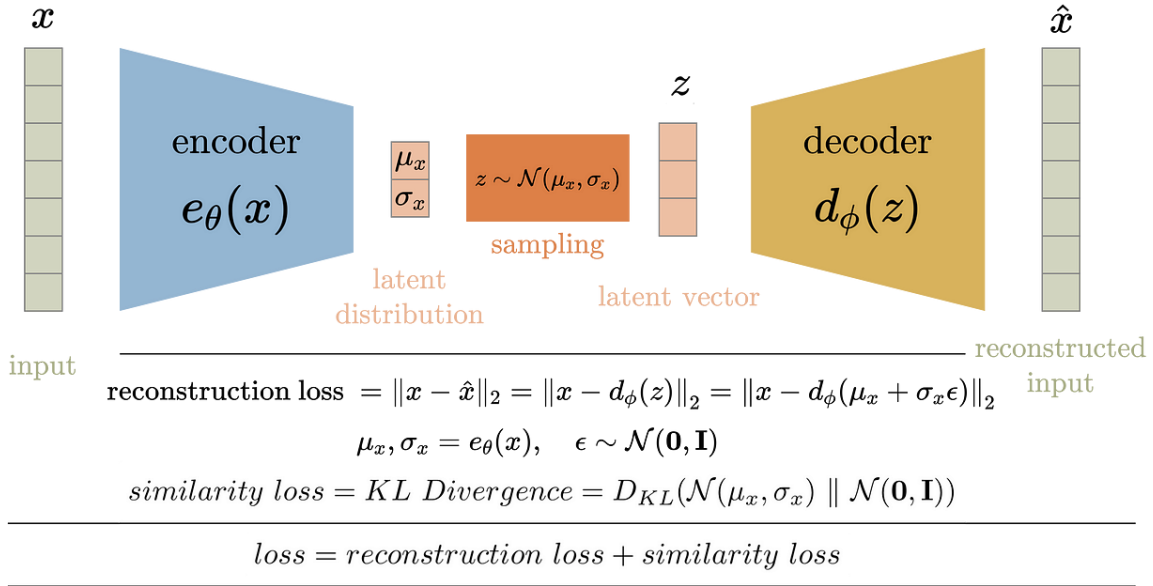


Figura 2.6: Variational Autoencoder. Imagen obtenida de [45].

Asimismo, para minimizar la diferencia entre la entrada y la entrada reconstruida se utiliza como función de pérdida dos funciones:

1. La función del error de la reconstrucción, que es el MSE, utilizada también en el Autoencoder:

$$\text{reconstruction loss} = \|x - \hat{x}\|^2 = \|x - d_\phi(z)\|^2 = \|x - d_\phi(\mu_x + \sigma_x \epsilon)\|^2$$

donde $\mu_x, \sigma_x = e_\theta(x), \quad \epsilon \sim \mathcal{N}(0, 1)$.

2. La función del error de similitud, que es la *divergencia de Kullback-Leibler* o divergencia KL entre la distribución del espacio latente y la gaussiana:

$$\text{similarity loss} = D_{KL}(\mathcal{N}(\mu_x, \sigma_x) \parallel \mathcal{N}(0, 1))$$

Sin embargo, al utilizar la divergencia KL para la inferencia variacional se tiene que el denominador de la ecuación es *intratable*, por lo que llevaría una cantidad de tiempo teóricamente infinita calcularlo directamente [25]. Para solucionar este problema, los VAE se aproximan a la minimización de la divergencia KL maximizando el límite inferior de evidencia (ELBO). La ecuación final que maximizaría el ELBO sería [55]:

$$\text{similarity loss} = -\frac{1}{2} \sum_j (1 + \log(\sigma_j^2) - \mu_j^2 - \sigma_j^2)$$

donde j es la dimensión del espacio latente.

La figura 2.7 visualiza el espacio latente del VAE, que está más regularizado que el del Autoencoder, pues no hay espacio entre los clústeres (aunque hay cierto solapamiento) y se asemeja a la distribución gaussiana multivariante.

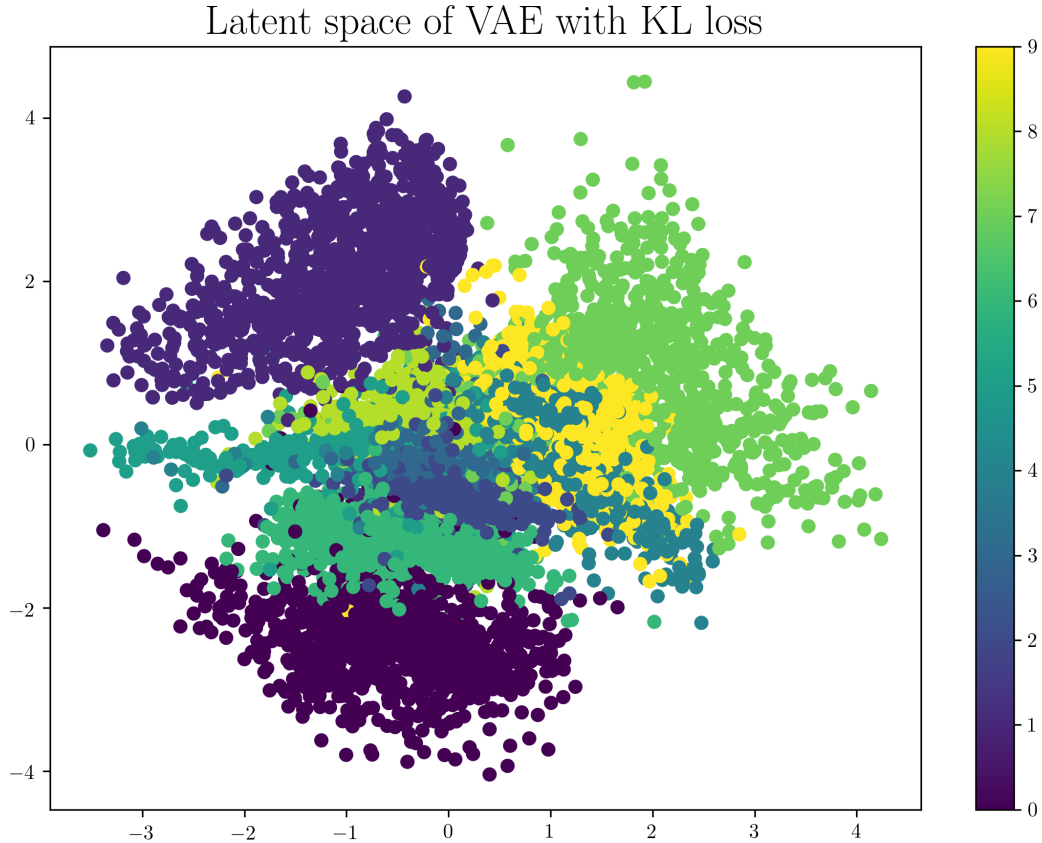


Figura 2.7: Espacio latente del Variational Autoencoder. Imagen obtenida de [45].

3. Estado del arte

3.1. Introducción

Los sistemas de recomendación musical han ido evolucionando a lo largo de las últimas décadas, siendo muy importantes en las plataformas de streaming y servicios digitales. Con el auge del machine learning, estos sistemas han pasado de filtrado basado en contenido y filtrado colaborativo a modelos más complejos que aprovechan técnicas de deep learning, procesamiento de señales y análisis de patrones de escucha.

Actualmente, los sistemas inteligentes de recomendación musical emplean diferentes técnicas, como modelos basados en contenido, filtrado colaborativo y modelos híbridos, además de técnicas avanzadas como embeddings generados mediante redes neuronales y métodos de clustering para identificar similitudes entre canciones y usuarios.

Este estado del arte analiza las principales técnicas utilizadas en la literatura, destacando los avances más recientes en el uso de deep learning, sistemas híbridos y, principalmente, en modelos basados en señales musicales. Además, se revisarán los puntos fuertes de estos sistemas y los desafíos actuales, como la eficiencia y la diversidad en las recomendaciones.

3.2. Características de audio

Antes de hablar de los sistemas de recomendación musical, es importante comentar una de las bases en las que se basan muchas aplicaciones de temática musical que utilizan modelos de machine learning. Estos sistemas suelen utilizar diferentes características, como el tempo o la melodía, para poder realizar diversas tareas, como clasificar el género de una canción o realizar recomendaciones.

Para entender qué características y técnicas se han utilizado, también se ha revisado literatura relacionada con la clasificación de canciones según el género. Las características principales utilizadas son:

- **STFT (Short Time-Frequency Transform):** Es una Transformada de Fourier diferente a la tradicional que descompone la señal de audio en ventanas de tiempo más cortas y calcula la FFT (Fast Fourier Transform) a cada una de ellas. Así, es posible obtener un conjunto de coeficientes complejos que representan las frecuencias activas en cada instante de tiempo [1].
- **Chroma vectors:** Es un vector doce-dimensional donde cada dimensión representa una clase de pitch que se calcula a partir de la STFT. El pitch es la percepción subjetiva de la frecuencia de un sonido, es decir, lo agudo o grave que percibimos el sonido. Es un término complejo de entender debido a que dos notas

en dos instrumentos diferentes pueden tener la misma frecuencia fundamental, por ejemplo un La a 440 Hz, pero percibirse con diferente timbre [58].

- **MFCC (Mel Frequency Cepstral Coefficients):** Son coeficientes que representan las características estáticas del audio en la Escala Mel, que se considera la aproximación más cercana a la percepción auditiva humana [1]. Estos coeficientes se calculan a partir de la STFT, y opcionalmente se puede añadir las deltas o las deltas-deltas, que representan la velocidad y la aceleración con la que cambian los MFCC [56].
- **BPM (Beats Per Minute):** Es una medida que define el tempo de un audio contando el número de beats (base rítmica) que caben en un minuto. Por ejemplo, $\text{♩} = 60$ indica que en un minuto caben 60 negras, por lo que cada negra ocupa 1 segundo.
- **ZCR (Zero crossing rate):** Es una característica que cuenta cuántas veces la onda cruza el eje X [37]. Así podemos observar la suavidad de la onda, y la principal utilidad de esta característica es distinguir entre sonidos agudos y sonidos de percusión. Asimismo, hay diferencias entre géneros musicales, pues una canción de un género como el metal tendrá un ZCR mayor a una canción de blues [12].

Estas propiedades, usándolas de forma combinada, podrán permitirnos recoger la información más importante de los audios de las canciones para construir el sistema de recomendación.

3.3. Sistemas de recomendación

Los sistemas de recomendación musical son sistemas que permiten mostrar al usuario canciones, álbumes, artistas o listas de reproducción en función de sus preferencias, historial de escucha o características de las canciones.

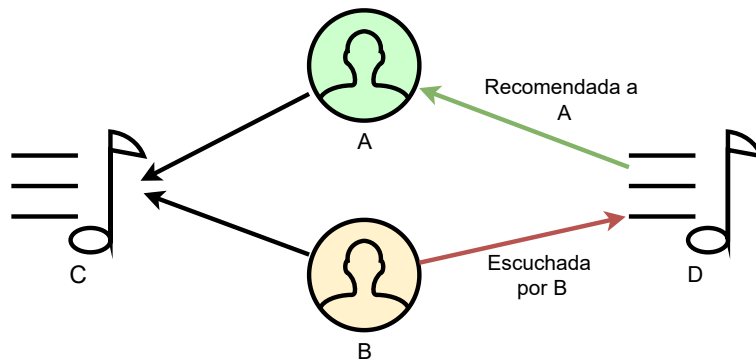


Figura 3.1: Ilustración de sistemas basados en filtrado colaborativo

En general, un sistema de recomendación compara el perfil de un usuario con algunas características de referencia. Las técnicas más utilizadas son el filtrado colaborativo y el filtrado basado en contenido. El filtrado colaborativo utiliza la lógica de que

a los usuarios similares les gustarán las mismas canciones. Dos usuarios serían similares si escuchan o les gustan canciones parecidas. En la figura 3.1 podemos observar dicha lógica. Si al usuario A y B les gusta la canción C, es probable que a A le guste la canción D porque también le gusta al usuario B.

Por otra parte, el filtrado basado en contenido utiliza la lógica de que si a un usuario le ha gustado o ha escuchado una canción, podrían gustarle canciones similares. Estos sistemas se basan exclusivamente en las preferencias pasadas del usuario, en contraposición con el sistema anterior que se basa en las preferencias de otros usuarios. En la figura 3.2 podemos observar dicha lógica: al usuario A le ha gustado la canción B, y las canciones C y B son parecidas. Por ello, es probable que a A también le guste C.

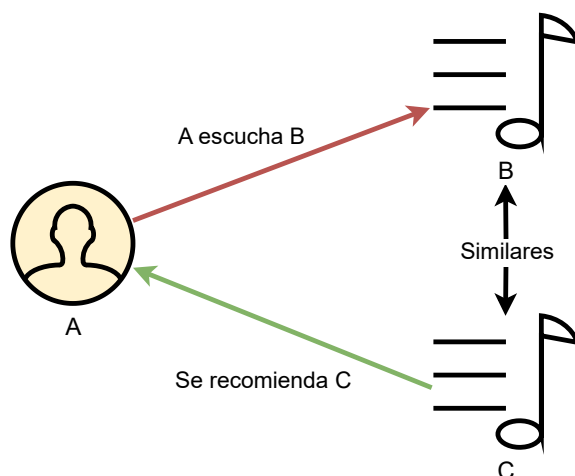


Figura 3.2: Ilustración de sistemas basados en filtrado basado en contenido

Para calcular la similitud entre dos canciones se suele utilizar la similitud coseno o la distancia euclídea. La similitud coseno es el coseno del ángulo que forman dos vectores, de forma que si el coseno es cercano a 1, los vectores son muy similares, y si es cercano a -1, son muy diferentes. En cambio, la distancia euclídea mide la distancia entre dos puntos vectoriales; cuanto menor sea esta distancia, más similares son los vectores.

Muchos sistemas de recomendación utilizan ambas técnicas; son los conocidos como sistemas híbridos.

3.3.1. Algoritmos utilizados en la literatura

Para implementar los sistemas de recomendación, se suelen utilizar diversos algoritmos de machine learning. Algunos de los más destacados en la literatura son:

- **SVM (Support Vector Machine):** Algoritmo de clasificación útil para características de grandes dimensiones que es capaz de trabajar con datos no lineales. Tiene gran utilidad a la hora de encontrar patrones y hacer recomendaciones basadas en las emociones del usuario [8].

- **KNN (k-Nearest Neighbour):** Algoritmo de clasificación que utiliza la etiqueta asociada a los K vecinos más cercanos para determinar la etiqueta del ejemplo actual. Este algoritmo parte de la idea de que los puntos cercanos en el espacio vectorial serán más similares entre sí. Este algoritmo ha sido utilizado para la clasificación de géneros [1].
- **Logistic Regression:** Algoritmo de clasificación para predecir una variable categórica en función de las variables predictoras [1]. Este algoritmo fue utilizado para predecir la emoción del usuario para recomendarle música según su estado de ánimo [8].
- **Random Forest:** Algoritmo ampliamente utilizado que combina múltiples árboles de decisión para llegar a un resultado con un rendimiento similar a la del boosting [8].

A su vez, han ido surgiendo sistemas más complejos que utilizan modelos de deep learning para intentar detectar nuevos patrones. Algunos de dichos algoritmos o técnicas son los siguientes:

- **Multilayer Perceptron:** Red neuronal artificial que soluciona los problemas del perceptrón simple y es capaz de resolver problemas que no son linealmente separables [1] [37].
- **Recurrent Neural Networks (RNN):** Red neuronal profunda entrenada con datos secuenciales o de series temporales para crear un modelo que pueda hacer predicciones secuenciales o conclusiones basadas en entradas secuenciales [24] [1].
- **Convolutional Neural Networks (CNN):** Red neuronal que usa datos tridimensionales para tareas de clasificación y aprendizaje de patrones en imágenes [33].
- **Embedding:** Técnica para transformar los datos iniciales en datos numéricos que permita que puedan ser manipulados matemáticamente, y así poder encontrar la similitud entre los datos [57].
- **Autoencoder:** Red neuronal que se entrena para comprimir datos y posteriormente reconstruirlos, por lo que es capaz de detectar patrones y características de los datos trabajando con un espacio de menor dimensionalidad [57].

3.3.2. Nuevos enfoques en los sistemas de recomendación

Dejando de lado los algoritmos utilizados, también se han explorado nuevos tipos de sistemas de recomendación. El primer ejemplo de esto sería un sistema basado en emociones [8]. Este sistema utiliza Natural Language Processing (NLP) para predecir el estado de ánimo del usuario, de forma que se le recomiende música que tenga como etiqueta ese mismo estado en la base de datos. No es el primer sistema que intenta recomendar música basado en las emociones del usuario, pero sí es un sistema muy poco intrusivo. Una persona solo debe escribir cómo se siente y el sistema se encargará de predecir su estado de ánimo y la música que debería escuchar.

Otro ejemplo sería un Cross-domain Recommendation System (CDRS), diseñado para abordar problemas como la escasez de datos, el cold start y la falta de diversidad en las recomendaciones [57]. Estos sistemas aprovechan el conocimiento de dominios fuente con mayor cantidad de información para mejorar las recomendaciones en dominios con datos limitados. Un ejemplo de caso práctico de estos sistemas podría ser la integración de información proveniente de redes sociales, donde se analicen las interacciones y los intereses de los usuarios para enriquecer las recomendaciones de contenido en plataformas de streaming musical.

Otro enfoque en los sistemas de recomendación musical se basa en el uso de huellas digitales de audio (audio fingerprints) para identificar similitudes entre canciones. Ulloa [52] propuso la combinación de diferentes características de audio, como los Coeficientes Cepstrales en las Frecuencias de Mel (Mel Frequency Cepstral Coefficients, MFCC), los chroma coefficients o el tempo, para construir un vector de alta dimensión que representa la huella digital de una canción. Luego, se aplicó Análisis de Componentes Principales (PCA) para reducir la dimensionalidad y calcular una matriz de similitud entre canciones. La evaluación del sistema mostró una precisión del 89 % al recomendar canciones del mismo género que la canción objetivo, lo que sugiere que este método puede ser eficaz para la recomendación basada en contenido.

Por otro lado, tendríamos los sistemas de recomendación secuencial, que han ganado interés en los últimos años, especialmente en plataformas de streaming de música. Un estudio reciente presentó Psychology-Informed Session embedding using ACT-R (PISA), un sistema que combina modelos de Transformers con la arquitectura cognitiva Adaptive Control of Thought-Rational (ACT-R) para capturar patrones dinámicos y repetitivos en el comportamiento de los usuarios [50]. Este modelo considera la importancia de la repetición en la escucha musical y genera recomendaciones que reflejan tanto las preferencias cambiantes del usuario como su tendencia a volver a escuchar canciones anteriores.

Finalmente, otra línea de investigación en sistemas de recomendación musical combina técnicas de clasificación de géneros con características extraídas directamente de las canciones [28]. Este integra tres factores principales: clasificación de género mediante una red neuronal, MFCC y el tempo de la canción. Se evaluaron cuatro estrategias diferentes de recomendación, donde se ponderaban estos factores de manera distinta. Los resultados mostraron que los MFCC tenían el mayor impacto en la calidad de las recomendaciones, seguidos por la clasificación de género y el tempo.

Aunque el uso de deep learning ha permitido que se desarrollen sistemas más refinados, Ndou et al. [37] consiguieron superar el rendimiento de los algoritmos de deep learning usando machine learning. Aunque el objetivo del artículo es diferente al de este trabajo, es relevante tenerlo en cuenta y abre puertas a realizar comparaciones entre algoritmos de diferente tipo o a priorizar los algoritmos de machine learning.

Un enfoque muy interesante es el de los autoencoders, que permiten aprender una representación de los datos de entrada, reduciendo su dimensionalidad y reconstruyéndolos para minimizar la diferencia entre la entrada y la reconstrucción. Tiene una funcionalidad parecida a PCA, con la ventaja de que un autoencoder involucra operaciones no lineales con una reducción mayor de la dimensionalidad [29]. Quijada [40] propone un sistema de recomendación basado en contenidos utilizando autoenco-

ders de 5 y 7 capas, y Saini and Singh [43] propone un sistema basado en contenidos combinando un stacked Long Short-Term Memory (LSTM) y un autoencoder basado en atención.

Un enfoque generativo similar al de los Variational Autoencoders (VAE) es el de Chen et al. [9], donde proponen un sistema de recomendación basado en un modelo de difusión condicionado. Otro sistema interesante es el de Salas [44], donde se utilizan redes siamesas en diferentes ámbitos.

Asimismo, la bibliografía que más impacto tendrá en el sistema de recomendación que se implementará serán los estudios de Kostrzewa et al. [28], Ulloa [52], Quijada [40] y Saini and Singh [43]. Esto se debe a que dependen directamente de características objetivas, obtenidas a través del audio de la canción y a que utilizan metodologías directamente aplicables al objetivo del trabajo. Otros sistemas como el de Behura et al. [8] dependen de la emoción con la que se haya catalogado cada canción de la base de datos, lo que es una propiedad bastante subjetiva. Por otra parte, replicar un sistema como el que propone Zhu et al. [57] sería muy complicado al necesitar datos de diferentes dominios que se pudieran combinar, y para replicar el estudio de Tran et al. [50] y Chen et al. [9] se necesitarían tanto datos de las canciones como de los usuarios en plataformas de streaming o páginas musicales.

3.4. Conclusiones

En este capítulo se ha revisado la evolución y el estado actual de los sistemas de recomendación musical, identificando las principales técnicas utilizadas en la literatura y los avances más recientes en la personalización de recomendaciones.

Se ha constatado que los métodos tradicionales de filtrado basado en contenido y filtrado colaborativo siguen siendo primordiales en estos sistemas. Sin embargo, la integración de técnicas más avanzadas, como el uso de embeddings generados con deep learning, modelos híbridos y enfoques basados en señales musicales, ha permitido mejorar la precisión y la adaptabilidad de las recomendaciones.

El análisis de las características de audio ha demostrado que propiedades como los MFCC, los vectores cromáticos y la STFT son esenciales para extraer información relevante y objetiva de las canciones. Estas características han sido ampliamente utilizadas en los sistemas de recomendación, así como en tareas de clasificación de géneros musicales y detección de patrones.

Asimismo, se han identificado nuevas tendencias en la literatura, como los sistemas de recomendación basados en emociones, los CDRS, el uso de huellas digitales de audio y los sistemas secuenciales que incorporan modelos cognitivos. Estas estrategias han permitido abordar desafíos en la recomendación musical, como el problema del cold start, la falta de diversidad y la adaptación a los cambios en las preferencias de los usuarios.

Finalmente, se destaca que, aunque el deep learning ha impulsado avances significativos en la personalización de recomendaciones, algunos estudios han demostrado que algoritmos clásicos de machine learning pueden ser igualmente efectivos en ciertos

contextos. Además, la elección del enfoque dependerá en gran medida de los datos utilizados, la interpretabilidad del modelo y la eficiencia computacional.

Los estudios revisados en este capítulo servirán de base para el desarrollo del sistema de recomendación que se implementará en este trabajo, priorizando aquellos enfoques que utilizan características extraídas directamente del audio y métodos de reducción de dimensionalidad para la mejora de la precisión en la recomendación.

4. Estudio previo

4.1. Introducción

En este capítulo presentaremos los objetivos, la metodología, la planificación y el presupuesto del proyecto. Se abordarán aspectos clave para la consecución de los objetivos del trabajo.

4.2. Objetivos

Este trabajo final de máster explora las tendencias de los sistemas de recomendación musicales, con el objetivo de implementar un sistema que utilice algunos de estos enfoques y demuestre su utilidad para detectar las necesidades e intereses musicales de los usuarios. Los diferentes objetivos a cumplir son los siguientes:

1. **Revisión de la literatura:** Se realizará una revisión de la literatura con el fin de identificar, clasificar y analizar las principales técnicas de recomendación musical. Esta revisión incluirá métodos clásicos y actuales de Machine Learning y Deep Learning, así como métricas utilizadas para evaluar su rendimiento.
2. **Diseño y construcción del sistema de recomendación:** Se diseñará y construirá un sistema de recomendación musical que utilice representaciones vectoriales derivadas de características de audio. Para ello, se aplicarán técnicas de Machine Learning (como PCA y k-NN) y Deep Learning (como Autoencoders y Variational Autoencoders).
3. **Comparación y evaluación:** Comparar y evaluar cuantitativamente el rendimiento de las diversas técnicas implementadas. Además, se analizará el impacto de diferentes métricas de similitud y se discutirán los resultados obtenidos.

4.3. Metodología

A continuación, se va a comentar la metodología utilizada a lo largo del proyecto. Como se ha comentado en los apartados anteriores, se va a realizar un sistema de recomendación musical basado en diferentes algoritmos de machine learning y deep learning. En la figura 4.1 se ven los pasos seguidos en la metodología.

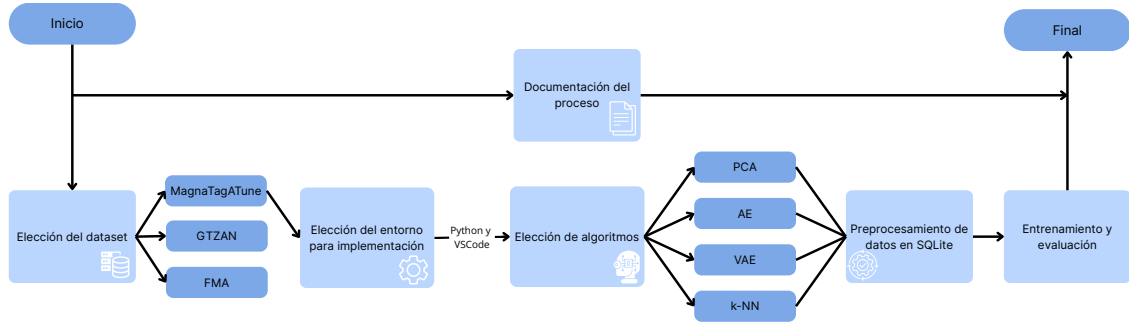


Figura 4.1: Metodología del proyecto

4.3.1. Documentación

Todo el proceso de investigación e implementación del sistema será documentado de forma exhaustiva y paralela al desarrollo del proyecto. Se utilizará Overleaf como editor LaTeX para esta tarea, principalmente por su amplio uso en la redacción de documentos científicos, su comodidad para estructurar la información y su facilidad para utilizar plantillas y crear entornos colaborativos frente a otros editores de texto de uso general como Word. Asimismo, se utilizará GPT-4 como apoyo para la redacción, mejorando la calidad de los textos y la profesionalidad de estos.

4.3.2. Elección del dataset

La elección del dataset es el primer paso para poder plantear el diseño del sistema. Para ello, es necesario que los datos cumplan ciertos requisitos:

- **Ficheros de audio.** Hay datasets que contienen la información ya procesada de los audios, lo que puede llegar a limitar el desarrollo si se quiere llegar a utilizar características diferentes del audio. Por ello, es recomendable poder tener los ficheros de audio directamente, ya sea en formato WAV o MP3. Asimismo, si se llegara a desplegar el sistema sería recomendable que el usuario pudiera escuchar las canciones recomendadas, y para ello se necesitarían los ficheros.
- **Metadatos de canciones.** También hay datasets que cumplen el apartado anterior pero que no dan información ninguna sobre el nombre de la canción o el artista. Para poder dar recomendaciones con cierto sentido sería importante que también este tipo de información.

- **Tamaño manejable.** Para garantizar que los modelos hayan sido entrenados con una cantidad suficiente de datos sin que el rendimiento de estos se vean afectados será importante no elegir un dataset demasiado grande. En el caso de que el dataset escogido tenga un mal rendimiento se podría intentar reducir el tamaño, pero esto podría provocar un desbalanceo de los datos si no se hiciera correctamente.

De acuerdo con los puntos anteriores, se analizaron tres datasets:

- **GTZAN:** Dataset que contiene 1.000 pistas de audio de 30 segundos cada una. Hay 10 géneros, 100 pistas por cada género y ocupa 1GB [51].
- **MagnaTagATune:** Dataset cuyos datos fueron obtenidos a través del juego TagATune y las canciones de Magnatune. Contiene 3GB de fragmentos de canciones de 30 segundos. Además, tiene varios ficheros con metadatos de las canciones [31].
- **FMA:** Dataset de gran escala que tiene diferentes versiones según el tamaño. La versión del dataset más pequeña ocupa 7GB y contiene 8.000 fragmentos de canciones balanceados para 8 géneros diferentes [11].

De estos tres, se descartó el primero porque no contenía información relativa a los metadatos de las canciones, y el tercero tenía un tamaño mínimo de 7 GB y máximo de 879 GB, dejando como mejor candidato el dataset de MagnaTagATune.

Este dataset contiene 3 GB de audios de diferentes canciones, fraccionados en fragmentos de 30 segundos de duración, así como ficheros con los metadatos de cada una de las canciones. Asimismo, contiene una gran variedad de etiquetas que aportan bastante información que será útil más adelante.

4.3.3. Implementación

Para el desarrollo del código del sistema de recomendación se utilizará un IDE (Entorno de Desarrollo Integrado). Este entorno se elegirá teniendo en cuenta el lenguaje y las librerías que se vayan a utilizar.

En primer lugar, los dos lenguajes de programación más utilizados en el campo de Data Science son R y Python. R es un lenguaje específico de dominio (DSL) ampliamente utilizado en estadística y análisis de datos, pero que se ve superado por Python al tener este una mayor cantidad de bibliotecas para machine learning y mayor soporte para integrarse con otros lenguajes y frameworks, o bases de datos SQL y NoSQL. Por estos motivos, es que se utilizará Python como lenguaje de desarrollo.

A su vez, tenemos la librería *librosa*, ampliamente utilizada en la literatura para el procesamiento de audio, que es otra ventaja que tendría utilizar Python frente a R.

Una vez decidido el lenguaje, tenemos que decidir el entorno. Algunos de los más utilizados serían Eclipse, Visual Studio Code y PyCharm.

- **Eclipse:** Este entorno cuenta con el plugin PyDev que permite el desarrollo en Python. Sin embargo, es un entorno más complejo y menos intuitivo que otros entornos más optimizados para Python [19].

- **Visual Studio Code:** VSCode es un editor ligero con soporte para Python a través de extensiones. Tiene compatibilidad con Jupyter Notebooks y con entornos virtuales, lo que lo convierte en una buena herramienta para Data Science [32].
- **PyCharm:** Es un IDE de pago específicamente diseñado para Python que cuenta con un plan gratuito para estudiantes. Incluye herramientas avanzadas para análisis de código y bibliotecas de Data Science y machine learning [27].

Por otra parte, tendríamos Google Colab, que es un servicio alojado de Jupyter Notebook que no requiere configuración.

De entre estos entornos, VSCode y PyCharm son notablemente mejores para el objetivo del proyecto. Por lo tanto, y por tener una mayor experiencia en su uso, utilizaremos VSCode. En caso de que el tiempo de ejecución de los modelos sea excesivo, se probarán los modelos en Google Colab.

4.3.4. Elección de algoritmos

Para la implementación del sistema de recomendación se utilizarán técnicas de Machine Learning y Deep Learning, aunque se dará más relevancia al Deep Learning.

Se utilizarán principalmente algunos de los algoritmos vistos en la literatura, explicados en la sección 2. Los algoritmos de ML que se usarán son PCA y k-NN, y de DL se usarán Autoencoders y Variational Autoencoders.

Aunque los algoritmos son un paso importante, no se debe descuidar la calidad de los datos. Según el concepto “Garbage in, Garbage out (GIGO)”, la calidad de los datos de salida de los algoritmos dependerá principalmente de la calidad de los datos de entrada. Esto sucede incluso si la lógica de los algoritmos es precisa. Por ello, tanto la calidad de los datos de entrada como la de los algoritmos son igual de importantes [7].

4.3.5. Procesamiento de datos

Una vez elegido el dataset, es necesario establecer la estrategia a seguir para el procesamiento de los datos. Como se explicó en el apartado anterior, hay 3 GB de audios de canciones en fragmentos de 30 segundos y ficheros con los metadatos de las canciones.

Las canciones vienen comprimidas en formato ZIP y los datos en CSV. Por lo tanto, se deberá tener en cuenta cómo se almacenarán los datos, pues esto condicionará la forma de implementar el sistema.

Para ello, hay dos opciones, una de ellas sería trabajar directamente con los ficheros como se han descargado. Así pues, solo se necesitaría tener un directorio donde guardar los ficheros y descomprimir las canciones. La desventaja de esto sería que habría que trabajar con dataframes muy grandes y no tendríamos un mecanismo sencillo de JOIN como en una base de datos (BDD) relacional.

Teniendo esto en cuenta, la otra opción sería trabajar con una BBDD y un almacenamiento interno o externo. La BBDD podría ser relacional o no relacional. Las tres alternativas de BBDD planteadas son:

1. **SQLite:** Base de datos relacional ligera que permite realizar peticiones de forma rápida porque no necesita un servidor dedicado. Al no tener que configurar un servidor, se utilizan simplemente archivos “.db”. No escala bien si hay muchas consultas simultáneas.
2. **PostgreSQL:** Sistema relacional más robusto que puede realizar consultas complejas y puede escalar mejor que SQLite.
3. **MongoDB:** Base de datos no relacional que almacena los datos en formato BSON (Binary JSON) en lugar de tablas con filas y columnas. La principal ventaja es poder escalar las lecturas rápidamente, pero tiene problemas si se necesitan consultas SQL avanzadas.

Como se realizará el sistema en un entorno local, se intentará buscar la solución más sencilla. Por ello, se utilizará en un principio SQLite. Como no se necesita configuración ninguna, podrá ser reemplazada fácilmente por otra BBDD si su rendimiento llega a ser ineficiente más adelante.

Asimismo, se pueden utilizar dos tipos de almacenamiento para los audios:

1. **Interno:** Es la forma más sencilla de almacenar los audios, utilizando el sistema de ficheros para tener un directorio donde estén todas las canciones.
2. **Externo:** Es una forma más compleja de almacenamiento, pues se tendría que utilizar Azure Blob Storage o Amazon S3, y esto añadiría problemas como la latencia o almacenar temporalmente las canciones descargadas para entrenar los modelos. Sin embargo, permitiría reducir de forma significativa el tamaño del sistema, al no necesitar tener descargado todo el tiempo los 3 GB de canciones.

Al igual que con las BBDD, se elegirá la solución más sencilla de implementar; por ello, se utilizará el almacenamiento interno para las canciones.

Para trabajar con las canciones, se procesarán los audios de 30 segundos usando la librería *librosa* y guardando los resultados de las diferentes características extraídas en ficheros separados. De esta forma, tendríamos, por una parte, los audios originales; por otra, las características, es decir, la información relevante; y, por otra, los metadatos.

4.3.6. Entrenamiento y evaluación

Para el entrenamiento y evaluación de los modelos se dividirá el dataset en dos conjuntos: un 80 % de los datos para entrenamiento y el 20 % restante para evaluación. Esta técnica permite garantizar que los datos de entrenamiento y de prueba sean independientes. Sin embargo, hay que tener en cuenta que el dataset viene preparado para poder utilizar K-fold Cross Validation, teniendo separadas en diferentes carpetas (donde cada una correspondería a un pliegue o ‘fold’) una parte equilibrada de las canciones totales. El problema está en que en el preprocesamiento de los datos vamos a tener que eliminar canciones enteras que estén corruptas o que tengan muchos fragmentos

sin género, y esto acabaría haciendo que las carpetas no tuvieran una cantidad similar de fragmentos. Por ello, para evitar evaluar el modelo de forma desequilibrada se utilizarán estos conjuntos de entrenamiento y evaluación. Una vez terminada la evaluación, se entrenarán los modelos finales con todo el dataset.

4.4. Planificación

En un primer momento, se decidió planificar el proyecto utilizando una metodología iterativa e incremental. Como el sistema que se va a desarrollar dependerá de la literatura revisada, las fases del proyecto se repetirán en cada iteración.

El objetivo al utilizar este marco de trabajo es que en cada fase se tengan nuevos modelos, y poder realizar mejoras sobre los modelos ya existentes.

Se dividirá el proyecto en iteraciones, cada una con una serie de fases, con las tareas a realizar y una duración estimada.

4.4.1. Cronograma inicial

En un inicio, se estimaron 2 iteraciones repartidas a lo largo del proyecto. Al planificar, el objetivo es que se termine el proyecto para la convocatoria de junio de 2025.

Antes de la planificación de las iteraciones se realizó una fase previa en diciembre de 2024 donde se planteó la metodología y se empezó el estado del arte. Esta fase comenzó una vez adjudicados el tutor y la temática del trabajo, y acabó a comienzos de enero.

La primera iteración serviría para tener un sistema inicial y la segunda serviría como revisión para añadir nuevos modelos o técnicas de evaluación.

En la primera iteración se realizarían estas tareas:

- **Estado del arte:** Se hará una revisión que servirá para las fases posteriores y condicionará tanto el diseño como la construcción del sistema. Esta revisión incluirá fundamentos sobre los sistemas de recomendación y técnicas de machine learning y deep learning utilizadas en estos, dando mayor importancia a aquellas utilizadas en recomendaciones musicales y explorando los límites y las oportunidades de estas.
- **Diseño:** Una vez priorizada la literatura relacionada, se definirá el entorno tecnológico, las herramientas y librerías a utilizar y la arquitectura del sistema de recomendación.
- **Implementación:** Se implementarán y evaluarán los modelos del sistema diseñado. La implementación incluirá tanto la obtención y preprocesamiento del dataset como el entrenamiento de los modelos. Para la evaluación se deberá utilizar métricas adecuadas que reflejen el rendimiento de las recomendaciones.

Como la segunda iteración serviría como revisión, será una iteración más corta al tener menor grueso de trabajo. Se realizarían las mismas tareas que en la anterior:

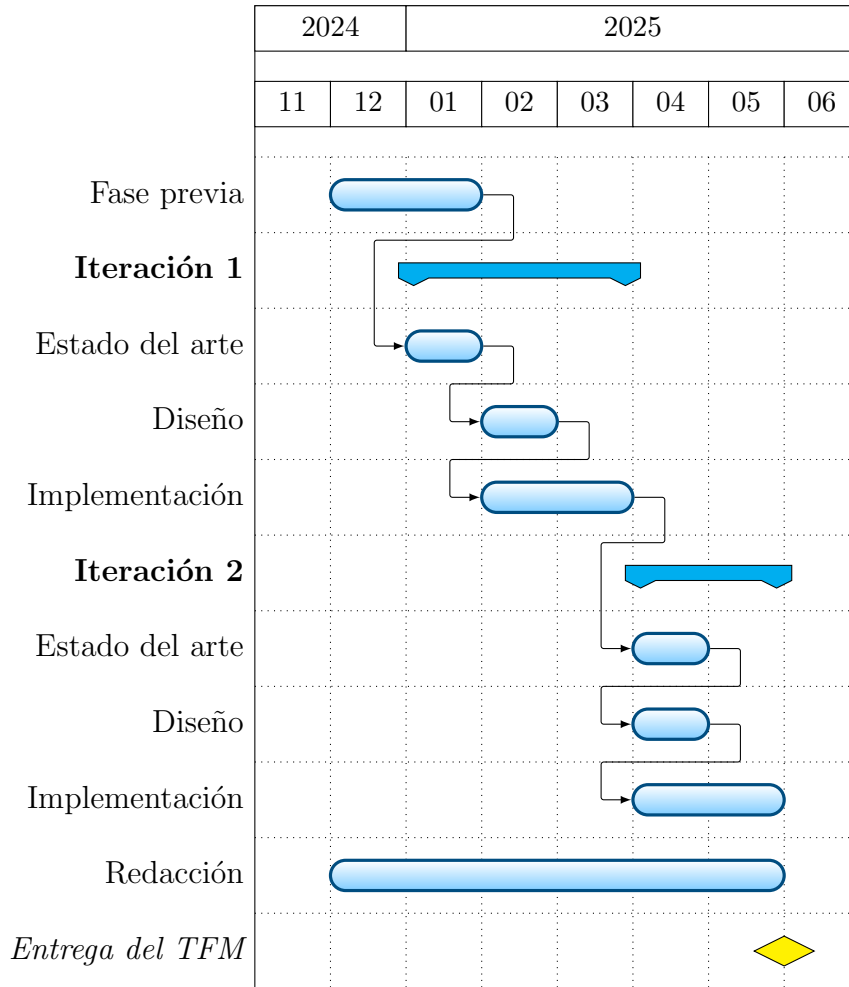
- **Estado del arte:** Se revisará tanto la literatura de la iteración anterior como nueva literatura. Se intentará buscar nuevas técnicas que complementen las ya implementadas para añadir complejidad al sistema y poder hacer más comparaciones del rendimiento del sistema si se ve necesario.
- **Diseño:** Se modificaría el diseño acorde con la nueva literatura.
- **Implementación:** Se construirían y evaluarían los nuevos modelos.

La redacción se haría de forma transversal a lo largo del proyecto, construyendo por secciones el trabajo realizado.

Fase	Período	Estimación de Horas
Fase previa	dic 2024 - ene 2025	30 h
Iteración 1	ene - mar 2025	100 h
Estado del arte	ene 2025	40 h
Diseño	feb 2025	15 h
Implementación	feb - mar 2025	45 h
Iteración 2	abr - may 2025	70 h
Estado del arte	abr 2025	30 h
Diseño	abr 2025	10 h
Implementación	abr - may 2025	30 h
Redacción	dic 2024 - may 2025	100 h
Total		300 h

Cuadro 4.1: Estimación inicial de horas por fase del proyecto

A continuación, podemos ver el diagrama de Gantt inicial:



4.4.2. Cronograma real

Como se comentó en el cronograma inicial, en un principio se realizó una fase previa en diciembre, donde se planteó la metodología y se comenzó el estado del arte.

Una vez terminada dicha fase previa que sirvió para plantear el proyecto de forma general, se empezaron las iteraciones. En cada iteración se llevó a cabo una revisión de la literatura, diseño si procedía e implementación.

Durante la primera iteración, se planeó realizar un sistema de recomendación al estilo de Kostrzewa et al. [28]. Con esta idea en mente, se empezó a implementar la obtención y preprocesamiento de los datos. Sin embargo, cuando se terminó esa implementación, se decidió empezar una iteración nueva porque se consideró que el enfoque de Ulloa [52] podría llegar a ser más interesante.

En esta segunda iteración se realizó de nuevo una revisión de literatura relacionada con técnicas de reducción de dimensionalidad y formas de calcular la similitud entre embeddings. Posteriormente, se diseñó la estructura de los vectores y la arquitectura del sistema. Finalmente, se implementó y evaluó dicha arquitectura.

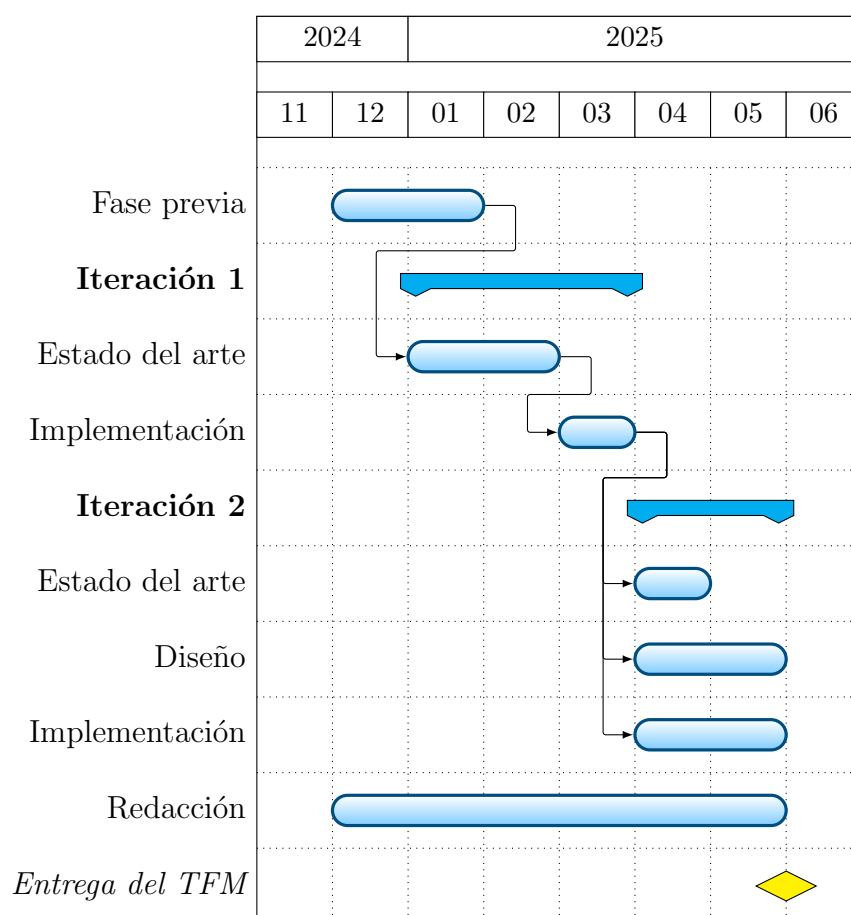
De forma transversal al proyecto, se fue documentando todo el proyecto, escri-

biendo de forma incremental los diferentes apartados del presente documento.

Fase	Período	Estimación de Horas
Fase previa	dic 2024 - ene 2025	15,3 h
Iteración 1	ene - mar 2025	74,6 h
Estado del arte	ene 2025	31,3 h
Implementación	feb - mar 2025	43,3 h
Iteración 2	abr - may 2025	119,6 h
Estado del arte	abr 2025	21,5 h
Diseño	abr 2025	12,8 h
Implementación	abr - may 2025	85,3 h
Redacción	dic 2024 - jun 2025	95 h
Total		304,5 h

Cuadro 4.2: Estimación real de horas por fase del proyecto

A continuación, podemos ver el diagrama de Gantt real:



4.5. Presupuesto

Para la elaboración del presupuesto se han tenido en cuenta diversos factores que representan costes directos e indirectos en el proyecto:

4.5.1. Recursos Intellectuales

El precio de los perfiles informáticos se ha obtenido mirando el sueldo medio de los Data Scientist Junior en España a través de [22] y [47]. Por ello, para las 300 horas estimadas del proyecto se ha estimado un coste por hora de 15 euros, lo que da un coste total de 4.500 euros. Este coste representa el valor del trabajo intelectual invertido en este trabajo.

4.5.2. Recursos materiales

Se ha utilizado un portátil con un coste de 950 euros para todo el desarrollo del proyecto. A este coste, se ha tenido que tener en cuenta el coste de la conexión a Internet y del consumo eléctrico. A su vez, se ha tenido que tener en cuenta los costes derivados del uso de GitHub Pro y el uso de Microsoft 365 incorporados en el plan de estudiante.

Para el consumo se ha mirado el consumo de energía de la batería del portátil Asus Rog Strix que es de 56 W/h, por lo que $\text{consumo} = 56W/h = 0,056kW/h$. En el siguiente cuadro 4.3 podemos ver el presupuesto estimado para el TFM:

Concepto	Detalle	Coste estimado
Horas de trabajo	304,5h x 15€/h	4.567,5€
Amortización de hardware a 3 años	Ordenador portátil gama media-alta (1 año)	316,67€
Suscripción a GitHub Pro	12€/mes x 6 meses	72€
Conexión a Internet	6 meses de fibra (20€/mes)	120€
Almacenamiento en la nube	OneDrive (Microsoft 365: 2€/mes x 6 meses)	12€
Total		5.088,17€

Cuadro 4.3: Presupuesto del proyecto

5. Descripción de la propuesta

5.1. Introducción

En este capítulo explicaremos el sistema a desarrollar y se presentará la propuesta de diseño, sin entrar en detalles técnicos de la implementación.

5.2. Propuesta

El propósito principal del sistema es que sea capaz de recomendar canciones utilizando las características de audio de las canciones. Este sistema sería entonces un sistema basado en contenido (content-based) que aprendiera las relaciones entre las diferentes características extraídas de los audios. Con este sistema, se pretende poder recomendar música sin necesidad de conocer información de usuario o historial de canciones escuchadas, necesarios en sistemas basados en filtrado colaborativo o sistemas híbridos.

5.3. Estructura del sistema de recomendación

Una vez tenemos clara la propuesta del sistema de recomendación que vamos a diseñar e implementar, podemos discutir qué partes tendrá. Como queremos hacer un sistema basado en contenido que utilice las características de audio, podemos trabajar con vectores de baja dimensionalidad aplicando reductores como PCA. Estos vectores serían del tipo 5.1.

$$x \in \mathbb{R}^n \quad (5.1)$$

Sin embargo, PCA utiliza operaciones lineales de todas las dimensiones de los vectores. Ahí es donde entran en juego los Autoencoders (AE o VAE). Estos aprenderían relaciones no lineales entre las características y proporcionarían vectores de menor dimensionalidad. El AE simplemente proporciona vectores latentes que contienen la información mínima de las dimensiones relevantes para ser capaz de reconstruir las entradas a partir de estos vectores, mientras que el VAE trabaja con distribuciones normales, es decir, el espacio latente que origina sigue una distribución probabilística $\mathcal{N}(0, 1)$.

El resultado obtenido del Autoencoder sería de la forma $Z = [z_1, z_2, \dots, z_m]$ donde cada z sería el embedding de cada canción del dataset. Para definir la distancia, tenemos varias opciones:

- **Distancia en el espacio latente (k-NN):** Para una canción query z_q , calcularemos la distancia (euclídea, coseno del ángulo, etc.) contra todas las $z_i \in$

$Z = [z_1, z_2, \dots, z_m]$. La ventaja principal es que es un método sencillo que permite ajustar el número de vecinos k fácilmente, aunque no utiliza la función de similitud óptima para el embedding.

- **Perceptrón multicapa (MLP) que aprenda una función de similitud:** Es un modelo supervisado que aprende si dos canciones son similares o no en función de sus embeddings. El modelo debería dar una puntuación de similitud a partir de dos embeddings z_1 y z_2 . La principal ventaja es que el MLP aprendería una función no lineal de similitud teniendo en cuenta nuestros datos, pero necesitaría entrenar con pares de canciones etiquetadas como similares y alguna heurística para construirlos.
- **Red siamesa:** Una red siamesa permite comparar a la vez varios elementos. En nuestro caso, compararíamos las canciones, minimizando la distancia entre canciones similares y maximizándola entre las no similares.

Este pipeline utilizaría una definición similar de los embeddings de Ulloa [52], concatenando los vectores de las features que queremos utilizar. Para añadir mayor complejidad al sistema, se utilizaría un Autoencoder en lugar de PCA, y se añadirían diferentes estrategias para la similitud. Entre ellas, se utilizaría k -NN como en [43] y se podría llegar a añadir una red siamesa con el objetivo de predecir la similitud entre canciones, similar al de Salas [44].

En la arquitectura final se recortará la adición del MLP y la red siamesa, pues habría muchas combinaciones de modelos que probar para comparar cuáles dan mejores resultados. Además, la solución para calcular la distancia que más se adapta a nuestros modelos sería utilizar k -Nearest Neighbors, pudiendo comparar diferentes funciones de distancia para medir cuál da mejores resultados.

5.3.1. Arquitectura final

Ya hemos discutido los aspectos clave de la estructura principal de nuestro sistema. Este seguirá el flujo visible en la figura 5.1:

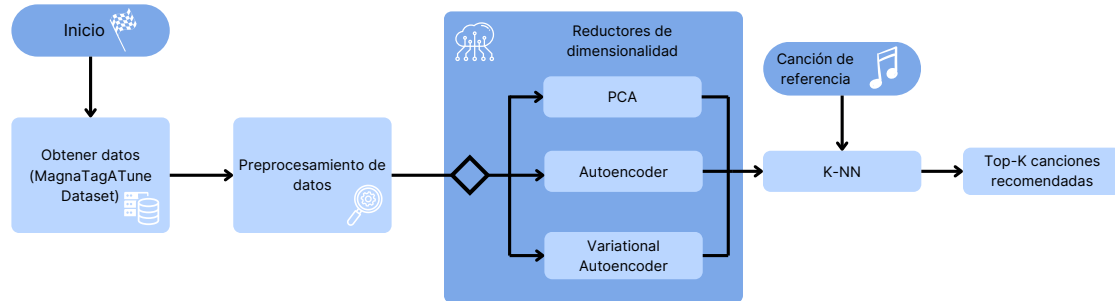


Figura 5.1: Arquitectura del sistema

1. **Preprocesamiento:** Transformamos los datos de los audios del dataset para tener, por una parte, información de los géneros, las etiquetas y otros metadatos; y por la otra, las características de audio extraídas.
2. **Construcción de embeddings:** Construimos para cada fragmento de audio un embedding que tenga toda la información importante relativo a este. Los embedding serán una concatenación de la Short Time Fourier Transform (STFT), los Mel Frequency Cepstral Coefficients (MFCC), los Chroma Coefficients y el Tempo. Esto nos daría vectores de alta dimensionalidad con las propiedades más relevantes de los audios.
3. **Reductores de dimensionalidad:** Para evitar trabajar con vectores demasiado grandes, y que el ruido empeore el rendimiento de las recomendaciones, reducimos la dimensionalidad de los embeddings. Analizaremos el rendimiento de PCA, un Autoencoder (AE) y un Variational Autoencoder (VAE). Los autoencoders aprenderán a comprimir la información obtenida de las entradas en vectores latentes, reduciendo la diferencia entre la entrada y la salida que se obtendría reconstruyendo la entrada a partir del vector latente. El VAE nos dará unos vectores latentes distribuidos de acuerdo con una distribución normal $\mathcal{N}(0, 1)$.
4. **K-NN:** Una vez obtenidos los embeddings reducidos utilizamos k-NN para recomendar las k canciones más cercanas a una dada según una métrica de distancia como la distancia euclídea.

En resumen, el sistema utilizaría aprendizaje no supervisado para comprimir la información al no utilizar etiquetas y dejar que los modelos descubran por sí mismos qué características de audio son importantes. Además, un autoencoder bien entrenado eliminaría aquellas dimensiones redundantes o irrelevantes, reduciendo el ruido que podría empeorar las recomendaciones. Por último, k-NN nos permitiría recomendar de

forma sencilla y efectiva canciones similares entre sí.

5.4. Embeddings

Como se comentó en la sección anterior, tendremos un embedding por cada fragmento del dataset. Se generarán dos embeddings por modelo, uno exclusivamente con características de audio (STFT, Chroma Coefficients, MFCC y tempo) y otro con las características de audio, los géneros y las etiquetas codificadas.

Vamos a explicar qué información representará cada característica, siguiendo las definiciones de las características utilizadas por Ulloa [52]:

- **Short-Time Fourier Transform:** Con la Short-Time Fourier Transform (STFT) queremos convertir la señal de audio $x[n]$ del dominio tiempo al dominio tiempo-frecuencia. Esta se calcula como:

$$\xi(m, k) = \sum_{n=0}^{N-1} x[n + mH] \cdot w[n] \cdot e^{-2j\pi kn/N}$$

Donde $x[n]$ es la señal discreta de audio, $w[n]$ es la ventana, N es el tamaño de la ventana, H es el salto, m es el índice del frame temporal y $k \in [0, N - 1]$ es el índice del bin de frecuencia. Tras esto tomamos el espectrograma elevando al cuadrado

$$\theta(m, k) = |\xi(m, k)|^2$$

donde el eje horizontal representa el tiempo y el eje vertical representa la frecuencia. Una vez tenemos el espectrograma, podemos calcular la media en el tiempo de la energía en cada frecuencia para tener un vector que represente cómo de fuerte es la energía en promedio a lo largo de cada fragmento.

- **Mel Frequency Cepstral Coefficients:** Con los Mel Frequency Cepstral Coefficients (MFCC) podemos medir la forma del espectro de frecuencias en la escala de Mel (escala perceptiva del oído humano). Los coeficientes tienen la siguiente forma:

$$S(n) = \sum_{k=0}^{N-1} |\xi(k)|^2 \cdot J_m(k)$$

Donde $0 \leq m \leq M - 1$, m es el filtro triangular y $J_m(k)$ es el peso de contribución del bin k al filtro triangular m . Es decir, cuánto aporta el bin k al filtro m . Una vez tenemos la energía espectral en la frecuencia de Mel, calculamos la Discrete Cosine Transform (DCT) para obtener un determinado número de coeficientes:

$$\Lambda_m(n) = \sum_{m=0}^{M-1} S(n) \cdot \cos\left(\frac{\pi n(m - 0,5)}{M}\right)$$

Donde $n \in \mathbb{N}$ es el número de MFCC que queremos, en concreto, 13 coeficientes de acuerdo con Ulloa [52]. El último paso sería calcular la media de cada coeficiente en el tiempo.

- **Chroma Coefficients:** Esta característica considera propiedades de la armonía y la melodía de la señal. En la música occidental, la música se divide en 12 tonos o pitches diferentes

$$\{C, C\sharp, D, D\sharp, E, F, F\sharp, G, G\sharp, A, A\sharp, B\},$$

que si traducimos del sistema de notación anglosajón al latino sería

$$\{Do, Do\sharp, Re, Re\sharp, Mi, Fa, Fa\sharp, Sol, Sol\sharp, La, La\sharp, Si\}$$

Esto se conoce como escala cromática, teniendo siete notas naturales y cinco alteraciones, $\sharp$ y $\flat$, que elevan la nota un semitono o la disminuyen, respectivamente. En total, en una octava hay 12 notas posibles, ya que notas como $Do\sharp$ y $Re\flat$ son equivalentes enarmónicamente. Para calcular los coeficientes, se calcula la STFT y se obtiene el espectrograma:

$$Y(n, p) = \sum_{k \in P(p)} |\xi(n, k)|^2$$

Donde $p \in [0 : 127]$ es el número de la nota MIDI, y representa el tono, y $P(p)$ es una función que mapea k a las notas MIDI. Las notas MIDI son una forma de dar una descripción de la altura de una nota a un instrumento musical electrónico, codificando cada nota como un número entre 0 y 127. Esto se puede codificar en 7 bits, y podríamos cubrir 11 octavas.

Una vez hemos obtenido este espectrograma podemos calcular los coeficientes sumando todos los coeficientes que pertenecen al mismo chroma:

$$\Psi(n, c) = \sum_{p \in [0:127]: p \bmod 12 \equiv c} Y(n, p)$$

Siendo $c \in [0 : 11]$. Finalmente calculamos la media temporal para cada nota.

- **Tempo:** El tempo nos indica la velocidad de la canción, es decir, cuantos pulsos suceden por minuto (BPM). Uno de los enfoques para beat tracking es la programación dinámica. Nuestro objetivo es detectar los onsets en la señal que tienen un ritmo regular. Para ello, se utiliza una función objetivo que localiza los beats y otra que penaliza las desviaciones:

$$\Xi(t_i) = \sum_{i=1}^N O(t_i) + \alpha \sum_{i=2}^N F(t_i - t_{i-1}, \tau_p)$$

Donde t_i es la secuencia de N beats, $O(t)$ es la fuerza de onset en el instante t , α es un parámetro para balancear la penalización de F , y τ_p es un tempo ideal. Para construir la mejor secuencia de beats posibles, tomamos la fórmula recursiva:

$$\Xi^*(t) = O(t) + \max_{\tau=0, \dots, t} \{\alpha F(t - \tau, \tau_p) + \Xi^*(\tau)\}$$

Así, para cada valor de t se decide si debería considerarse como beat teniendo en cuenta si hay una alta activación de onset $O(t)$ y si el tiempo desde el beat anterior τ es coherente con el tempo esperado τ_p .

6. Implementación

6.1. Introducción

En este capítulo explicaremos el proceso de investigación y construcción del sistema de recomendación musical. Hablaremos, por una parte, de las herramientas utilizadas, del preprocesamiento y visualización realizados, y de los modelos entrenados, por otra.

6.2. Herramientas

En primer lugar, vamos a comentar el entorno tecnológico utilizado, cómo se ha preparado dicho entorno y las librerías utilizadas.

6.2.1. Entorno de ejecución

El sistema ha sido desarrollado en una máquina con Windows 11 en Visual Studio Code. Los modelos se han implementado utilizando un AMD Ryzen 7 6000 series y una gráfica Nvidia 3060 RTX; es decir, se han implementado teniendo en mente su ejecución en un entorno local.

6.2.2. Preparación del entorno de desarrollo

Antes de empezar a trabajar en la implementación, se tuvo que preparar el entorno de trabajo con todas las configuraciones necesarias. El IDE elegido ha sido Visual Studio Code (VSCoDe) por su simplicidad.

6.2.3. Instalación de dependencias

El lenguaje de programación fue Python 3.11.0. Para aislar las dependencias, también se configuró un entorno virtual. Para ello, se utilizó el siguiente comando:

```
python -m venv myenv
```

Donde `myenv` es el nombre del entorno virtual. Las librerías utilizadas y sus versiones están disponibles en el fichero “requirements.txt” y se pueden instalar con el siguiente comando usando `pip`:

```
pip install -r requirements.txt
```

6.2.4. Librerías utilizadas

A lo largo del proyecto se han utilizado diversas librerías. Para el procesamiento de las señales de audio ha sido clave *librosa*, que tiene herramientas para poder cargar audios, extraer características y poder visualizarlos y reproducirlos.

En cuanto a los modelos, por una parte ha sido importante el uso de *scikit-learn* para construir el modelo k-NN y las métricas. Por otra, *tensorflow* y *keras* han permitido construir y entrenar los Autoencoders y Variational Autoencoders.

Otras librerías útiles han sido *pandas* y *numpy* para trabajar con datos tabulares y vectores; *matplotlib* y *seaborn* para gráficos; *rich* para embellecer la salida en consola; *os* para la manipulación de ficheros y directorios; y *requests* para automatizar la descarga del dataset.

6.3. Preprocesamiento

La primera tarea que se ha tenido que realizar ha sido el preprocesamiento de los datos del dataset. Para la obtención de los datos se desarrolló un módulo llamado “01-download.py” que permite descargar tanto las tres partes del dataset en formato ZIP y descomprimirlas, como descargar los CSV de los metadatos de las canciones. Al ejecutar dicho script se crea una carpeta llamada *dataset* que contendrá toda la información. El objetivo de los siguientes pasos será conseguir almacenar de forma local; es decir, en la propia máquina, tanto los audios de las canciones como la información relativa a los metadatos y las características extraídas en bases de datos SQLite.

A continuación, se va a explicar qué pasos se han seguido para trabajar con los datos. Todo el preprocesamiento está disponible en un módulo llamado “02-database.py”, que preprocesa y crea todas las tablas de nuestra base de datos. Los pasos realizados han sido los siguientes:

1. **Separar etiquetas de géneros:** El CSV que contiene los géneros contiene además diferentes etiquetas que las personas que participaron en el juego de TagATune añadieron, lo que implica que es necesario un buen tratamiento de estas etiquetas. Sabiendo esto, se observó que había muchos géneros y subgéneros mezclados, además de muchas etiquetas que tenían significados muy parecidos. En total teníamos 190 columnas, de las cuales 188 eran booleanas.

Otro problema a tener en cuenta es la dificultad para decir a qué género pertenece una canción, pues no es una propiedad completamente objetiva del sonido, sino una construcción cultural con límites difusos al haber muchas canciones que combinan varios géneros, o que dentro de un género hay muchos subgéneros con diferencias notables. A esto hay que sumarle el hecho de que una canción puede ser de un género diferente para dos personas, lo que para uno es “rock alternativo”, para otra puede ser “indie rock”. Estas dificultades hacen que haya que tomar una buena estrategia para intentar minimizar el impacto de que los géneros asignados no tengan mucha representación en el esquema global o que hayan sido mal impuestas por los participantes de TagATune.

Como solución a este problema, en un principio se separó el CSV en dos BBDD, una para los géneros y otra para las etiquetas. Para los géneros se tuvo que agrupar muchos subgéneros en sus géneros más grandes utilizando la función OR para evitar tener muchos géneros con baja representación, y, además se tuvo que añadir algunas etiquetas que permitirían identificar de mejor forma ese género. También cabe destacar que se agruparon términos con similar semántica. Vamos a pasar a comentar algunos de los géneros cuya fusión ha sido más compleja:

- *Música clásica*: En el caso de la música clásica se fusionaron palabras como “clasical” o “classic” pero también se agrupó el subgénero barroco y las canciones catalogadas como orquestales.
- *Árabe*: Se fusionó con la música de oriente medio, al comprender muchos de los países que dan origen a este género musical.
- *Electrónica*: Se agruparon subgéneros como el techno, el house o el dance.
- *Ópera*: Se separó este género de la música clásica debido a que al incorporar gran cantidad de voces podría tener bastantes diferencias con las piezas musicales plenamente instrumentales. En este caso se fusionaron las óperas masculinas y femeninas, así como las piezas interpretadas por sopranos.

2. Transformar los CSV en BBDD Sqlite: Tras tener los dataframes con la información separada en géneros y etiquetas se guardaron en dos BBDD SQLite diferentes.

De forma transversal, se tuvo que realizar el tratamiento de valores nulos y outliers. Este tratamiento es importante para evitar que haya muchas canciones que añadan ruido a los algoritmos y que impidan un correcto funcionamiento de estos.

Cuando se visualizó si había algún problema con los datos después del procesamiento, se observó que había más de 10.000 fragmentos sin género (ningún género siendo True) y más de 6.000 sin etiqueta. Es decir, que teníamos sobre un 40 % de fragmentos sin género y sobre un 25 % sin etiqueta.

Por lo tanto, había que encontrar una solución a este problema. Las alternativas que se plantearon son las siguientes:

- **Modelo preentrenado**: Utilizar un modelo ya entrenado nos permitiría obtener algunos géneros para todas las canciones, y podríamos desechar el CSV de los géneros. Esto implicaría también que no se utilicen las etiquetas, y por lo tanto, la detección del género tome más peso en el sistema de recomendación. Algunas opciones serían Musicnn¹, que utiliza CNN con espectrogramas, o VGGish² para obtener embeddings de los audios, y entrenar después un SVM o k-NN para predecir los géneros.
- **Web scraping**: Utilizando web scraping podríamos obtener información de las canciones de la base de datos de Magnatune. Sin embargo, probando de forma manual se ha observado que hay álbumes que ya no están disponibles, por lo que esta alternativa queda descartada porque las canciones son poco conocidas y sería

¹Musicnn: <https://github.com/jordipons/musicnn>

²VGGish: <https://www.kaggle.com/models/google/vggish>

difícil encontrar un único sitio web aparte de Magnatune con todas las canciones del dataset.

- **Eliminar canciones sin género:** Aunque drástica, esta opción nos permitiría tener el dataset más limpio, y no arrastraríamos errores como podría pasar con las dos primeras opciones. Es cierto que se perdería un porcentaje considerable de instancias del dataset, pero podría ser una pérdida asumible teniendo en cuenta la cantidad de canciones disponibles.
- **Asignar género mayoritario a las canciones:** Se podría designar para un fragmento, el género mayoritario asignado a otros fragmentos de su misma canción. Además, se podrían plantear técnicas para asignar solamente canciones con una buena representación de géneros en sus canciones.

Finalmente, las opciones elegidas han sido la de asignar el género mayoritario a las canciones sin género y eliminar las que no se puedan asignar. Son las opciones que nos dejarán unos resultados más fiables, ya que cualquier opción tendría un margen de error.

Tras un análisis de los fragmentos de audio se ha visto que hay bastantes canciones que solamente tienen algunos fragmentos sin género, es decir, que la mayoría de las canciones poseen al menos algún fragmento con género asignado. Por lo tanto, también podemos asignar el género mayoritario de los fragmentos de la canción a los fragmentos sin género y borrar las canciones que, tras esta asignación, sigan sin género. La estrategia que se seguirá será asignar el género mayoritario a aquellos fragmentos que pertenezcan a una canción con géneros en al menos el 50 % de sus partes.

Además, aunque no tiene por qué cumplirse siempre, normalmente si varios fragmentos se perciben de un género en concreto, el resto de la canción debería percibirse del mismo modo. Así, si una canción de 9 fragmentos tiene géneros en 7 de estos, y todos son clasificados como clásica, los otros dos los asignaríamos como música clásica también.

Una vez realizadas estas acciones para limpiar el dataset, podríamos utilizar libros para extraer audio features como los MFCC o el tempo. Las características que se han extraído son los Chroma Coefficients, STFT, MFCC y el tempo. Después de todos estos pasos, tendríamos cuatro archivos sqlite, uno con los géneros, otro con las etiquetas, otro con otros metadatos y el último con las audio features.

6.4. Visualización

La siguiente tarea realizada es la visualización, donde podemos observar de forma gráfica los datos para comprenderlos mejor. También ayuda a visualizar outliers o patrones en los datos. Todo el código está disponible en el Notebook de Jupyter “03-visualize”.

En primer lugar, tenemos que saber con cuántos datos contamos después de todo el preprocesamiento. En un principio teníamos 25.863 fragmentos, 50 géneros y 133

etiquetas. Tras el preprocesamiento, nos quedamos con un total de 17.783 fragmentos de audio, 19 géneros diferentes y 76 etiquetas.

Todos estos fragmentos forman un total de 4.825 canciones, de las cuales 2.192 tienen sólo un género, 1.539 tienen dos, 668 tienen tres y 426 tienen más de tres. En la siguiente tabla 6.1 podemos ver cuántas canciones tienen estos géneros. Podemos

Género	Canciones
ambient	882
arabic	148
blues	117
classical	1872
country	316
eastern	235
electronic	1329
folk	563
funk	188
hip hop	73
indian	481
jazz	293
metal	259
opera	531
oriental	130
pop	675
punk	89
reggae	32
rock	981

Cuadro 6.1: Número de canciones por género

observar que el dataset tiene cierto desbalanceo, teniendo canciones con alta representación como la música clásica o la electrónica frente a otros géneros escasos como el punk o el hip hop. Esto se puede ver mejor en la siguiente figura 6.1.

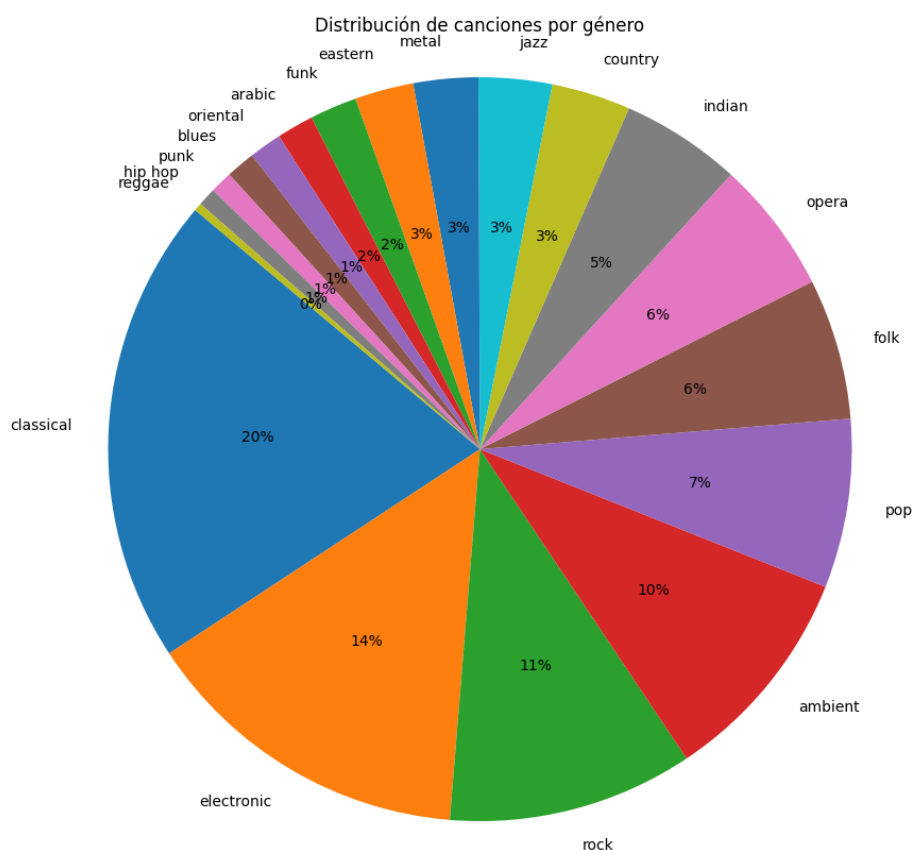


Figura 6.1: Densidad de canciones por género

Podemos observar que la música clásica representa de por sí el 20 % de todas las canciones, mientras que el reggae representa casi el 0 %. Será importante tener esto en cuenta por si hiciera falta eliminar aquellos géneros raros, y con ello, las canciones que se queden sin asignar.

En cuanto a las etiquetas, tenemos 373 canciones con sólo una, 564 con dos, 767 con tres y 3.046 con más de tres. Cabe destacar que hay canciones que pueden no tener ninguna etiqueta, pero no se ha tomado una decisión de eliminar aquellas canciones porque no se ha considerado tan importante como el caso de los géneros.

Para representar esto, y como son tantas las etiquetas que tenemos en nuestro dataset, vamos a visualizar el siguiente diagrama de sectores [6.2](#).

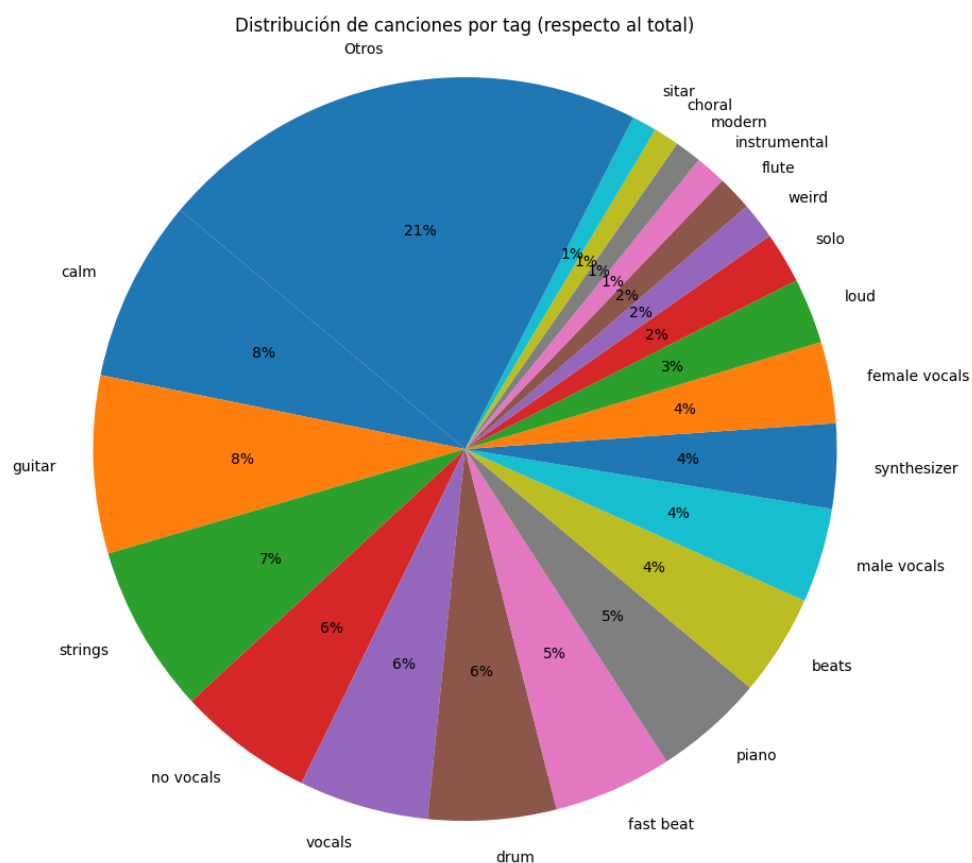


Figura 6.2: Densidad de canciones por etiqueta

Este diagrama representa de forma ordenada las 20 etiquetas más frecuentes en las canciones, y vemos que las más frecuentes serían “calm” o “guitar”, mientras que el 21% restante representaría las 51 etiquetas restantes. Por lo tanto, las etiquetas también sufren un desbalanceo que habrá que estudiar si sería necesario solucionar o se podría aceptar simplemente.

Si en vez de observar las canciones observamos los fragmentos que las conforman, es decir, las filas de nuestro dataset, podemos observar en la figura 6.3 que la proporción de densidad se mantiene.

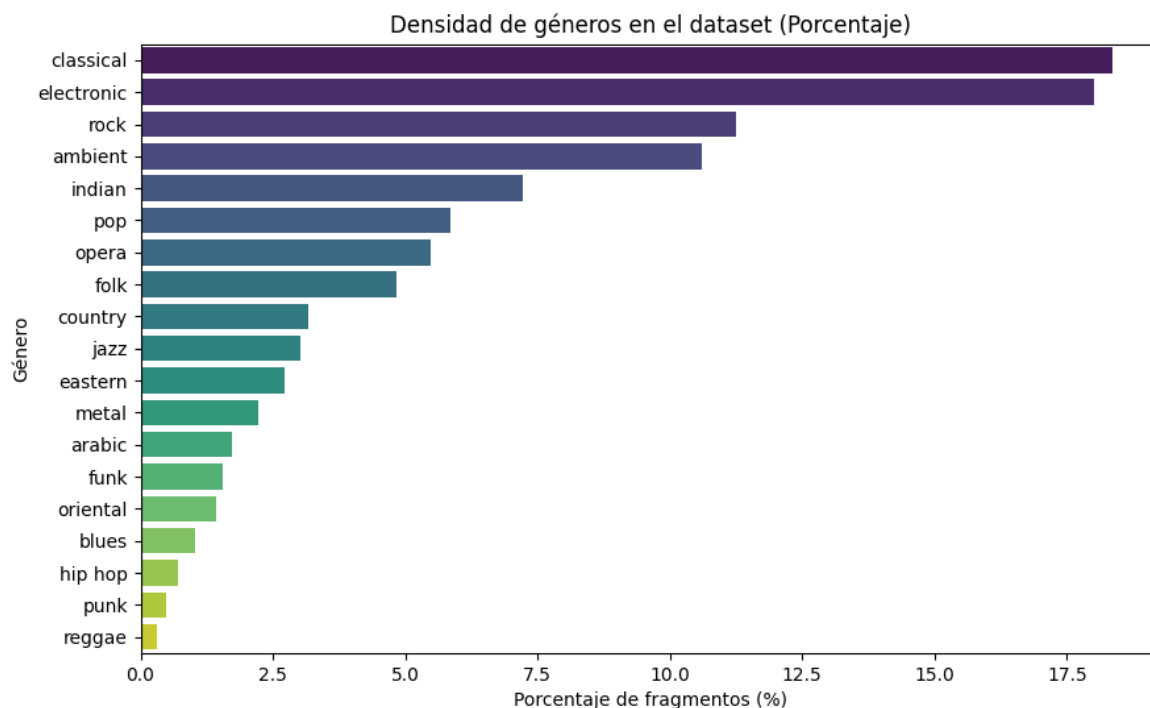


Figura 6.3: Densidad de fragmentos por género

La música clásica y electrónica seguirían siendo los más frecuentes, con un 18 % de representación aproximadamente, y el reggae representa menos del 1 % de los fragmentos.

En cuanto a las etiquetas, sería imposible representar tantas etiquetas, por lo que vamos a comparar las etiquetas más frecuentes con las más raras. En 6.4 podemos observar que hay etiquetas presentes casi en el 0 % del dataset, frente a etiquetas como “guitar” y “calm” (que estaban en el 8 % de canciones del dataset) que están en el 18 % de todo el dataset.

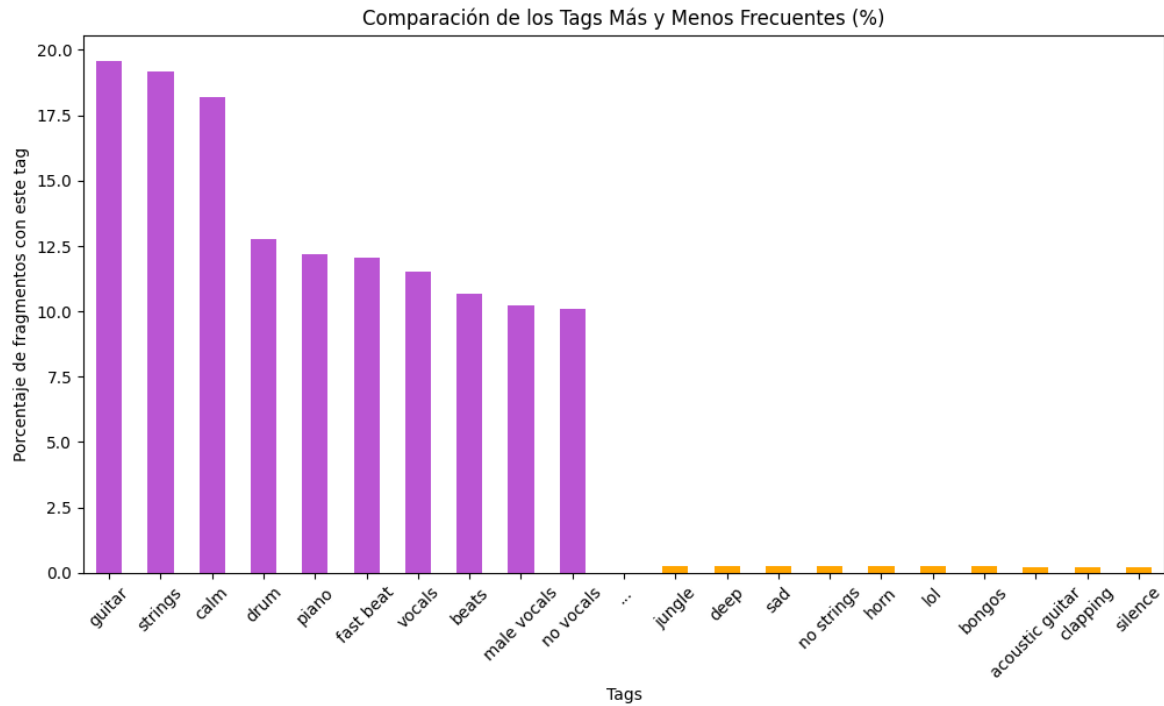


Figura 6.4: Comparación de etiquetas más y menos frecuentes

Vamos a ver la coocurrencia de los géneros en el dataset, es decir, qué géneros suelen aparecer juntos en el dataset. En [6.5](#) se puede ver una matriz cuadrada simétrica, cuya diagonal representa la cantidad de fragmentos que tienen dicho género. Cuanto más claro sea el color, mayor es la coocurrencia entre dos géneros. Teniendo esto en cuenta, podemos ver cómo géneros como ópera y clásica, rock y metal, o ambient y electrónica suelen aparecer juntos.

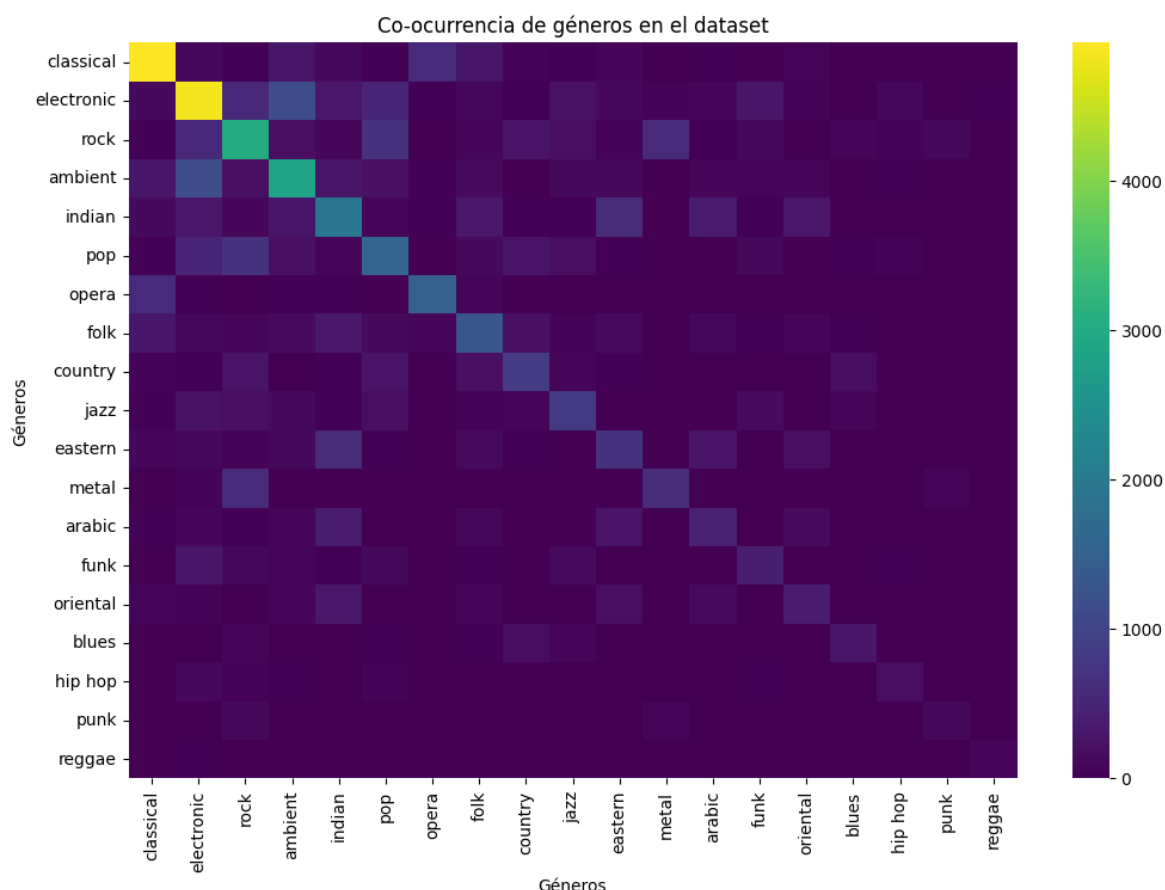


Figura 6.5: Coocurrencia de géneros

A continuación, vamos a visualizar la coocurrencia de las etiquetas, ya que hay etiquetas que tienen sentido que suelen aparecer juntas. En 6.6 podemos observar que las etiquetas que más salen juntas son “guitar” con “strings” y “calm”, “piano” con “strings” que tiene sentido en muchas obras de música clásica y “beats” con “drum”, algo que tiene mucho sentido pues están relacionadas. Además, también hay otras coocurrencias que no tienen tanto peso pero también tienen sentido como la presencia de “guitar” con “electric guitar” o “classical guitar”, y la ausencia de que “guitar” aparezca con “no guitar” que sería una contradicción.

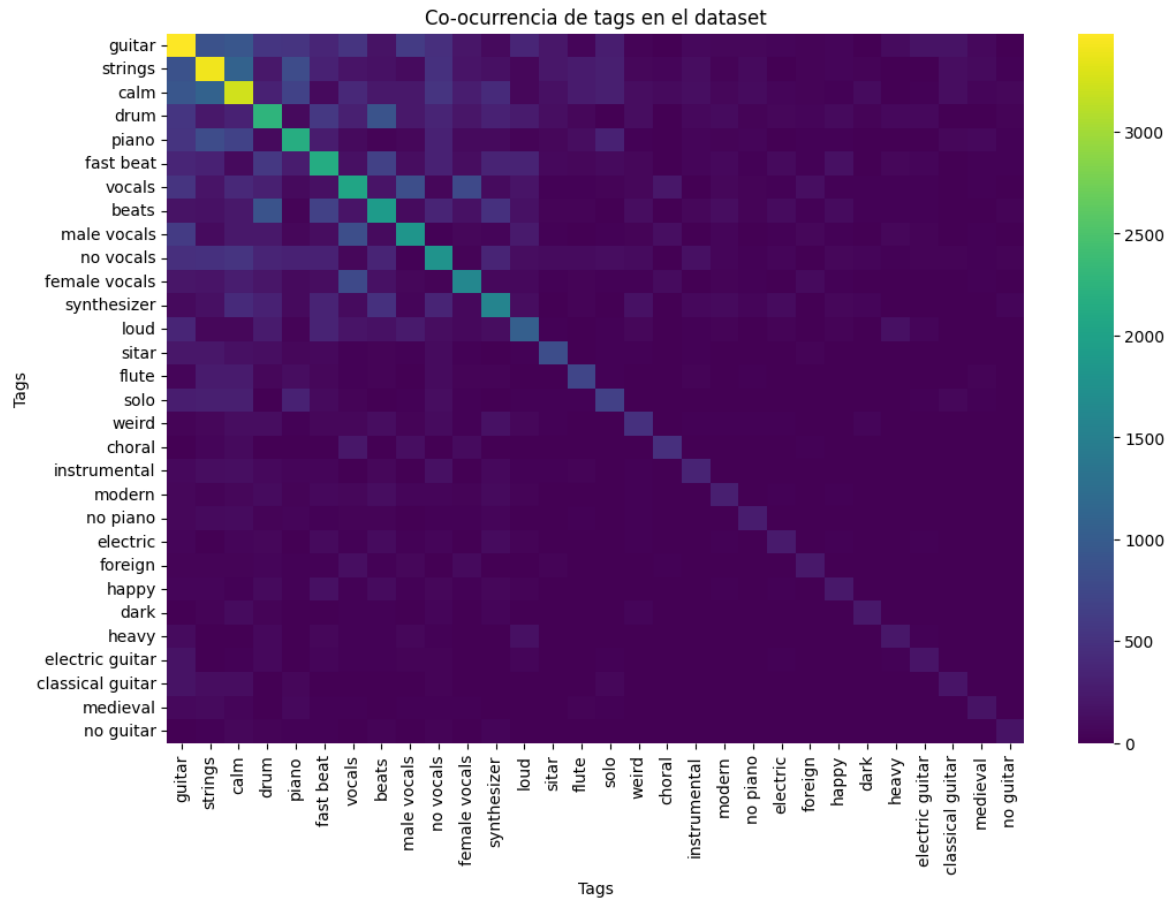


Figura 6.6: Coocurrencia de etiquetas

Finalmente, vamos a analizar la correlación de Pearson entre los géneros y las etiquetas más frecuentes. En concreto, se han elegido 15 géneros y 30 etiquetas por visibilidad de la matriz.

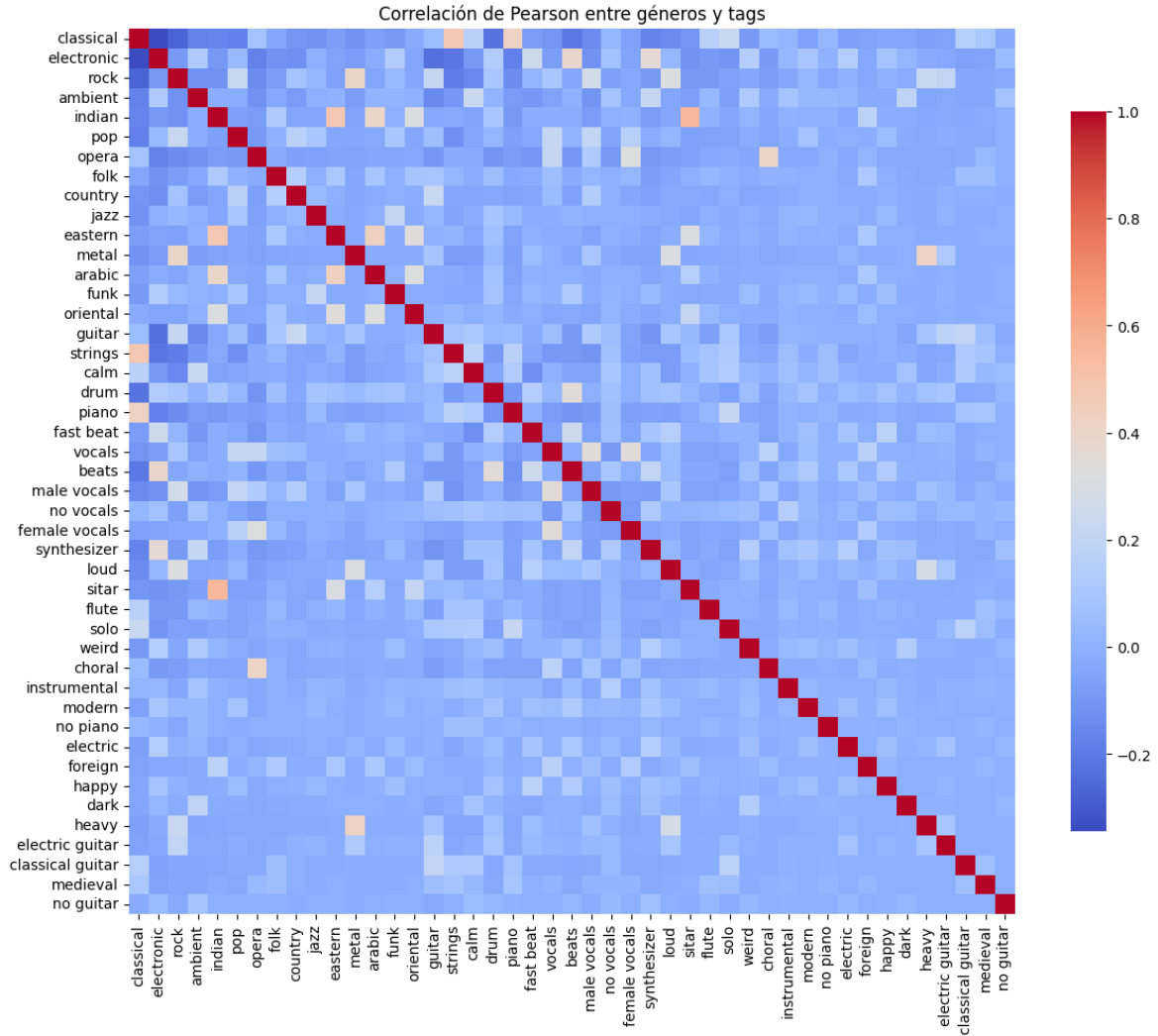


Figura 6.7: Correlación entre géneros y etiquetas

En 6.7 podemos ver que hay etiquetas muy relacionadas con ciertos géneros. Los casos más destacables serían “sitar” con la música india, “strings” y “piano” con la música clásica, “heavy” con el metal o “choral” con la ópera.

6.5. Modelos

Una vez que tenemos el dataset preparado, podemos hablar de la implementación de la estructura que tendrá nuestro sistema de recomendación. Como comentamos en la sección anterior 5, este sistema estará formado por un pipeline donde tendríamos las canciones representadas con un vector numérico del tipo 6.1.

$$x \in \mathbb{R}^n \quad (6.1)$$

Estos vectores tendrían características en diferentes escalas (ej. tempo vs MFCC) por lo que tendríamos que normalizarlos por dimensión. Una vez hecho esto, los vectores entrarían en un Autoencoder (AE o VAE). El encoder maximizaría la compacidad del

vector, minimizando la diferencia entre el vector de entrada y la entrada reconstruida por el decoder. La principal ventaja de utilizar este tipo de redes neuronales frente a técnicas más simples como PCA es que PCA utiliza operaciones lineales, lo que puede llegar a ser una limitación dependiendo de los datos. Los vectores del espacio latente serán los que utilizaremos para decidir la similitud entre las canciones. El resultado obtenido del Autoencoder sería de la forma $Z = [z_1, z_2, \dots, z_m]$ donde cada z sería el embedding de cada canción del dataset.

El primer modelo que se ha construido ha sido un PCA que reduzca la dimensión de los embeddings de entrada a 128 para compararlo con el resto de modelos.

Para la construcción del Autoencoder se ha optado por una estructura sencilla [48], cuyo código 6.1 está disponible en el Notebook “04-models”. El encoder está formado por dos capas: la capa de entrada que recibe el embedding de tamaño `input_dim` y una capa oculta de 512 neuronas con función de activación ReLU, cuya salida pasa a una última capa de 128 neuronas (también con activación ReLU) que generará el vector en el espacio latente y la entrada del decoder. El decoder consta de una capa oculta de 512 neuronas y una capa de salida con activación lineal, que intenta reconstruir la entrada original a partir del vector latente.

```

1 def build_autoencoder(input_dim, latent_dim):
2     """Construye un autoencoder simple"""
3     input_layer = Input(shape=(input_dim,))
4     encoded = Dense(512, activation='relu')(input_layer)
5     encoded = Dense(latent_dim, activation='relu')(encoded)
6
7     decoded = Dense(512, activation='relu')(encoded)
8     decoded = Dense(input_dim, activation='linear')(decoded)
9
10    autoencoder = Model(inputs=input_layer, outputs=decoded)
11    encoder = Model(inputs=input_layer, outputs=encoded)
12
13    autoencoder.compile(optimizer=Adam(learning_rate=0.001), loss='mse')
14    return autoencoder, encoder
15
16 autoencoder, encoder = build_autoencoder(input_dim=X.shape[1], latent_dim
    =128)
17 print(encoder.summary())

```

Extracto de código 6.1: Construcción del autoencoder

En el caso del Variational Autoencoder, se ha tenido en cuenta una arquitectura más compleja que un Autoencoder tradicional, como se puede observar en 6.2. La arquitectura está compuesta por tres partes principales: el encoder, el bloque de sampling (reparametrización) y el decoder.

El encoder comienza con una capa de entrada que recibe el embedding de dimensión `input_dim`. A continuación, una capa densa oculta con 512 neuronas y activación ReLU proyecta los datos a una representación intermedia que se bifurca en dos ramas paralelas:

- Una capa que genera el vector de medias `z_mean` del espacio latente.
- Una capa que produce la varianza de logaritmos `z_log_var` para cada dimensión

latente.

Estas dos salidas definen una distribución gaussiana multivariada por cada entrada. A diferencia de un autoencoder clásico, el VAE no codifica directamente en un punto del espacio latente, sino que aprende una distribución desde la cual se puede muestrear.

Posteriormente, para permitir la retropropagación a través del muestreo estocástico se utiliza la reparametrización. Para ello, definimos una capa llamada **Sampling**, que genera una muestra z mediante:

$$z = \mu + \exp\left(\frac{1}{2} \log \sigma^2\right) \cdot \epsilon$$

donde $\mu = z_{mean}$, $\log \sigma^2 = z_{log_var}$ y $\epsilon \sim \mathcal{N}(0, 1)$ [45]. Esta operación permite el paso del gradiente a través del muestreo.

Finalmente, el decoder recibe como entrada el vector muestreado del espacio latente. A partir de este, intenta reconstruir la entrada original utilizando una capa densa oculta de 512 neuronas con activación ReLU, seguida de una capa de salida con activación sigmoide.

Para la función de pérdida se ha combinado por una parte la pérdida de reconstrucción (reconstruction loss), que mide la similitud de la salida reconstruida con respecto a la entrada original utilizando el Error Cuadrático Medio (MSE), y la pérdida KL (KL divergence), que penaliza la diferencia entre la distribución latente aprendida y una distribución normal estándar [55]. Se calcula como:

$$KL = -\frac{1}{2} \sum (1 + \log \sigma^2 - \mu^2 - \sigma^2)$$

En el entrenamiento se ha utilizado early stopping para evitar sobreajuste y se ha utilizado el optimizador Adam con una tasa de aprendizaje de 0.001 y normalización del gradiente para evitar explosiones de gradiente.

```
1 class Sampling(tf.keras.layers.Layer):
2     """Clase de muestreo para el VAE"""
3     def call(self, inputs):
4         z_mean, z_log_var = inputs
5         batch = tf.shape(z_mean)[0]
6         dim = tf.shape(z_mean)[1]
7         epsilon = tf.random.normal(shape=(batch, dim))
8         return z_mean + tf.exp(0.5 * z_log_var) * epsilon
9
10 def build_vae(input_dim, latent_dim):
11     # Encoder
12     inputs = Input(shape=(input_dim,), name='encoder_input')
13     encoded = Dense(512, activation='relu')(inputs)
14     z_mean = Dense(latent_dim, name='z_mean')(encoded)
15     z_log_var = Dense(latent_dim, name='z_log_var')(encoded)
16     z = Sampling(name='z')([z_mean, z_log_var])
17
18     encoder = Model(inputs, [z_mean, z_log_var, z], name='encoder')
19
20     # Decoder
21     latent_inputs = Input(shape=(latent_dim,), name='z_sampling')
```

```

22     h_decoded = Dense(512, activation='relu')(latent_inputs)
23     outputs = Dense(input_dim, activation='sigmoid')(h_decoded)
24
25     decoder = Model(latent_inputs, outputs, name='decoder')
26
27     # VAE
28     outputs = decoder(encoder(inputs)[2])
29     vae = Model(inputs, outputs, name='vae')
30
31     return vae, encoder, decoder
32
33 def reconstruction_loss(x, x_decoded):
34     return MeanSquaredError()(x, x_decoded)
35
36 def kl_loss(z_mean, z_log_var):
37     return -0.5 * tf.reduce_mean(tf.reduce_sum(1 + z_log_var - tf.square(
38         z_mean) - tf.exp(z_log_var), axis=1))
39
40 def vae_loss(x, x_decoded, z_mean, z_log_var):
41     r_loss = reconstruction_loss(x, x_decoded)
42     k_loss = kl_loss(z_mean, z_log_var)
43     return r_loss + k_loss
44
45 early_stopping = EarlyStopping(monitor='val_loss', patience=5,
46     restore_best_weights=True)
47
48 vae, vae_encoder, vae_decoder = build_vae(input_dim=X.shape[1],
49     latent_dim=128)
50
51 vae.compile(optimizer=Adam(learning_rate=0.001, clipnorm=1.0), loss=
52     lambda x, x_decoded: vae_loss(x, x_decoded, vae_encoder(x)[0],
53         vae_encoder(x)[1]))
54
55 vae.summary()
56 vae.fit(X_train, X_train, epochs=50, batch_size=256, shuffle=True,
57     validation_data=(X_test, X_test), callbacks=[early_stopping])

```

Extracto de código 6.2: Construcción del Variational Autoencoder

Una vez contruidos los modelos, estos se entrenan con el conjunto de entrenamiento y se evalúan con el de pruebas. Tras evaluar los reductores de dimensionalidad, entrenaremos los modelos con todo el dataset y los guardaremos para calcular las recomendaciones.

Para definir la distancia entre vectores latentes, utilizaremos k-Nearest Neighbors. Todo el código está disponible en el Notebook “05-recommender”. En el código 6.3 podemos observar la implementación. La idea principal es buscar como máximo los X fragmentos en el espacio latente cercanos al fragmento de referencia, y a partir de ellos, seleccionar las canciones que corresponden a dichos fragmentos.

Dada una matriz de embeddings de fragmentos de audio, recibimos como entrada el índice en el dataset de un fragmento `fragment-index`. Como algoritmo para k-NN se utiliza un árbol de partición de tipo *ball tree*, eficiente para nuestros vectores de alta

dimensión. Como función de distancia utilizamos Minkowski:

$$D(x, y) = \left(\sum_{i=1}^N |x_i - y_i|^p \right)^{\frac{1}{p}}$$

Cuyo valor por defecto del parámetro p es 2. Esto significa que, por defecto, se está utilizando la distancia euclidiana.

Una vez obtenidos los `search_pool` vecinos más cercanos al fragmento de referencia, filtramos aquellos fragmentos cuya canción a la que pertenecen (`song_id`) coincida con el del fragmento base y guardamos las recomendaciones en un diccionario para mantener la unicidad de las canciones recomendadas, preservando el orden según la distancia. Verdaderamente, `search_pool` no nos puede garantizar que vayamos a obtener `n_songs` únicas porque puede haber fragmentos que pertenezcan a la misma canción y que la canción base se descarte directamente, por lo que si entre los `search_pool` vecinos hay muchas canciones repetidas, no llegaremos a `n_songs`.

```
1 def recommend_songs_by_fragment_knn(fragment_index, embeddings,
2   fragment_to_song, n_songs=5, search_pool=50, n_jobs=6):
3   """
4   Devuelve hasta 'n_songs' diferentes basados en fragmentos similares.
5   - fragment_index: índice del fragmento base en 'embeddings'.
6   - embeddings: matriz de embeddings de fragmentos.
7   - fragment_to_song: lista que mapea fragmento -> canción.
8   - n_songs: cantidad de canciones únicas deseadas.
9   - search_pool: cuántos vecinos buscar inicialmente (se expandirá si
10  no se alcanzan n_songs).
11  """
12
13  knn = NearestNeighbors(n_neighbors=search_pool, algorithm='ball_tree',
14  n_jobs=n_jobs, metric='minkowski')
15  knn.fit(embeddings)
16  distances, indices = knn.kneighbors(embeddings[fragment_index].
17  reshape(1, -1), n_neighbors=search_pool)
18
19  base_song_id = fragment_to_song[fragment_index]
20
21  seen_songs = OrderedDict()
22  for dist, idx in zip(distances[0], indices[0]):
23      song_id = fragment_to_song[idx]
24      if song_id == base_song_id:
25          continue
26      if song_id not in seen_songs:
27          mp3_path = metadata_df.iloc[idx]['mp3_path']
28          seen_songs[song_id] = (mp3_path, dist)
29      if len(seen_songs) >= n_songs:
30          break
31
32  return [(song_id, mp3_path, dist) for song_id, (mp3_path, dist) in
33  seen_songs.items()]
```

Extracto de código 6.3: Recomendación con k-NN

En la siguiente sección 7, compararemos las métricas conseguidas con las diferentes funciones para cada uno de los embeddings obtenidos de los diferentes Autoencoders utilizados.

7. Pruebas

7.1. Introducción

Este capítulo está dedicado a la evaluación tanto de los modelos de reducción de dimensión desarrollados en la sección anterior como del sistema de recomendación.

7.2. Modelos

Para evaluar los diferentes modelos se han utilizado como medidas el Error Cuadrático Medio (MSE), la Raíz del Error Cuadrático Medio (RMSE) y el Error Absoluto Medio (MAE). Para calcular el MSE utilizamos la fórmula 7.1:

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \tilde{y}_i)^2 \quad (7.1)$$

Donde y_i es el valor real, $\tilde{y}_i$ es el valor reconstruido por el modelo y n es el número de valores de entrada. Esta métrica nos permite ver la distorsión entre los embeddings originales y las reconstrucciones. El RMSE se mide como 7.2:

$$RMSE = \sqrt{MSE} \quad (7.2)$$

Al ser la raíz cuadrada del MSE, no penaliza tanto los errores. El MAE utiliza la siguiente fórmula 7.3:

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \tilde{y}_i| \quad (7.3)$$

Donde y_i es el valor real, $\tilde{y}_i$ es el valor reconstruido por el modelo y n es el número de valores de entrada.

En la siguiente tabla 7.1 podemos ver el rendimiento de los diferentes modelos evaluados mediante el MSE. Para cada técnica se han entrenado dos variantes, una utilizando exclusivamente los embeddings a partir de las características de audio seleccionadas y otra (denominada full) que además incorpora información sobre los géneros y etiquetas.

Los mejores resultados los obtienen los modelos PCA y Autoencoder, al tener las menores medias y desviaciones estándar, seguidos de las versiones *full* de ambos modelos. Por otro lado, tanto VAE como VAE full muestran un rendimiento significativamente inferior, con errores medios y desviaciones mucho mayores con respecto al resto de modelos.

Modelo	Media	Desviación
PCA	0.0849	0.4780
PCA full	0.1513	0.6628
Autoencoder	0.0964	0.4772
Autoencoder full	0.1230	0.4471
VAE	0.9573	6.5291
VAE full	0.9637	5.9890

Cuadro 7.1: MSE

A continuación, en la tabla 7.2 se presentan los resultados obtenidos para el RMSE. Al igual que en el caso anterior, comparamos distintas técnicas de reducción de dimensionalidad, tanto en su versión básica como en la versión *full*.

Observamos que los modelos PCA y Autoencoder vuelven a obtener los mejores resultados, con valores medios de RMSE más bajos y menor desviación. En cambio, los modelos VAE y VAE full vuelven a tener un rendimiento bastante inferior, reflejado en una media de error y una desviación muy superiores.

Modelo	Media	Desviación
PCA	0.2000	0.2114
PCA full	0.3096	0.2353
Autoencoder	0.2192	0.2200
Autoencoder full	0.2735	0.2194
VAE	0.6943	0.6902
VAE full	0.7351	0.6506

Cuadro 7.2: RMSE

Finalmente, en la tabla 7.3 vemos los resultados obtenidos para el MAE, siguiendo la misma estructura que las dos tablas anteriores.

Modelo	Media	Desviación
PCA	0.1146	0.1242
PCA full	0.1553	0.1320
Autoencoder	0.1463	0.1423
Autoencoder full	0.1748	0.1413
VAE	0.4940	0.4326
VAE full	0.4792	0.3962

Cuadro 7.3: MAE

Vamos a analizar la varianza explicada para las 128 dimensiones de nuestros embeddings reducidos con PCA y PCA full. La varianza explicada por cada componente principal representa cuánta información (o variabilidad) de los datos originales captura esa componente. Por tanto, una alta varianza acumulada implica que el modelo

estará reteniendo una representación fiel de los datos en menos dimensiones, lo cual es deseable.

Con PCA hemos conseguido retener un 92.70 % de la varianza total del dataset original, mientras que con PCA full un 86.74 %. Esto indica que, en ambos casos, la mayor parte de la información original ha sido conservada en el espacio reducido, aunque la versión full, al incluir más atributos, distribuye la varianza entre más dimensiones, lo que puede indicarnos por qué se ha retenido menor varianza. Podemos observar la evolución de la varianza para diferentes valores de dimensionalidad en la figura 7.1.

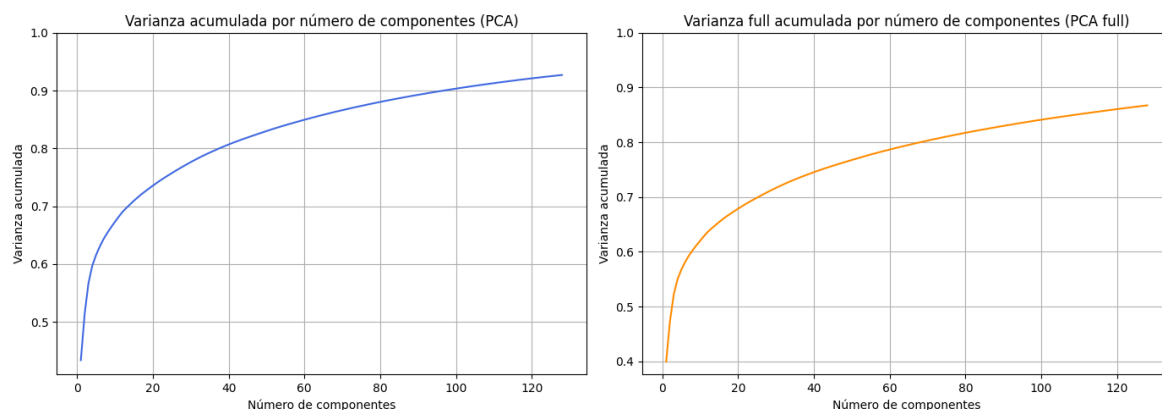


Figura 7.1: Varianza acumulada por número de componentes

Vamos a centrarnos ahora en los autoencoders. Vamos a comparar la distribución de errores de las métricas de los cuadros anteriores para ver dónde hay outliers. En la figura 7.2 observamos diferencias en la distribución de RMSE y de MAE, teniendo una mayor cantidad de instancias con errores cercanos a cero en el autoencoder normal. La mayoría de los errores son cercanos a 0, lo que es un buen signo. Sin embargo, es posible ver algunos outliers en la gráfica del autoencoder full, cuyo valor de RMSE es superior a 2, lo que nos da una indicación de por qué el eje X de la gráfica de MSE tiene valores por encima de 20. En total, hay 41 instancias cuyo RMSE es mayor a 1, lo que nos indica que son posibles outliers.

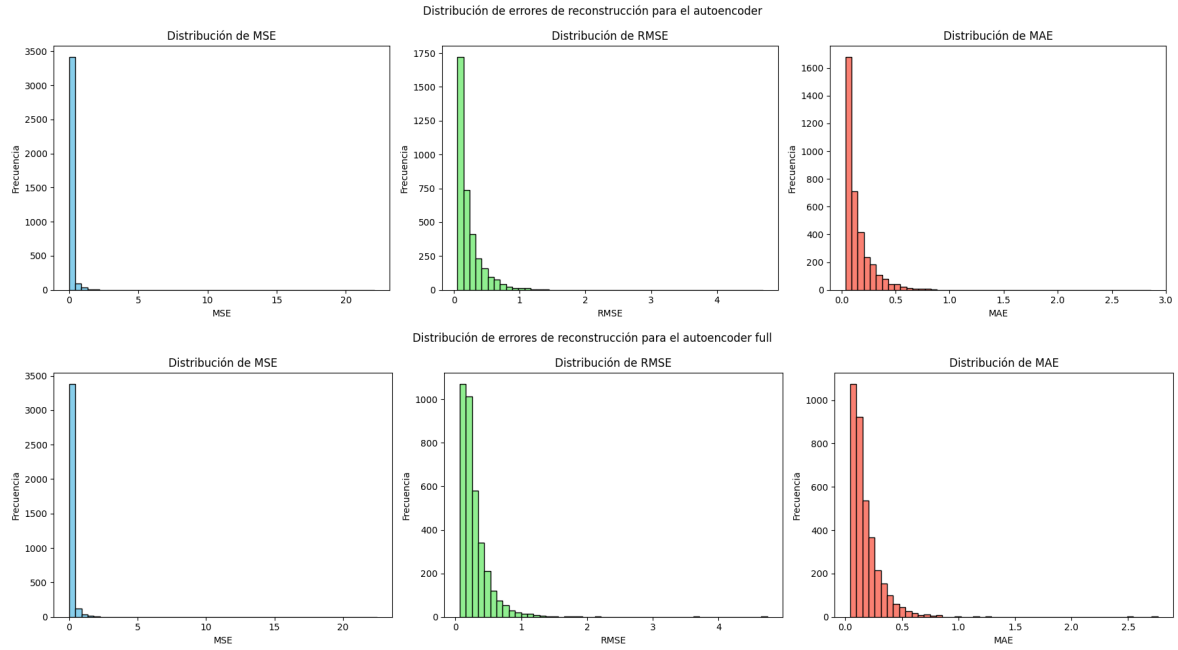


Figura 7.2: Comparación de resultados: (arriba) Autoencoder, (abajo) Autoencoder full

En la figura 7.3 podemos ver la distribución de los errores en los modelos VAE y VAE full. Como ha sucedido en los autoencoders, la mayoría de los errores han sido cercanos a 0, teniendo más instancias con errores mayores. Esto se puede ver en la escala del eje X, teniendo en el caso del RMSE instancias que llegan hasta 17.5 y en el MSE instancias que llegan a errores de 300. Al tener más outliers, es entendible el mal rendimiento observado en los cuadros anteriores.

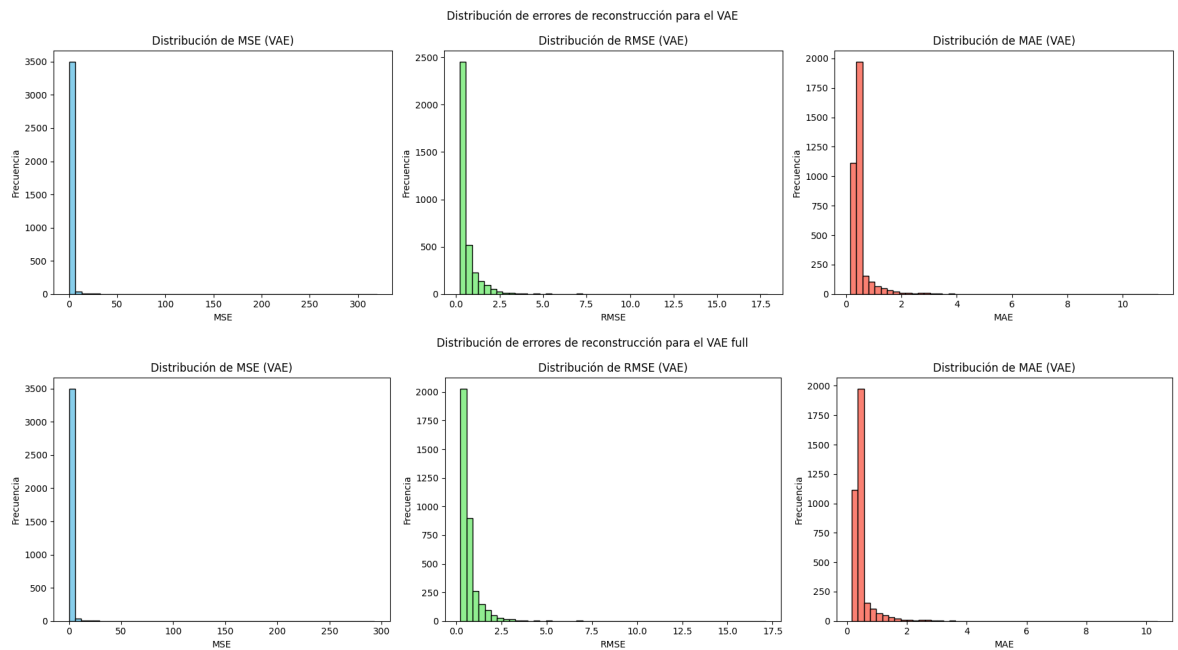


Figura 7.3: Comparación de resultados: (arriba) VAE, (abajo) VAE full

En la siguiente figura 7.4 podemos observar claramente los outliers en las métricas. La métrica que mejor los evidencia es el MSE, donde se aprecia que los outliers de los Autoencoders (AE) tienen una diferencia enorme con los de los Variational Autoencoders (VAE).

Asimismo, tanto en el RMSE como en el MAE, los AE muestran errores muy próximos a cero en prácticamente todas las instancias, mientras que los VAE presentan múltiples casos con errores considerablemente mayores. Esto explica por qué en el MSE, que penaliza más los errores grandes, los outliers del VAE alcanzan valores exponencialmente superiores.

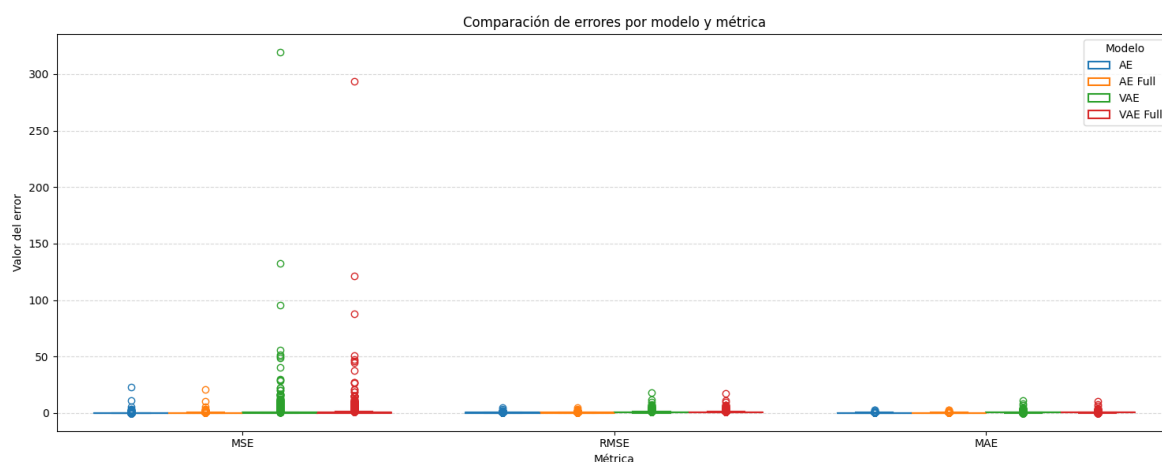


Figura 7.4: Comparación de errores por modelo y métrica

Asimismo, en la figura 7.5 se observa el espacio latente del Autoencoder y el VAE. Se ha coloreado cada fragmento combinando los colores de cada uno de sus géneros. En el espacio del Autoencoder vemos que hay algunos géneros que suelen moverse en ciertas partes del espacio, aunque no es algo estricto porque el género no es una métrica completamente objetiva que nos permita separar las canciones de forma totalmente clara.

En el caso del Variational Autoencoder, al estar regularizado el espacio, vemos que los puntos están centrados sobre el origen y que las canciones de un mismo género están distribuidas por todo el espacio.

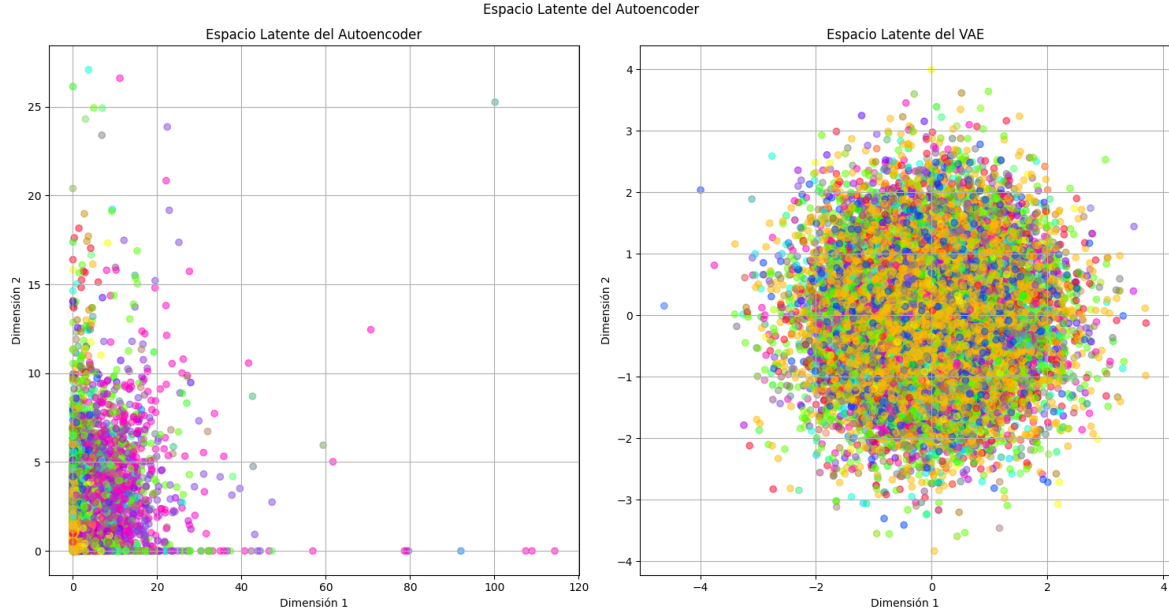


Figura 7.5: Espacio latente: (izquierda) Autoencoder, (derecha) VAE

7.3. Recomendaciones

Para evaluar el rendimiento de las recomendaciones de nuestro sistema, evaluaremos la eficacia de k-NN para recomendar las canciones utilizando los embeddings reducidos con cada una de las técnicas previamente descritas. Como trabajamos con fragmentos de canciones y queremos recomendar canciones, tenemos que evaluar las recomendaciones dando relevancia a que las canciones compartan género. Antes de definir las métricas, definimos:

- Sea G es el conjunto de canciones relevantes.
- Sea $R_k = \{r_1, r_1, \dots, r_k\}$ el conjunto ordenado de las k canciones recomendadas.
- Sea $\mathbb{I}(\cdot)$ la función que determina si una canción es relevante (1) o no (0).
- Sea $\text{rel}(i) = \mathbb{I}(r_i \in G)$ la relevancia binaria de la canción recomendada en la posición i

Las métricas que usaremos son las siguientes [15]:

- **Precision@k:** Proporción de canciones recomendadas entre las k primeras que comparten en su fragmento al menos una etiqueta con el fragmento de la canción base. Mide la precisión inmediata del sistema en las k primeras posiciones del ranking.

$$\text{Precision@k} = \frac{1}{k} \sum_{i=1}^k \mathbb{I}(r_i \in G)$$

Una alta precisión indica que el sistema recomienda bien.

- **Recall@k:** Proporción de canciones relevantes totales que fueron encontradas en las k recomendaciones. Evalúa la capacidad de recuperación del sistema.

$$\text{Recall@k} = \frac{1}{|G|} \sum_{i=1}^k \mathbb{I}(r_i \in G)$$

- **HitRate@k:** Indica si al menos una canción relevante fue recomendada. Es una métrica binaria (0 o 1) por cada consulta.

$$\text{HitRate@k} = \begin{cases} 1, & \text{si } R_k \cap G \neq \emptyset \\ 0, & \text{si } R_k \cap G = \emptyset \end{cases}$$

Donde R_k es el conjunto de canciones recomendadas en el top-k y G es el conjunto de canciones relevantes.

- **F1@k:** Media armónica entre precision y recall. Equilibra ambos factores en una sola métrica.

$$\text{F1@k} = 2 \cdot \frac{\text{Precision@k} \cdot \text{Recall@k}}{\text{Precision@k} + \text{Recall@k}}$$

- **AveragePrecision@k:** Promedio de precisiones acumuladas cuando aparece una canción relevante. Parte de la premisa de que el orden en las recomendaciones importa, por lo que las canciones más relevantes deberían ser las primeras que se recomienden. Penaliza que las relevantes estén muy abajo en el ranking.

$$\text{AveragePrecision@k} = \frac{1}{\min(|G|, k)} \sum_{i=1}^k \text{Precision@i} \cdot \text{rel}(i)$$

Donde:

$$\text{Precision@i} = \frac{1}{i} \sum_{j=1}^i \mathbb{I}(r_j \in G)$$

- **nDCG@k (Normalized Discounted Cumulative Gain):** Esta métrica mide la ganancia acumulada de las canciones relevantes, descontada logarítmicamente según su posición, y la normaliza con respecto al mejor ranking posible. Evalúa tanto la relevancia como la posición dentro del top-k. Primero definimos el DCG:

$$\text{DCG@k} = \sum_{i=1}^k \frac{\text{rel}(i)}{\log_2(i+1)}$$

Luego, definimos el IDCG (Ideal DCG, con todas las canciones relevantes al principio):

$$\text{IDCG@k} = \sum_{i=1}^{\min(|G|, k)} \frac{1}{\log_2(i+1)}$$

Finalmente:

$$\text{nDCG@k} = \frac{\text{DCG@k}}{\text{IDCG@k}}$$

Donde IDCG@k es el DCG ideal (orden perfecto). Una nDCG cercana a 1 indica que las canciones relevantes están en las primeras posiciones del ranking, en el mejor orden posible [2].

A continuación, vamos a estudiar el rendimiento de nuestras recomendaciones para diferentes funciones de distancia.

7.3.1. Distancia euclídea

En la siguiente tabla 7.4 podemos ver las recomendaciones con cada uno de los embeddings reducidos para $k=5$.

Modelo	Precision@5	Recall@5	F1@5	HitRate@5	AP@5	nDCG@5
PCA	0.7026	0.0031	0.0062	0.9374	0.8059	0.6332
PCA full	0.8396	0.0041	0.0081	0.9771	0.9001	0.7616
Autoencoder	0.7087	0.0032	0.0063	0.9389	0.8080	0.6378
Autoencoder full	0.8685	0.0045	0.0088	0.9854	0.9192	0.8010
VAE	0.3145	0.0012	0.0024	0.7803	0.4608	0.2598
VAE full	0.2986	0.0012	0.0023	0.7728	0.4480	0.2433

Cuadro 7.4: Evaluación de las recomendaciones con distancia euclídea

Los embeddings que mejor resultado han dado han sido los del Autoencoder full, que supera al resto de modelos en todas las métricas, incluso ligeramente por encima de PCA full. Asimismo, todos los modelos, con excepción del VAE, mejoran significativamente al usar tanto los géneros como las etiquetas (versión full):

- **PCA full vs PCA:** Hay una mejora clara en Precision, AveragePrecision, nDCG y HitRate.
- **Autoencoder full vs Autoencoder:** La mejora es aún mayor, lo que nos indica que los autoencoders se benefician con más información para aprender a comprimir la información de forma más efectiva.
- **VAE full vs VAE:** Empeora ligeramente, y los resultados son muy bajos con respecto a los otros métodos. Este mal rendimiento puede deberse a que el enfoque probabilístico del VAE introduce ruido que afecta negativamente a la calidad del espacio latente, o a que se ha utilizado una mala configuración de la arquitectura.

Por otro lado, en todos los casos se ve un Recall@5 muy bajo, lo cual es esperable dado que hay una gran cantidad de canciones relevantes posibles, mientras que sólo evaluamos 5 recomendaciones. Por lo tanto, esta métrica será baja, y no nos aportará mucha información teniendo el resto de métricas. En consecuencia, el F1@5 también será bajo, aunque Precision@5 sea muy alta.

También vamos a ver la evolución de estas métricas para valores de $k \in [1 : 10]$. En 7.6 vemos cómo el HitRate y el F1 aumentan con mayores valores de k , mientras que el resto de métricas se queda estables o descienden. También observamos que PCA full está mejor entrenado, mostrando una mayor estabilización y valores más altos para todas las métricas.

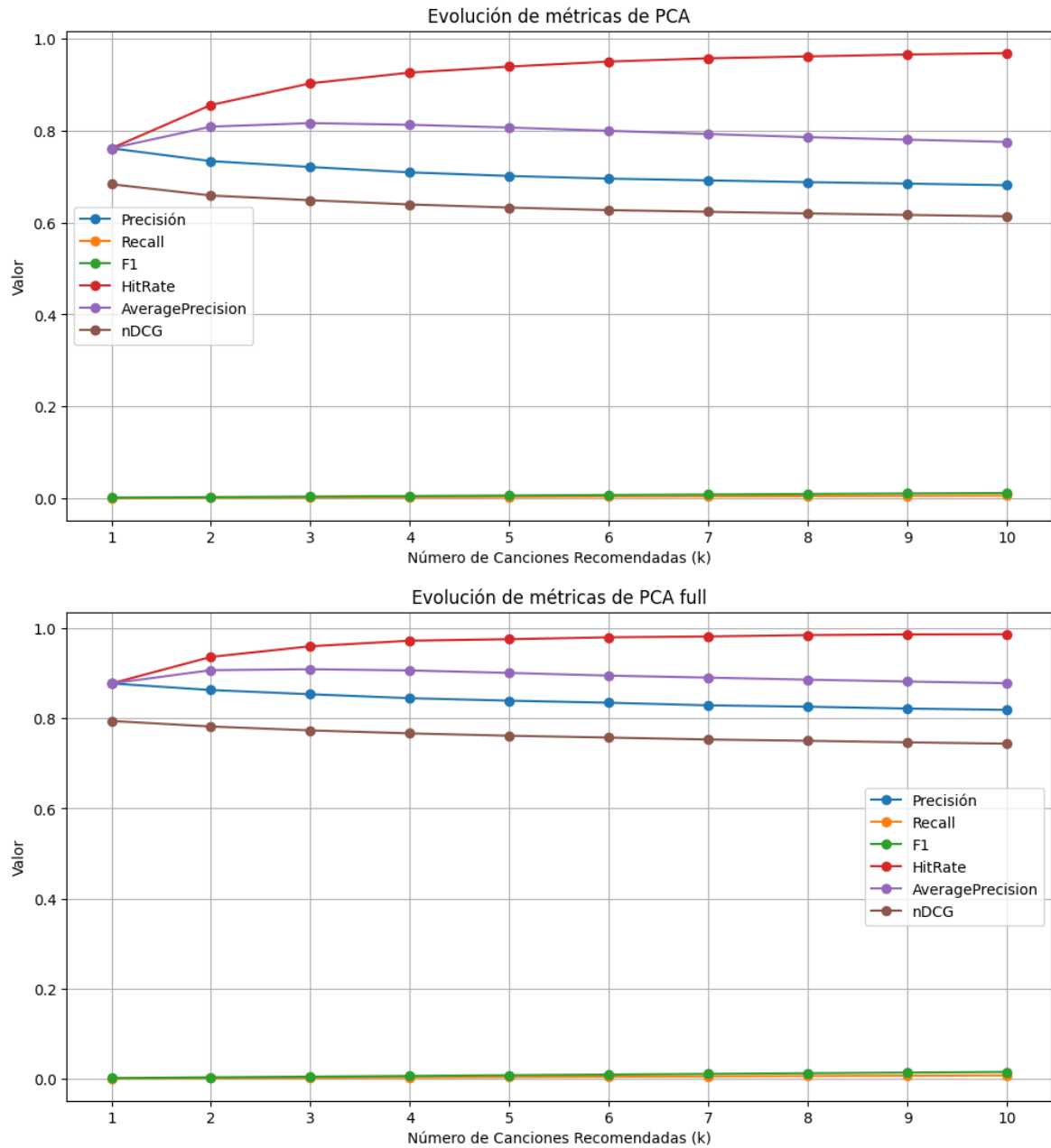


Figura 7.6: Comparación de recomendaciones con distancia euclídea: (arriba) PCA, (abajo) PCA full

En la figura 7.7 sucede casi lo mismo que en el caso anterior. Como se comentó anteriormente, las versiones full de los modelos dan mejores rendimientos que sus versiones estándar.

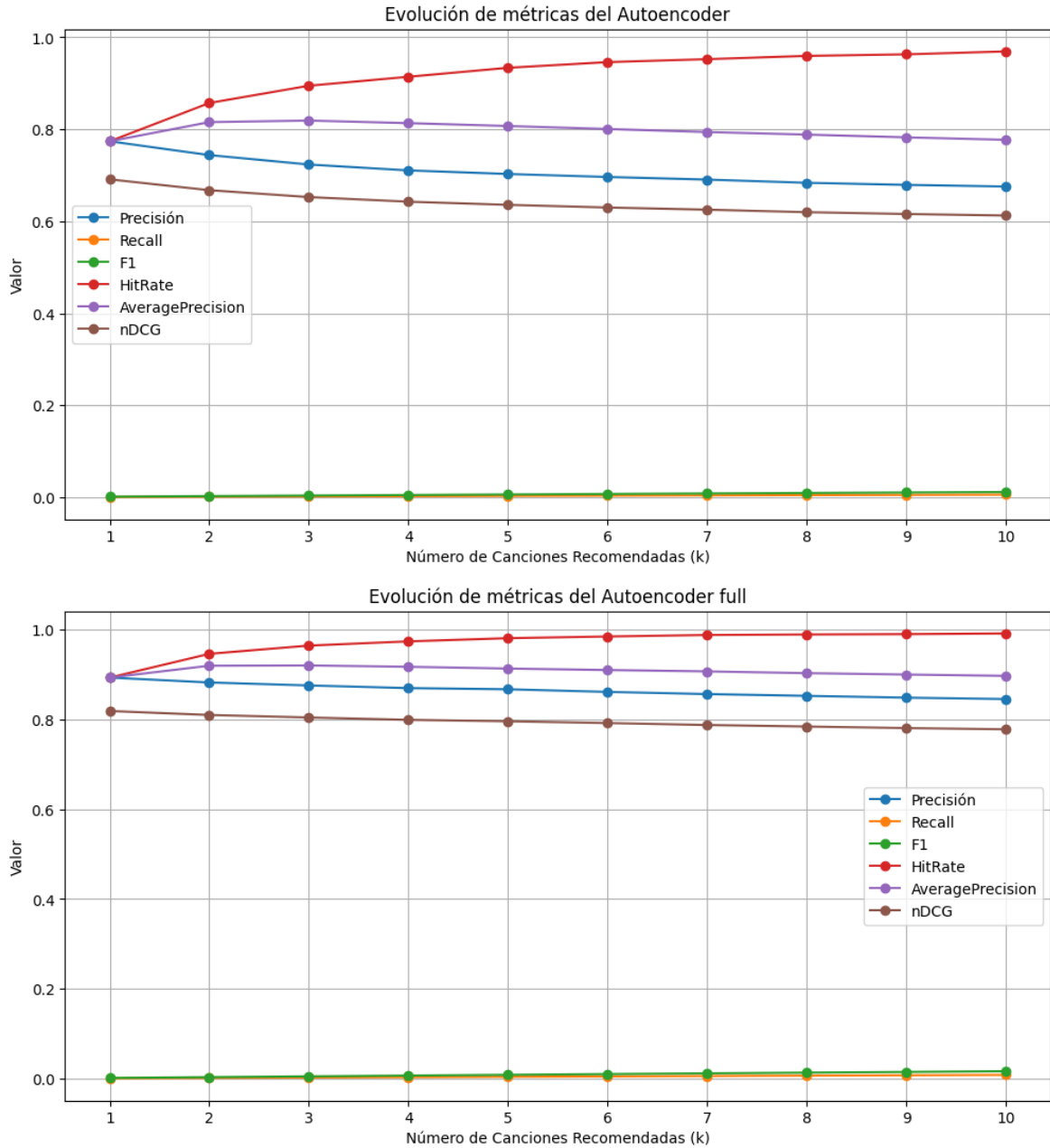


Figura 7.7: Comparación de recomendaciones con distancia euclídea: (arriba) Autoencoder, (abajo) Autoencoder full

Finalmente, en el caso del VAE en la figura 7.8, observamos una curva muy pronunciada en el caso del HitRate, llegando a $\sim 0,93$ en $k = 10$ lo que nos indica que al menos una canción relevante suele aparecer en el top-k. Sin embargo, en los modelos anteriores se alcanzan valores cercanos a 1 mucho antes. Asimismo, vemos cómo la Precision y el nDCG se mantienen bajos y casi estables a lo largo de la curva, lo que nos indica que el VAE no está priorizando correctamente las canciones más relevantes en las primeras posiciones del ranking.

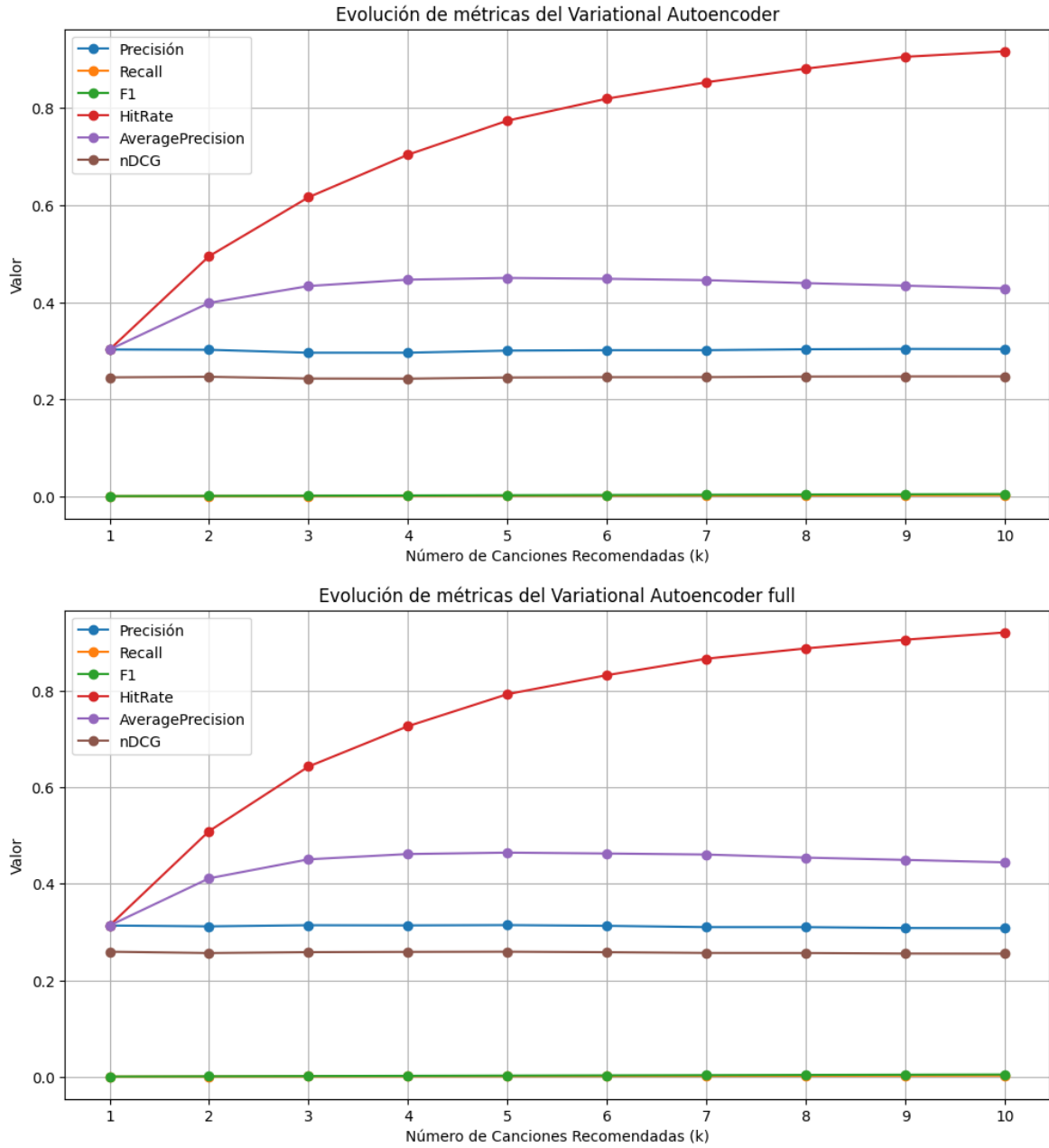


Figura 7.8: Comparación de recomendaciones con distancia euclídea: (arriba) VAE, (abajo) VAE full

7.3.2. Distancia Manhattan

En la siguiente tabla 7.5 podemos ver las recomendaciones con cada uno de los embeddings reducidos para $k = 5$. Vemos que los resultados son muy similares a los obtenidos con la distancia euclídea, teniendo un leve aumento en general en PCA full, Autoencoder y Autoencoder full.

Modelo	Precision@5	Recall@5	F1@5	HitRate@5	AP@5	nDCG@5
PCA	0.6920	0.0031	0.0061	0.9310	0.7960	0.6264
PCA full	0.8469	0.0042	0.0083	0.9816	0.9067	0.7724
Autoencoder	0.7036	0.0032	0.0063	0.9396	0.8090	0.6350
Autoencoder full	0.8658	0.0045	0.0088	0.9846	0.9171	0.7992
VAE	0.3110	0.0012	0.0024	0.7840	0.4652	0.2565
VAE full	0.3027	0.0012	0.0024	0.7897	0.4590	0.2478

Cuadro 7.5: Evaluación de las recomendaciones con distancia Manhattan

Como las tablas son muy parecidas, mantenemos que las versiones full de los modelos aprenden mejor a construir unos embeddings robustos y que el Recall@5 es una métrica que no nos aporta mucha información porque lo normal será que tome valores muy cercanos a 0.

También vamos a ver la evolución de estas métricas para valores de $k \in [1 : 10]$. En las figuras 7.9, 7.10 y 7.11 no se aprecian diferencias significativas con respecto a las gráficas obtenidas con distancia euclídea.

Esto nos sugiere, por una parte, que el espacio latente aprendido es robusto a pequeñas variaciones en la métrica de similitud. Por otra, la distancia Manhattan tiende a ser más tolerante a valores atípicos y menos sensible a grandes diferencias en pocas dimensiones, lo que podría explicar la pequeña mejora observada en algunos modelos.

En consecuencia, parece que los modelos con mejor desempeño (como el Autoencoder full) están siendo capaces de representar relaciones relevantes entre los embeddings de forma efectiva, independientemente de la métrica utilizada.

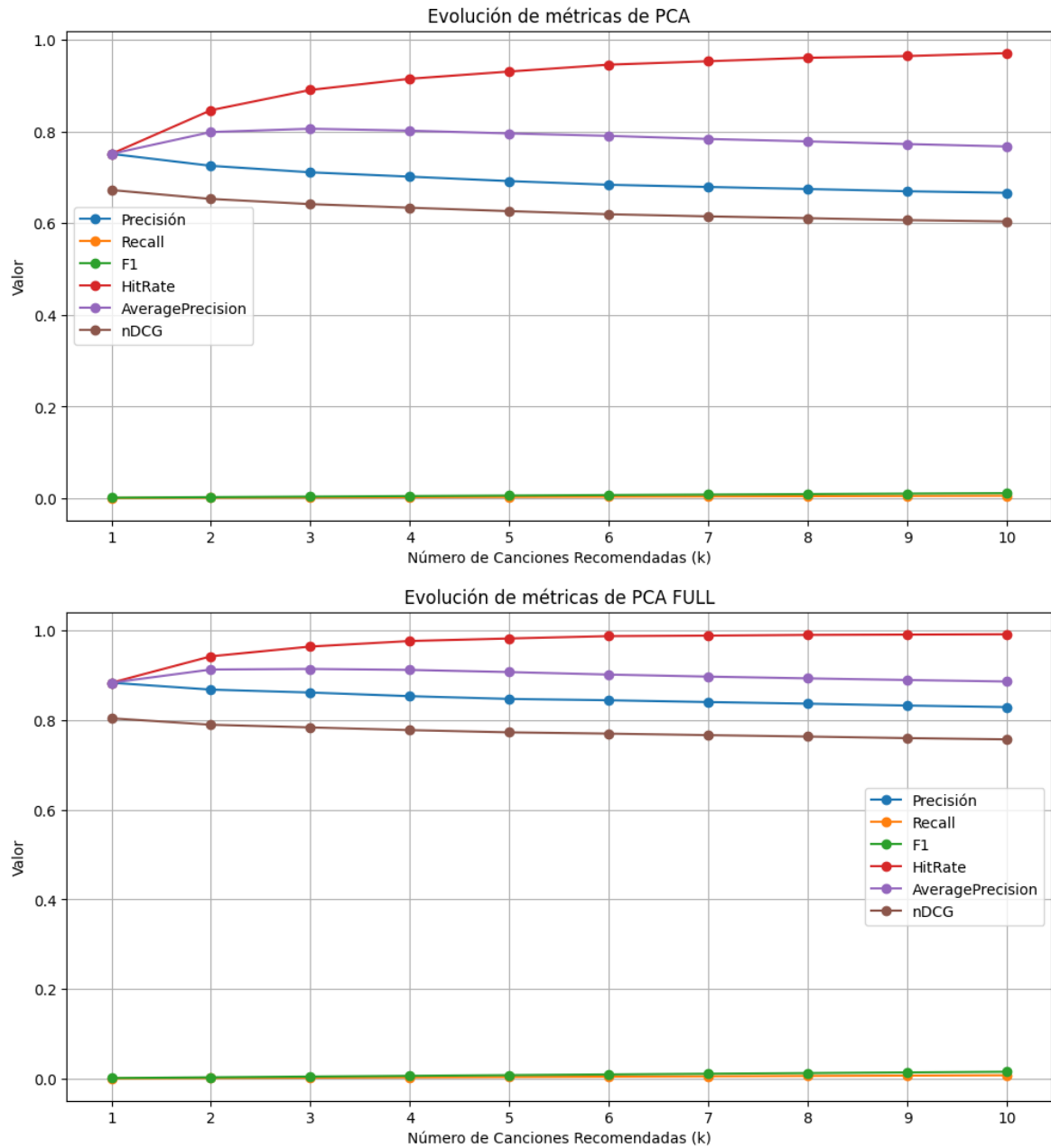


Figura 7.9: Comparación de recomendaciones con distancia Manhattan: (arriba) PCA, (abajo) PCA full

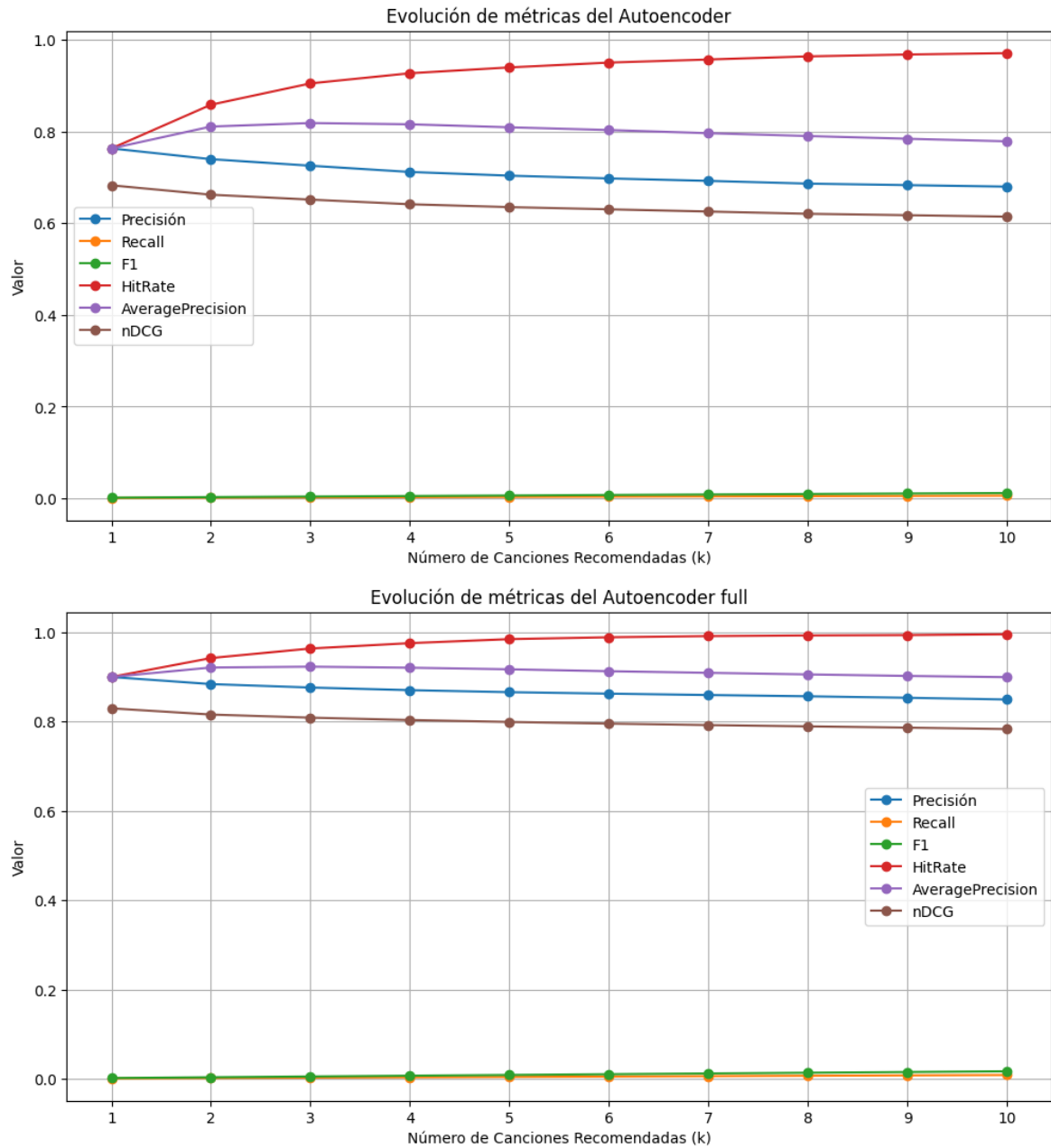


Figura 7.10: Comparación de recomendaciones con distancia Manhattan: (arriba) Autoencoder, (abajo) Autoencoder full

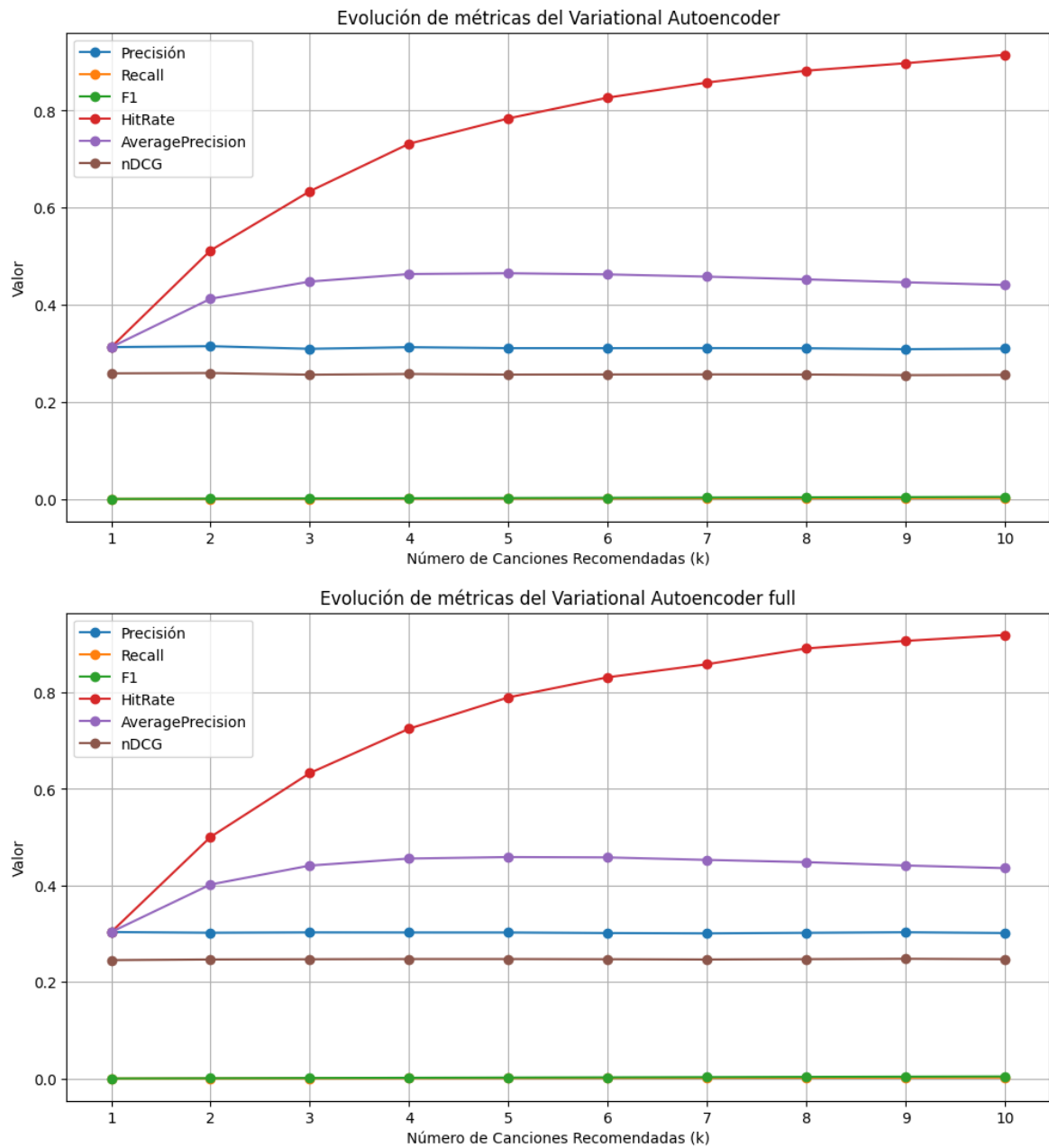


Figura 7.11: Comparación de recomendaciones con distancia Manhattan: (arriba) VAE, (abajo) VAE full

8. Conclusiones

En este proyecto hemos conseguido implementar un sistema de recomendación basado en características de audio y metadatos, codificados en embeddings. Para resolver el problema de la alta dimensionalidad de dichos embeddings, hemos implementado PCA, Autoencoders y Variational Autoencoders, siendo los dos primeros los que ofrecieron mejores resultados.

A la hora de comprimir la información, los que mejores resultados han dado han sido el PCA y el Autoencoder entrenados con las características de audio.

No obstante, al evaluar la calidad de las recomendaciones generadas con k-NN sobre el espacio latente, se observó que el Autoencoder que combinaba tanto las características como los metadatos de las canciones tuvo los mejores resultados en todas las métricas evaluadas. Esto sugiere que aportar más información a los Autoencoders mejora el espacio latente generado y, en consecuencia, la calidad de las recomendaciones.

Por el contrario, los Variational Autoencoders han dado un resultado deficiente. Este resultado podría deberse a diversos factores, como la complejidad del modelo, que el modelo no se haya adaptado a los datos utilizados, o una mala configuración de los hiperparámetros.

Cabe destacar que estos resultados de las recomendaciones se han obtenido considerando que un fragmento de una canción es relevante si comparte al menos un género con el fragmento de referencia. Por lo tanto, cambiar el criterio de relevancia podría modificar las métricas.

Como puntos de mejora, se proponen los siguientes:

- Evaluar el sistema con otros de los datasets que se consideraron al principio del proyecto como GTZAN o FMA, para visualizar si el rendimiento variaría drásticamente.
- Añadir mecanismos para solucionar el desbalanceo de los géneros en el dataset, eliminando los géneros con menor presencia o aplicando técnicas de *under-sampling*.
- Implementar modelos descartados para la similitud entre canciones como el MLP o la red siamesa, comparando el rendimiento del k-NN ya implementado.
- Cambiar el criterio de relevancia. Un criterio como que una canción es relevante para recomendar si tiene tanto algún género o alguna etiqueta en común podría cambiar de forma notoria las métricas.
- Implementar en un sistema real el sistema de recomendación diseñado y analizar la evolución del rendimiento conforme aumenta el tamaño del dataset al añadir nuevas canciones.

En conclusión, el uso de Autoencoders ha sido muy satisfactorio a la hora de reducir la dimensionalidad de los embeddings y mejorar la calidad de las recomendaciones musicales.

A. Bibliografía

- [1] Moataz Ahmed, Sherif Fadel, Manal Helal, and Abdel Moneim Wahdan. Arabic music genre identification. *Journal of Advanced Research in Applied Sciences and Engineering Technology*, 2024. URL <https://doi.org/10.37934/araset.46.1.187200>.
- [2] Evidently AI. Normalized discounted cumulative gain (ndcg) explained, 2023. URL <https://www.evidentlyai.com/ranking-metrics/ndcg-metric>.
- [3] Archi Alpízar. Melodía y duración, mayo 2015. URL <https://archialpizar.wordpress.com/2015/05/27/melodia-y-duracion/>.
- [4] José Alvarado-García, Janet Hernández-García, Esaú Villatoro-Tello, Gabriela Ramirez-de-la Rosa, and Christian Sánchez-Sánchez. Sistema de recomendación de música basado en aprendizaje semi-supervisado. *Research in Computing Science*, 94:97–109, 12 2015. doi: 10.13053/rcs-94-1-8.
- [5] Daniel P. W. Ellis and. Beat tracking by dynamic programming. *Journal of New Music Research*, 36(1):51–60, 2007. doi: 10.1080/09298210701653344.
- [6] Aprende Machine Learning. Comprende principal component analysis (pca), 2018. URL <https://www.aprendemachinelearning.com/comprende-principal-component-analysis/>.
- [7] Rahul Awat. garbage in, garbage out (gigo), 2023. URL <https://www.techtarget.com/searchsoftwarequality/definition/garbage-in-garbage-out>.
- [8] Subhranshu Behura, Arham Alam, Nishtha Phutela, Atul Mishra, and Goldie Gabrani. Tune into your feelings: Nlp-powered emotion driven music recommender system. In Amita Dev, Arun Sharma, S. S. Agrawal, and Ritu Rani, editors, *Artificial Intelligence and Speech Technology*, pages 426–439, Cham, 2025. Springer Nature Switzerland. URL https://doi.org/10.1007/978-3-031-75164-6_32.
- [9] Ruixin Chen, Jianping Fan, Meiqin Wu, and Sining Ma. Conditional diffusion model for recommender systems. *Neural Networks*, 185:107204, 2025. ISSN 0893-6080. doi: <https://doi.org/10.1016/j.neunet.2025.107204>. URL <https://www.sciencedirect.com/science/article/pii/S0893608025000838>.
- [10] Cifra Club. Teoría musical para principiantes: armonía, melodía y ritmo, s.f. URL <https://www.cifraclub.com/blog/teoria-musical-para-principiantes-armonia-melodia-ritmo/>.
- [11] Michaël Defferrard, Kirell Benzi, Pierre Vandergheynst, and Xavier Bresson. Fma: A dataset for music analysis, 2017. URL <https://arxiv.org/abs/1612.01840>.

- [12] Divy Dwivedi, Ashutosh Ganguly, and V.V. Haragopal. 6 - contrast between simple and complex classification algorithms. In Tilottama Goswami and G.R. Sinha, editors, *Statistical Modeling in Machine Learning*, pages 93–110. Academic Press, 2023. ISBN 978-0-323-91776-6. doi: <https://doi.org/10.1016/B978-0-323-91776-6.00016-6>. URL <https://www.sciencedirect.com/science/article/pii/B9780323917766000166>.
- [13] Elastic. ¿qué es knn?, s.f. URL <https://www.elastic.co/es/what-is/knn>.
- [14] Graph Everywhere. Sistemas de recomendación: qué son, tipos y ejemplos, 2023. URL <https://www.grapheverywhere.com/sistemas-de-recomendacion-que-son-tipos-y-ejemplos/>.
- [15] Evidently AI. Precision@k and recall@k – ranking evaluation metrics, s.f. URL <https://www.evidentlyai.com/ranking-metrics/precision-recall-at-k>.
- [16] Flatiron School. The (data) science behind netflix recommendations, 2021. URL <https://flatironschool.com/blog/science-behind-netflix-recommendations>.
- [17] Foqum. ¿qué es un autoencoder?, s.f.. URL <https://foqum.io/blog/termino/autoencoder/>.
- [18] Foqum. ¿qué es un embedding?, s.f.. URL <https://foqum.io/blog/termino/embedding/>.
- [19] Eclipse Foundation. Eclipse ide: The leading open platform for professional developers, 2025. URL <https://eclipseide.org>.
- [20] Marcos García. El cifrado americano: sistema de notación musical, 2021. URL <https://marcosgarcia.net/el-cifrado-americano-sistema-de-notacion-musical/>.
- [21] Geek Culture. Variational autoencoder (vae), 2021. URL <https://medium.com/geekculture/variational-autoencoder-vae-9b8ce5475f68>.
- [22] Glassdoor. Sueldo: Ingeniero de software, 2025. URL https://www.glassdoor.es/Sueldos/ingeniero-de-software-sueldo-SRCH_K00,21.htm.
- [23] Red Hat. What is ai/ml, and why does it matter to your business?, 2023. URL <https://www.redhat.com/es/blog/what-aiml-and-why-does-it-matter-your-business>.
- [24] IBM. ¿qué es una rnn?, 2024. URL <https://www.ibm.com/es-es/topics/recurrent-neural-networks>.
- [25] IBM. Variational autoencoder (vae), s.f. URL <https://www.ibm.com/es-es/think/topics/variational-autoencoder>.
- [26] Gatan Inc. Nyquist frequency, 2020. URL <https://www.gatan.com/nyquist-frequency>.

- [27] JetBrains. Pycharm: El ide de python para profesionales de datos y web, 2025. URL <https://www.jetbrains.com/pycharm/>.
- [28] Daniel Kostrzewa, Jonatan Chrobak, and Robert Brzeski. Attributes relevance in content-based music recommendation system. *Applied Sciences*, 14(2), 2024. ISSN 2076-3417. doi: 10.3390/app14020855. URL <https://www.mdpi.com/2076-3417/14/2/855>.
- [29] Mark A. Kramer. Nonlinear principal component analysis using autoassociative neural networks. *AIChE Journal*, 37(2):233–243, 1991. doi: <https://doi.org/10.1002/aic.690370209>. URL <https://aiche.onlinelibrary.wiley.com/doi/abs/10.1002/aic.690370209>.
- [30] Bas Larrosa. Spotify algorithm: la guía definitiva, 2023. URL <https://www.larrosa.pro/post/spotify-algorithm-la-gua-definitiva>.
- [31] Edith Law, Kris West, Michael Mandel, Mert Bay, and J. Downie. Evaluation of algorithms using games: The case of music tagging. In *10th International Society for Music Information Retrieval Conference (ISMIR 2009)*, pages 387–392, 01 2009.
- [32] Microsoft. Visual studio code, 2025. URL <https://code.visualstudio.com>.
- [33] Himadri Mukherjee, Matteo Marciano, Ankita Dhar, and Kaushik Roy. Dufcalf: Instilling sentence in computerized song analysis. *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*, 15300 LNAI:277 – 292, 2025. doi: 10.1007/978-3-031-78014-1_21. URL https://www.scopus.com/inward/record.uri?eid=2-s2.0-85210870138&doi=10.1007%2f978-3-031-78014-1_21&partnerID=40&md5=0a9dc31e871292ec0cc9393d1cdb8600.
- [34] Music Tomorrow. How spotify recommendation system works: A complete guide (2022), 2022. URL <https://www.music-tomorrow.com/blog/how-spotify-recommendation-system-works-a-complete-guide-2022>.
- [35] Meinard Müller. Lab course: Short-time fourier transform (stft), 2016. URL https://www.audiolabs-erlangen.de/content/05_fau/professor/00_mueller/02_teaching/2016s_apl/LabCourse_STFT.pdf.
- [36] Meinard Müller. Onset detection, 2023. URL https://www.audiolabs-erlangen.de/resources/MIR/FMP/C6/C6S1_OnsetDetection.html.
- [37] Ndiatenda Ndou, Ritesh Ajoodha, and Ashwini Jadhav. Music genre classification: A review of deep-learning and traditional machine-learning approaches. In *2021 IEEE International IOT, Electronics and Mechatronics Conference (IEM-TRONICS)*, pages 1–6, 2021. doi: 10.1109/IEMTRONICS52119.2021.9422487.
- [38] Neurosnap. Understanding the differences between ai, machine learning, and deep learning, 2023. URL <https://neurosnap.ai/blog/post/understanding-the-differences-between-ai-machine-learning-and-deep-learning/64279cadfeb3e5ca5ba0904a>.

- [39] Han-Saem Park, Ji-Oh Yoo, and Sung-Bae Cho. S.b.: A context-aware music recommendation system using fuzzy bayesian networks with utility theory. In *Fuzzy Systems and Knowledge Discovery*, volume 4223, pages 970–979, 09 2006. ISBN 978-3-540-45916-3. doi: 10.1007/11881599_121.
- [40] Adrián Quijada. Sistema de recomendación musical para ehealth. Master’s thesis, Universitat Oberta de Catalunya, 2020. URL <https://openaccess.uoc.edu/bitstream/10609/120146/9/aquijadagTFM0620memory.pdf>.
- [41] Kel Rodolfo. Spotify y las matemáticas: así es la fórmula de sus recomendaciones, 2023. URL <https://www.linkedin.com/pulse/spotify-y-las-matematicas-as-es-la-frmula-de-sus-rodolfo-kel6f/>.
- [42] Ignacio Rodríguez. Formato midi, 2002. URL https://www.lpi.tel.uva.es/~nacho/docencia/ing_ond_1/trabajos_01_02/formatos_audio_digital/html/midiformat.htm.
- [43] Kapil Saini and Ajmer Singh. A content-based recommender system using stacked lstm and an attention-based autoencoder. *Measurement: Sensors*, 31:100975, 2024. ISSN 2665-9174. doi: <https://doi.org/10.1016/j.measen.2023.100975>. URL <https://www.sciencedirect.com/science/article/pii/S2665917423003112>.
- [44] Nicolás Serrano Salas. Estudio y análisis del uso de redes siamesas en estrategias de recomendación basadas en contenido y de filtrado colaborativo. Master’s thesis, Universidad Autónoma de Madrid, 2023. URL <https://abellogin.github.io/2022/NSS.pdf>.
- [45] Data Science. Difference between autoencoder (ae) and variational autoencoder (vae), 2020. URL <https://medium.com/data-science/difference-between-autoencoder-ae-and-variational-autoencoder-vae-ed7be1c038f2>.
- [46] Jonathon Shlens. The mathematics behind principal component analysis. <https://medium.com/data-science/the-mathematics-behind-principal-component-analysis-fff2d7f4b643>, 2019.
- [47] Talent.com. Salario ingeniero de software en españa, 2025. URL <https://es.talent.com/salary?job=ingeniero+de+software>.
- [48] TensorFlow. Autoencoder, 2024. URL <https://www.tensorflow.org/tutorials/generative/autoencoder?hl=es-419>.
- [49] Tonus México. Escala cromática y construcción de escalas mayores, 2022. URL <https://www.tonus.com.mx/blog/escala-cromatica-y-construccion-escalas-mayores>.
- [50] Viet-Anh Tran, Guillaume Salha-Galvan, Bruno Sguerra, and Romain Hennequin. Transformers meet act-r: Repeat-aware and sequential listening session recommendation, 2024. URL <https://arxiv.org/abs/2408.16578>.

- [51] George Tzanetakis, Georg Essl, and Perry Cook. Automatic musical genre classification of audio signals, 2001. URL <http://ismir2001.ismir.net/pdf/tzanetakis.pdf>.
- [52] Diego Saldaña Ulloa. Music recommendation based on audio fingerprint, 2023. URL <https://arxiv.org/abs/2310.17655>.
- [53] Ultralytics. Clasificación de imágenes con el dataset mnist, s.f. URL <https://docs.ultralytics.com/es/datasets/classify/mnist/>.
- [54] Universidad Virtual de Quilmes. Duración, tono, intensidad y timbre, n.d. URL https://libros.uvq.edu.ar/spm/321_duracin_tono_intensidad_y_timbre.html.
- [55] Charel van Hoof. Learn by example variational autoencoder, 2019. URL <https://www.kaggle.com/code/charel/learn-by-example-variational-autoencoder>.
- [56] Wikipedia. Mfcc, 2024. URL <https://es.wikipedia.org/wiki/MFCC>.
- [57] Feng Zhu, Yan Wang, Chaochao Chen, Jun Zhou, Longfei Li, and Guanfeng Liu. Cross-domain recommendation: Challenges, progress, and prospects. *CoRR*, abs/2103.01696, 2021. URL <https://arxiv.org/abs/2103.01696>.
- [58] Iñaki Úcar. Conceptos musicales: nota, tono, figura y ‘pitch’, 2009. URL <https://www.enchufa2.es/archives/conceptos-musicales-nota-tono-figura-y-pitch.html>.